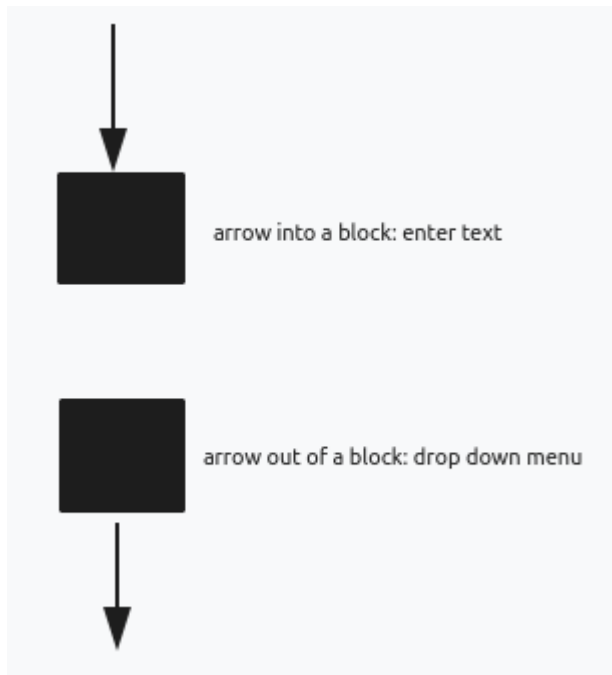


Reference

Legend



In []:

Index

- [Data Science & The Nature of Data](#)
 - [Import a library](#)
 - [Load a dataframe](#)
 - [List rows of data](#)
 - [List columns of data](#)
 - [Filter data](#)
- [Plotting](#)
 - [Generate a scatterplot](#)
 - [Label graph](#)
 - [Generate a bar plot](#)
 - [Group datapoints](#)
 - [Generate a line plot](#)
 - [Generate a histogram](#)
- [Descriptive Statistics](#)
 - [Calculate the mean of a dataset](#)
 - [Calculate the mode of a dataset](#)
 - [Calculate the median of a dataset](#)
 - [Summary of a dataset](#)

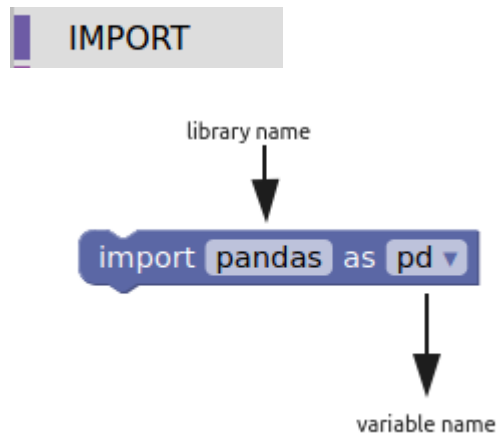
- Get a calculation of a random sample set
- Measures of Association
 - Plot a labeled correlation heatmap
 - Create a contingency table
- Clustering
 - Load data with identifier (ID)
 - Create a dendrogram
 - Plot Clusters
- KNN Classification
 - Split X and Y into train and test sets
 - Define the LDA Model
 - Train the LDA model
 - Predict classification with the LDA model
 - Define the KNN classification model
 - Train the KNN classification model
 - Predict classification with the KNN model
 - Get confusion matrix of a model
 - Get accuracy of a model
 - Get recall and precision of a model
- KNN Regression
 - Create a pipeline with a scaler and KNN regression
 - Train the KNN regression model
 - Predict classification using the KNN model
 - Model scatterplot of observed data
 - Model scatterplot of predictions
- Simple Linear Regression
 - Create linear regression model
 - Train the linear regression model
- Multiple Linear Regression
 - Define the multiple linear regression model
 - Train the multiple linear regression model
 - Get predictions from the multiple linear regression model
 - Check linearity and equal variance
- Logistic Regression
 - Define the Logistic regression model
- Ridge and Lasso Regression
 - Define the logistic regression model with ridge penalty
 - Get predictions from the ridge regression model
 - Train the logistic regression model with lasso penalty
 - Get predictions from the lasso regression model
- Decision tree
 - Build a decision tree
 - Create the decision tree model
 - Fit the decision tree model

- Visualize the decision tree model & interpret algorithm fit -Regression Trees
- Prepare train/test sets
- Fit the model
- Get predictions
- Evaluate the model
- Visualize the model -Random Forest
- Create bagging classifier
- Fit and score bagging classifier
- Get out of bag (OBB) score
- Create random forest classifier
- Fit and score random forest
- Visualize ensemble model
- Gradient Boosting
 - Create gradient boosting classifier
 - Fit the classifier
 - Get predictions
 - Evaluate the model
 - Visualize results
 - Detecting overfitting with deviance
- Support Vector Machines
 - Prepare train/test sets
 - Fit the linear SVM model
 - Predict using the linear SVM model
 - Evaluate the linear SVM model
 - Fit the nonlinear SVM model
 - Predict with the nonlinear SVM model
 - Evaluate the nonlinear SVM model
- Naive Bayes
 - Predict train/test sets
 - Fit the model
 - Get predictions
 - Evaluate the model
- Crossvalidation and Nested crossvalidation
 - Prepare train/test sets with crossvalidation
 - Train model & predict with crossvalidation
 - Evaluate the model
 - Train model & predict with nested crossvalidation
 - Evaluate the model
- Data cleaning, transformations, & versioning
 - Missing data
 - Transforming data
 - Common scale
 - Non-normality
 - Create a new variable

Data Science & The Nature of Data

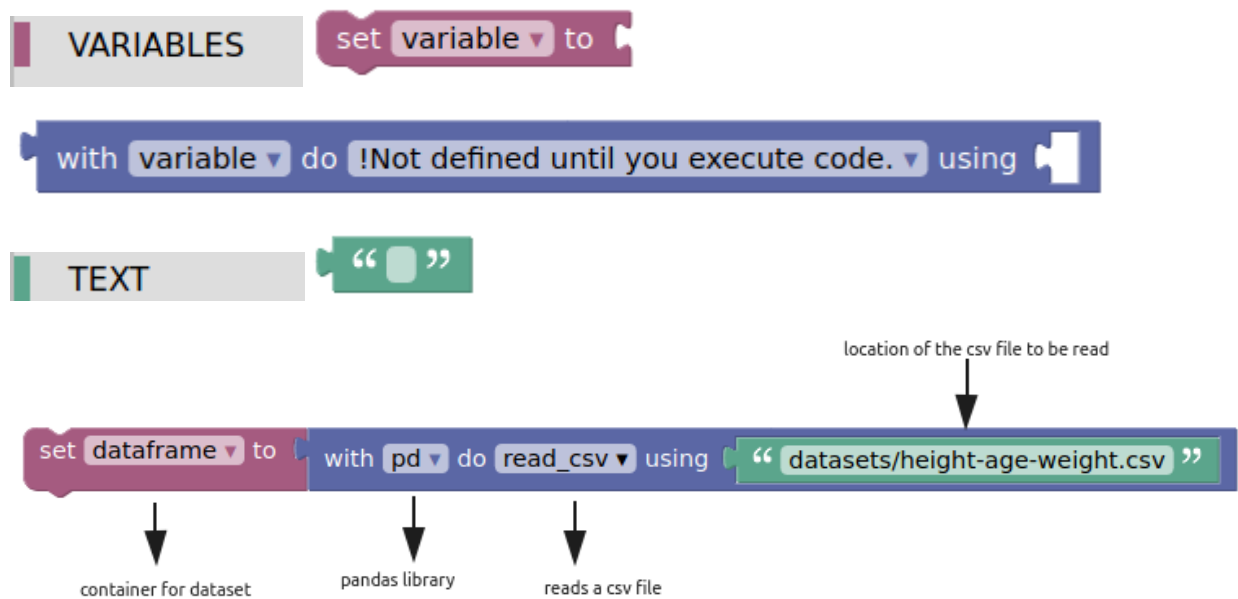
Import a library

This command lets you import any library and call it using a shortened variable.



Load a dataframe

This command loads a dataset into the container dataframe.



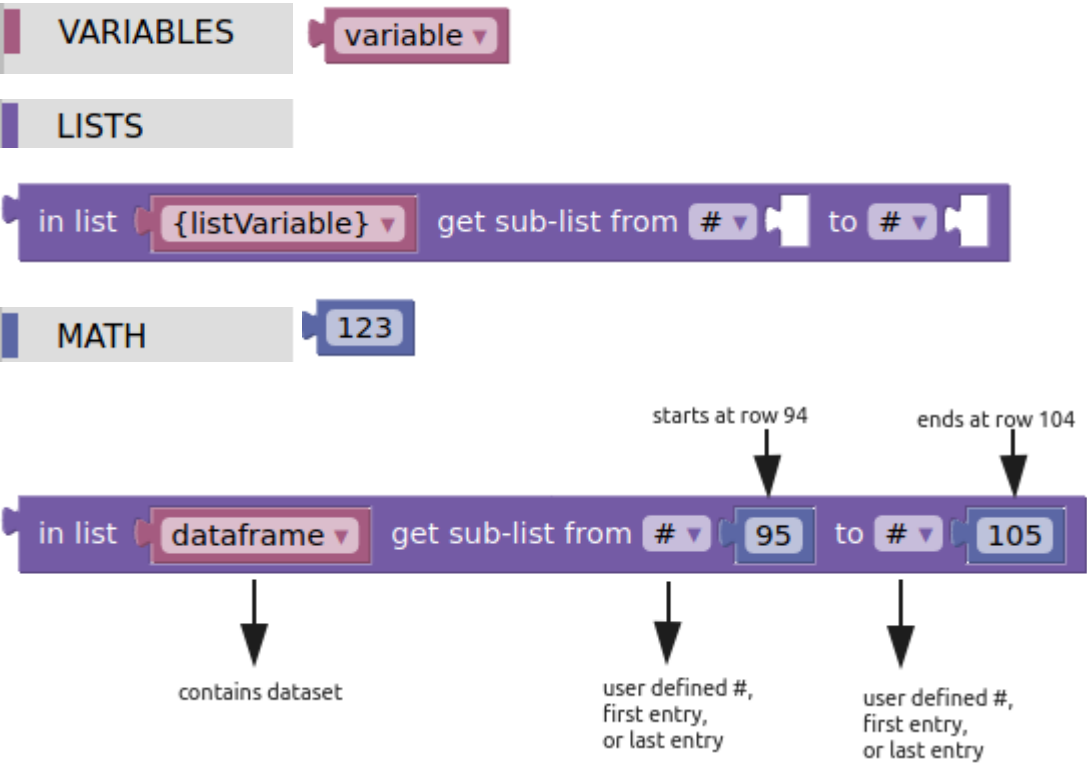
Output:

	File	PetalColor	PetalShape	Size
0	0001.png	multicolor	rounded	medium
1	0002.png	unicolor	rounded	medium
2	0003.png	unicolor	unrounded	large
3	0004.png	multicolor	rounded	medium
4	0005.png	multicolor	rounded	small
...	...	...	...	...
205	0206.png	multicolor	rounded	large
206	0207.png	unicolor	rounded	large
207	0208.png	unicolor	unrounded	large
208	0209.png	multicolor	rounded	medium
209	0210.png	unicolor	rounded	small

210 rows × 4 columns

List rows of data

This command lists a user defined location of rows in the dataset.

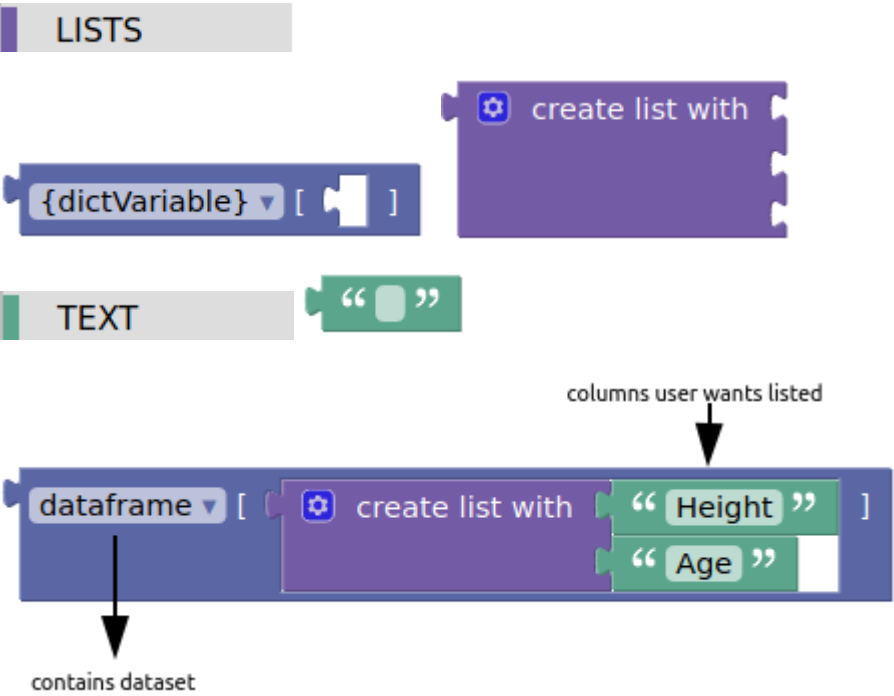


Output:

	File	PetalColor	PetalShape	Size
94	0095.png	unicolor	unrounded	small
95	0096.png	unicolor	rounded	medium
96	0097.png	unicolor	unrounded	medium
97	0098.png	multicolor	rounded	medium
98	0099.png	unicolor	rounded	medium
99	0100.png	unicolor	rounded	medium
100	0101.png	multicolor	unrounded	medium
101	0102.png	unicolor	unrounded	medium
102	0103.png	unicolor	unrounded	small
103	0104.png	unicolor	rounded	small
104	0105.png	multicolor	unrounded	large

List columns of data

This command lists a user defined location of columns in the dataset.

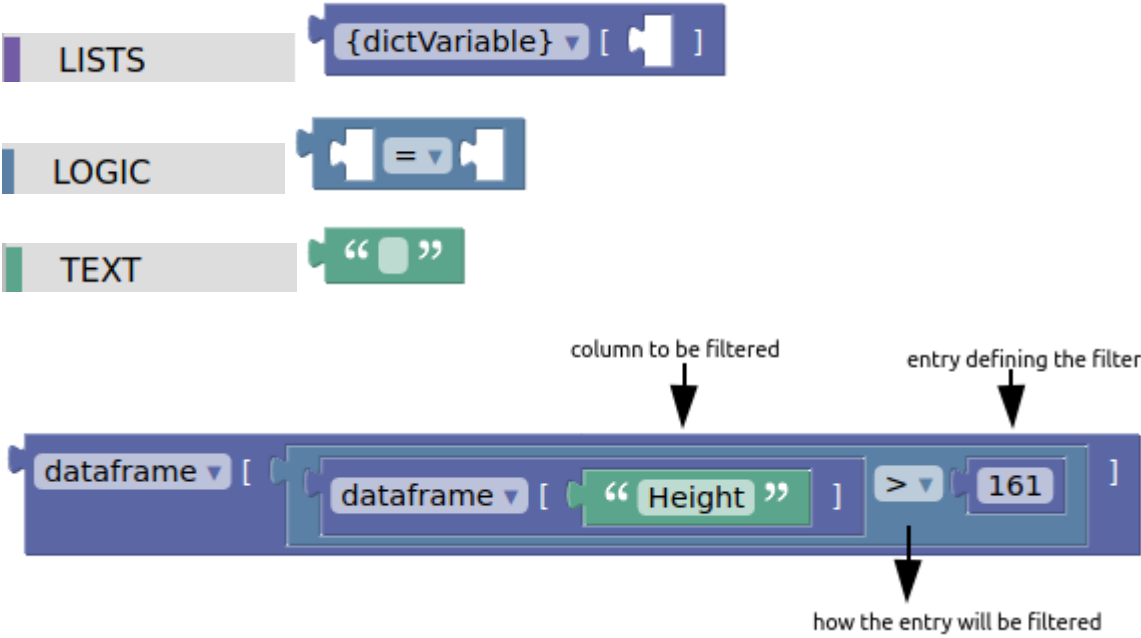


Output:

	Height	Age
0	161	50
1	161	17
2	155	33
3	180	51
4	186	18

Filter data

This command filters data so only entries that satisfy the equation (result in true) will be shown.



Output:

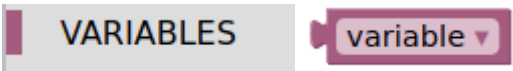
	File	PetalColor	PetalShape	Size
1	0002.png	unicolor	rounded	medium
2	0003.png	unicolor	unrounded	large
5	0006.png	unicolor	rounded	small
6	0007.png	unicolor	rounded	medium
7	0008.png	unicolor	rounded	medium
...	...	...	...	...
203	0204.png	unicolor	unrounded	small
204	0205.png	unicolor	unrounded	medium
206	0207.png	unicolor	rounded	large
207	0208.png	unicolor	unrounded	large
209	0210.png	unicolor	rounded	small

129 rows × 4 columns

Plotting

Generate a scatterplot

This command generates a scatterplot using the plotly.express library



with **variable** do !Not defined until you execute code. using

LISTS

FREESTYLE type code here...

with **px** do **scatter** using **create list with** **dta_iris** **x="SepalWidth"** **y="SepalLength"**

dataset to be plotted

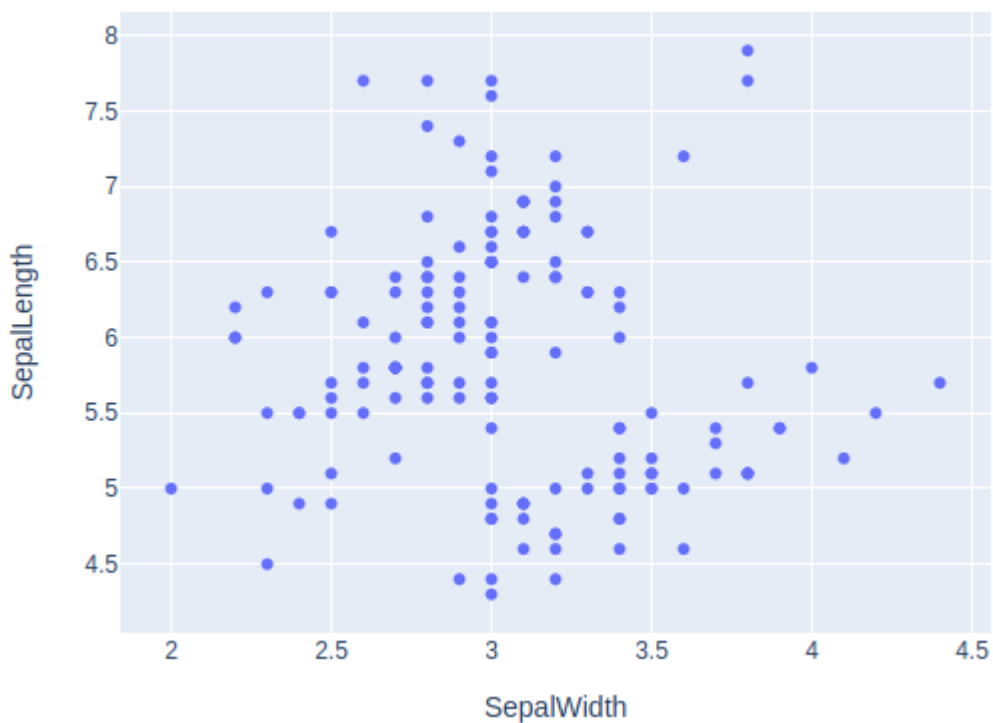
column to be used on x-axis

column to be used on y-axis

plotly express library

scatter plot

Output:



Label graph

This command adds a title and color coding to elements of the plot.

VARIABLES **variable**

with **variable** do !Not defined until you execute code. using

LISTS

FREESTYLE

create list with

with **px** do **scatter** using

create list with

dta_iris → dataset to be plotted

x="SepalWidth" → column to be used on x-axis

y="SepalLength" → column to be used on y-axis

title="Relationship between Sepal Length and Sep..." → name of plot

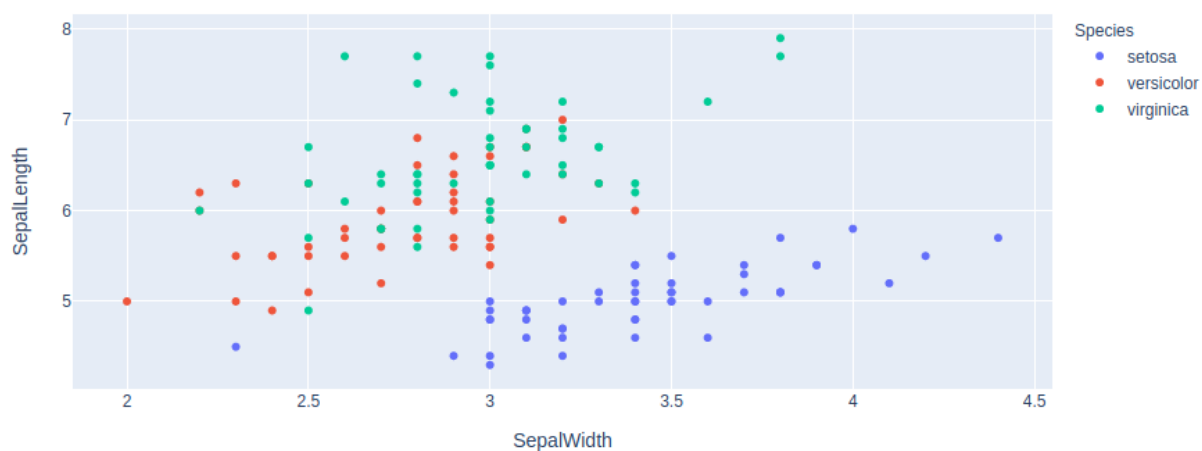
color="Species" → field to be color-coded

plotly express library

scatter plot

Output:

Relationship between Sepal Length and Sepal Width

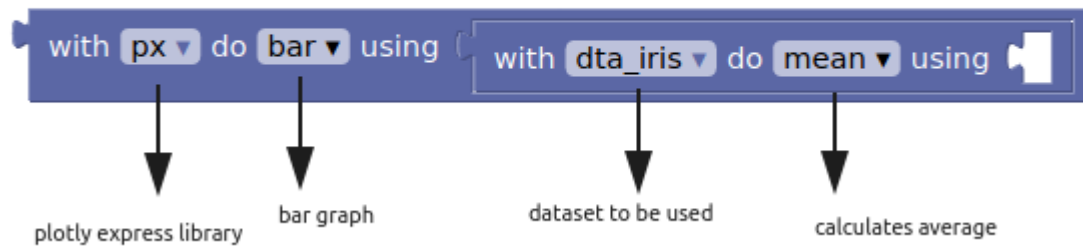


Generate a bar plot

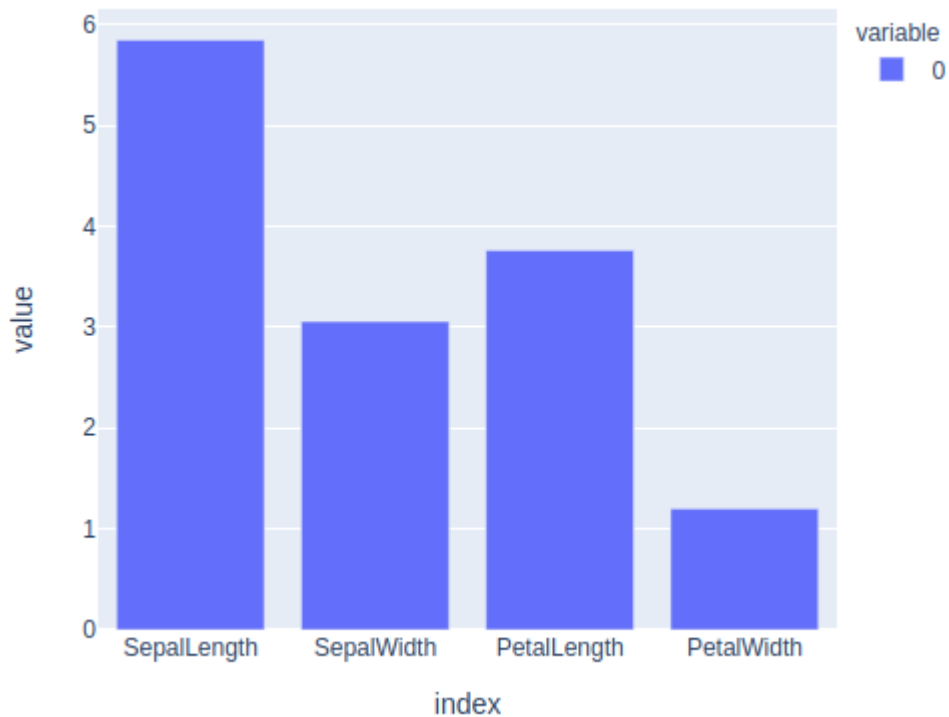
This command generates a bar plot using the plotly.express library

VARIABLES

with **variable** do **!Not defined until you execute code.** using

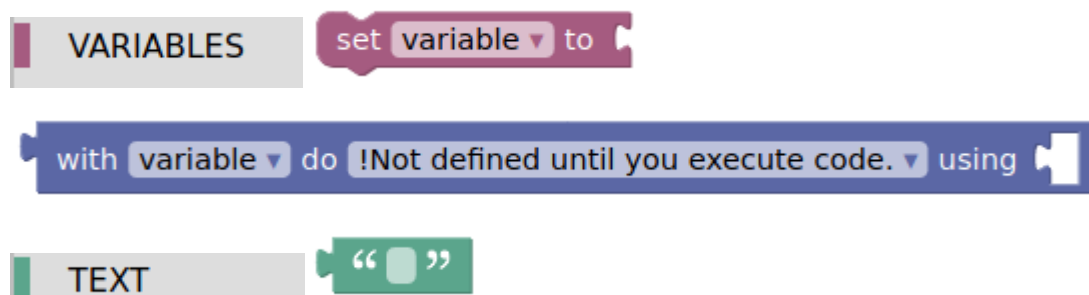


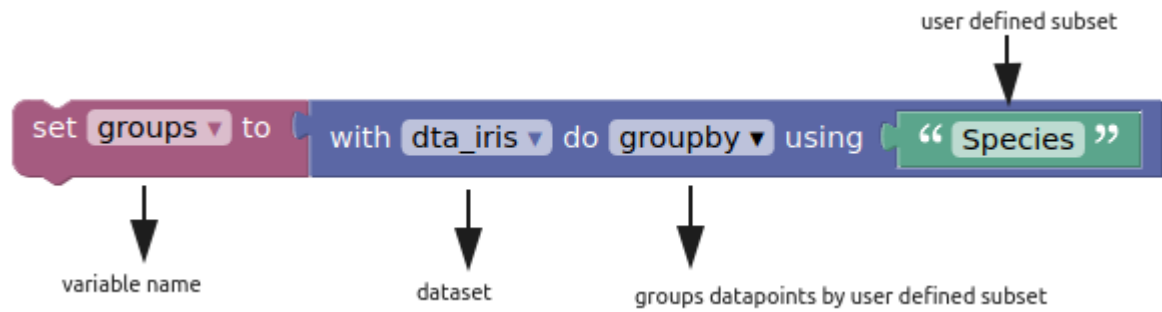
Output:



Group datapoints

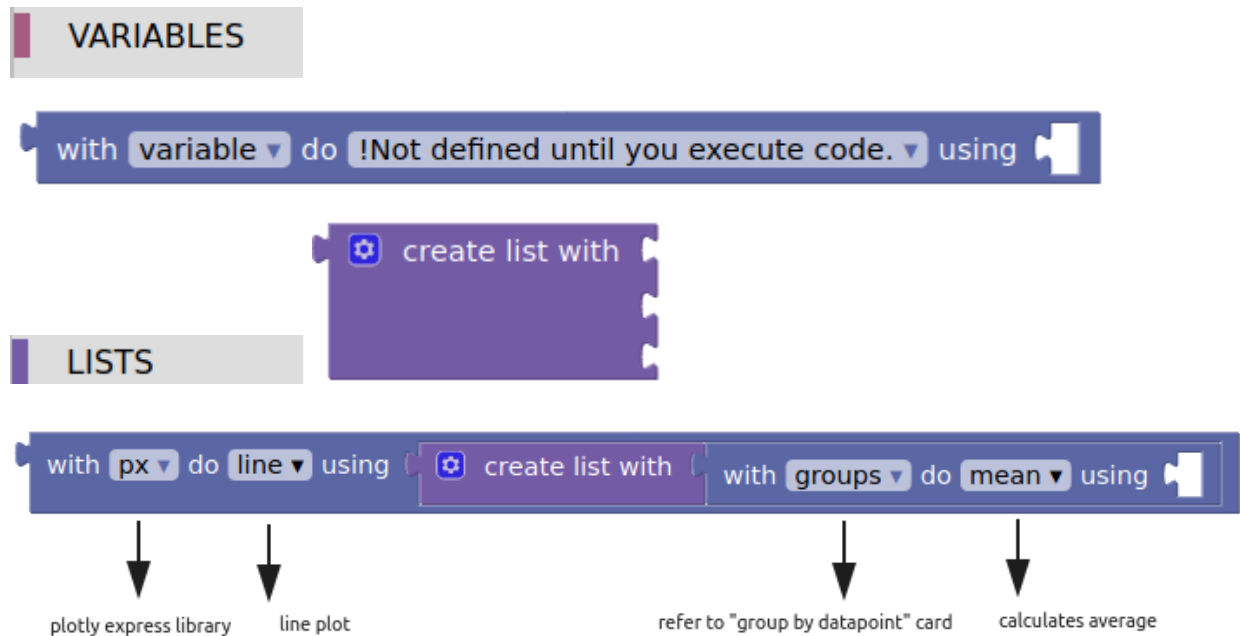
This command groups datapoints by a user defined sub-set



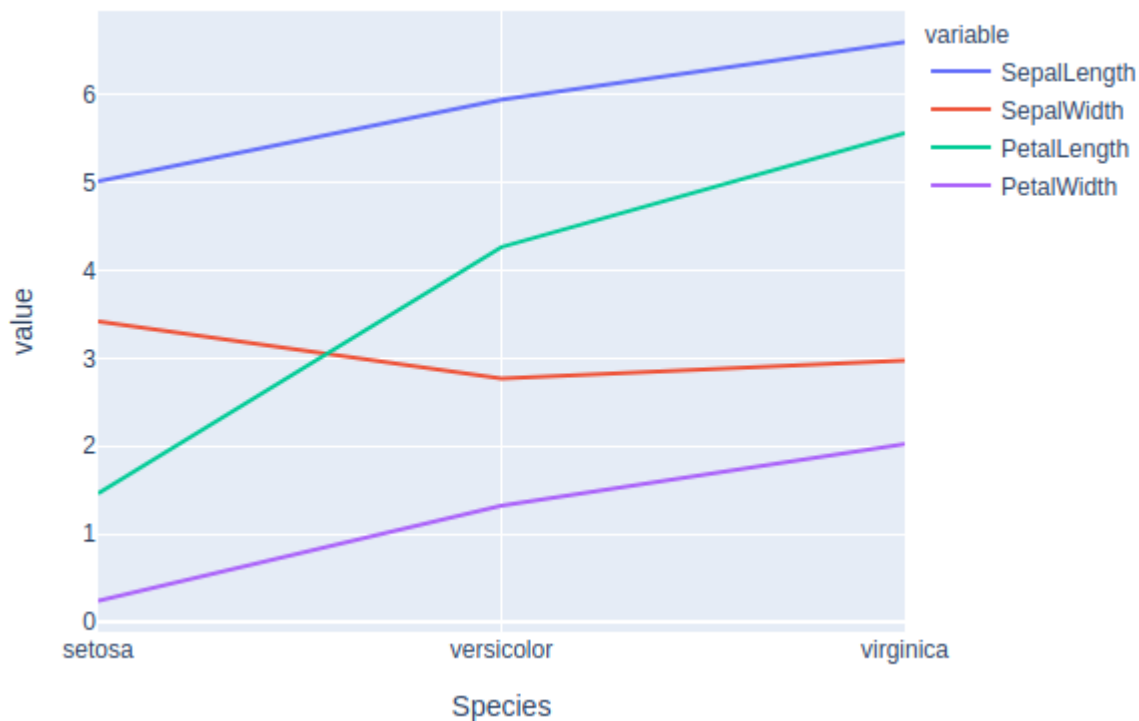


Generate a line plot

This command generates a line plot using the plotly.express library



Output:



Generate a histogram

This command generates a histogram using the plotly.express library

VARIABLES variable ▾

with variable ▾ do !Not defined until you execute code. ▾ using ▶

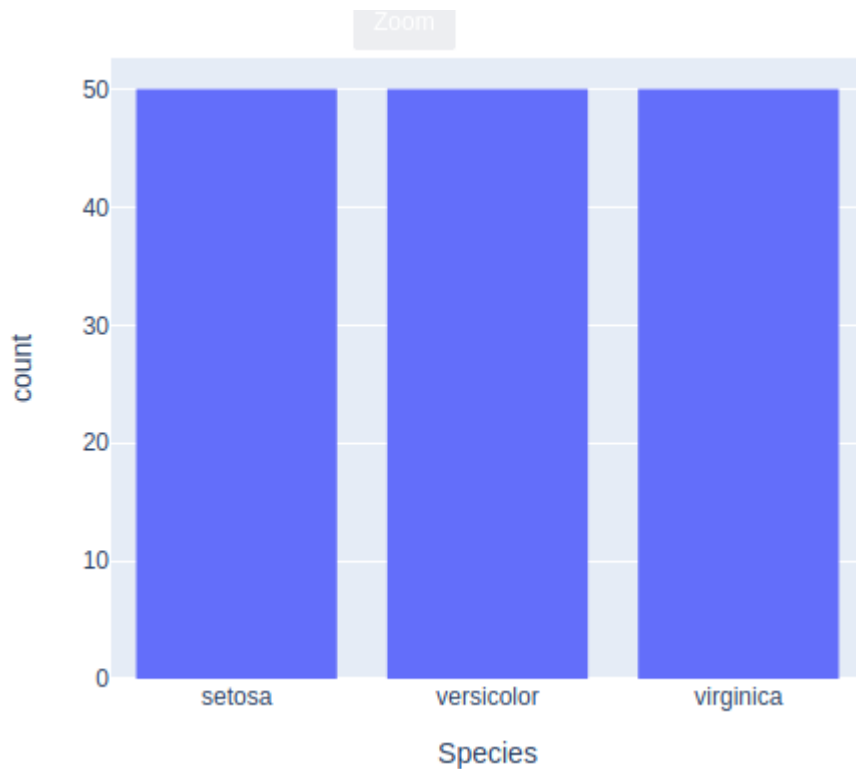
LISTS create list with

FREESTYLE type code here...

with px ▾ do histogram ▾ using create list with dta_iris ▾ x="SepalWidth"

plotly express library plot histogram dataset to be plotted column to be used as x-axis

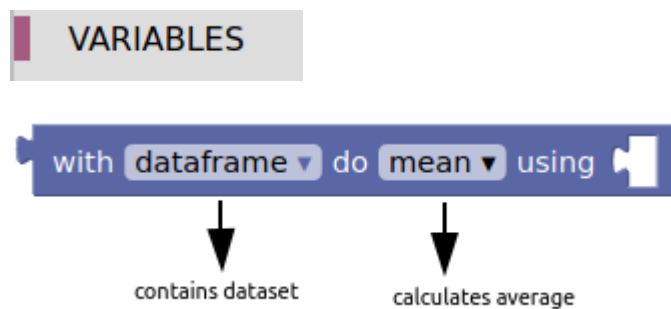
Output:



Descriptive Statistics

Calculate the mean of a dataset

This command calculates the mean of all columns in the dataset dataframe

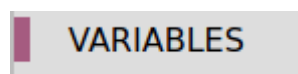


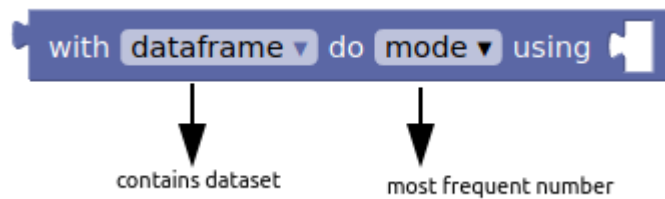
Output:

```
SepalLength    5.843333
SepalWidth     3.054000
PetalLength    3.758667
PetalWidth     1.198667
dtype: float64
```

Calculate the mode of a dataset

This command calculates the mode of all columns in the dataset dataframe



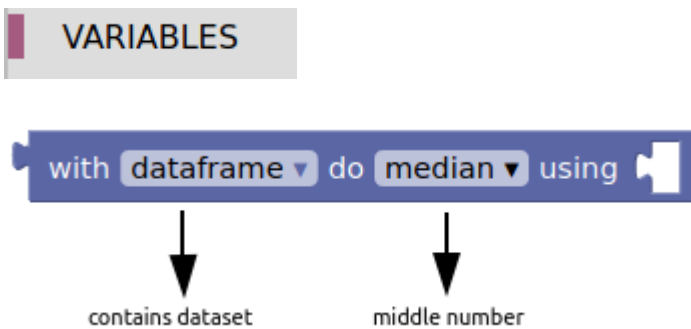


Output:

	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
0	5.0	3.0	1.5	0.2	setosa
1	NaN	NaN	NaN	NaN	versicolor
2	NaN	NaN	NaN	NaN	virginica

Calculate the median of a dataset

This command calculates the median of all columns in the dataset dataframe

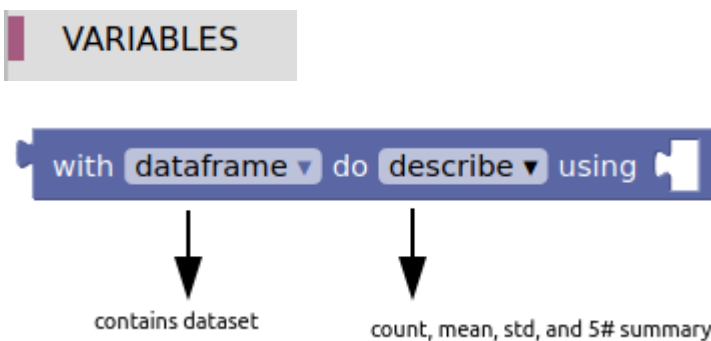


Output:

```
SepalLength    5.80
SepalWidth     3.00
PetalLength    4.35
PetalWidth     1.30
dtype: float64
```

Summary of a dataset

This command calculates the total number of entries, mean, standard deviation, minimum entry, first quartile, median (second quartile), third quartile, and maximum entry of all columns in the dataset dataframe.

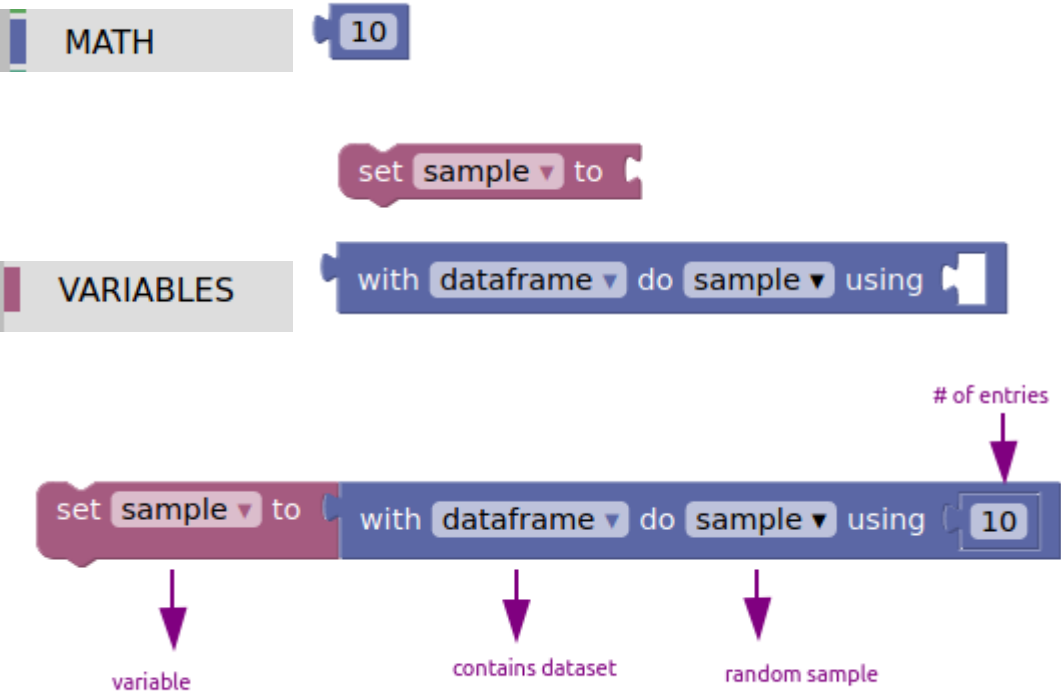


Output:

	SepalLength	SepalWidth	PetalLength	PetalWidth
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Get a calculation of a random sample set

This command generates 10 random entries from the dataset dataframe.



Output:

	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
113	5.7	2.5	5.0	2.0	virginica
129	7.2	3.0	5.8	1.6	virginica
42	4.4	3.2	1.3	0.2	setosa
101	5.8	2.7	5.1	1.9	virginica
131	7.9	3.8	6.4	2.0	virginica
123	6.3	2.7	4.9	1.8	virginica
111	6.4	2.7	5.3	1.9	virginica
61	5.9	3.0	4.2	1.5	versicolor
41	4.5	2.3	1.3	0.3	setosa
141	6.9	3.1	5.1	2.3	virginica

Measures of Association

Plot a labeled correlation heatmap

This command displays a correlation matrix as a heatmap.

VARIABLES

set **variable** to **variable**

with **variable** do **!Not defined until you execute code.** using

from **variable** get **!Not defined until you execute code.**

LISTS

create list with

FREESTYLE

type code here...

set **fig_iris** to **with px** do **imshow** using **create list with**

correlation matrix of dataframe

x-axis label

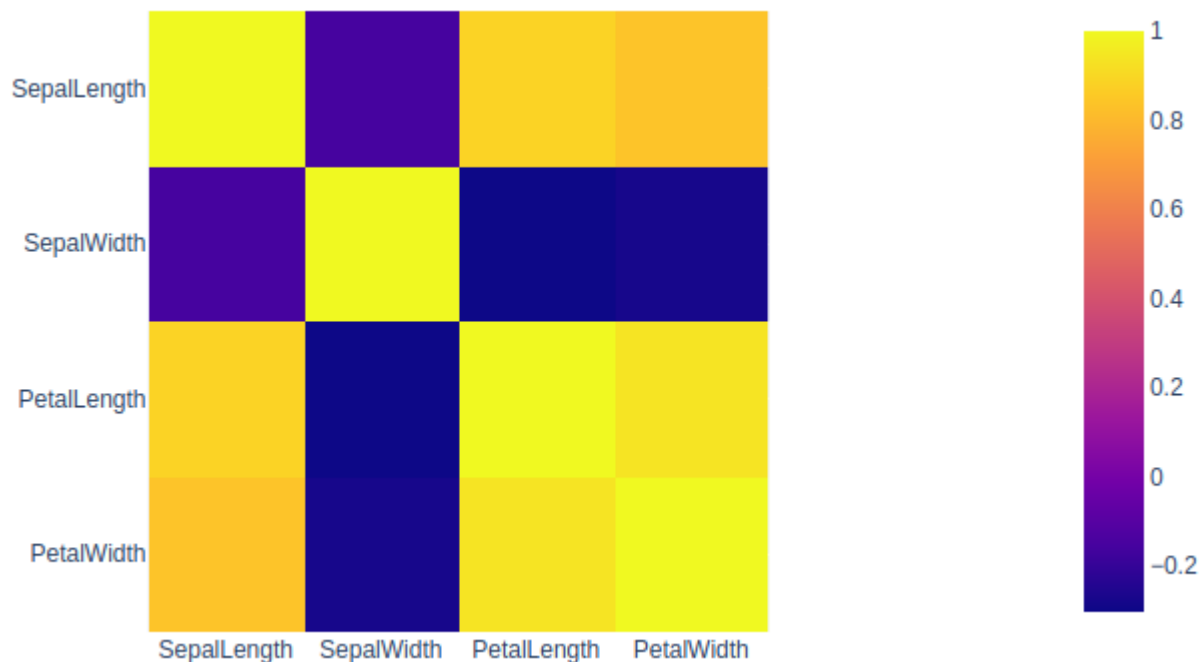
y-axis label

variable / dataset

plotly express library

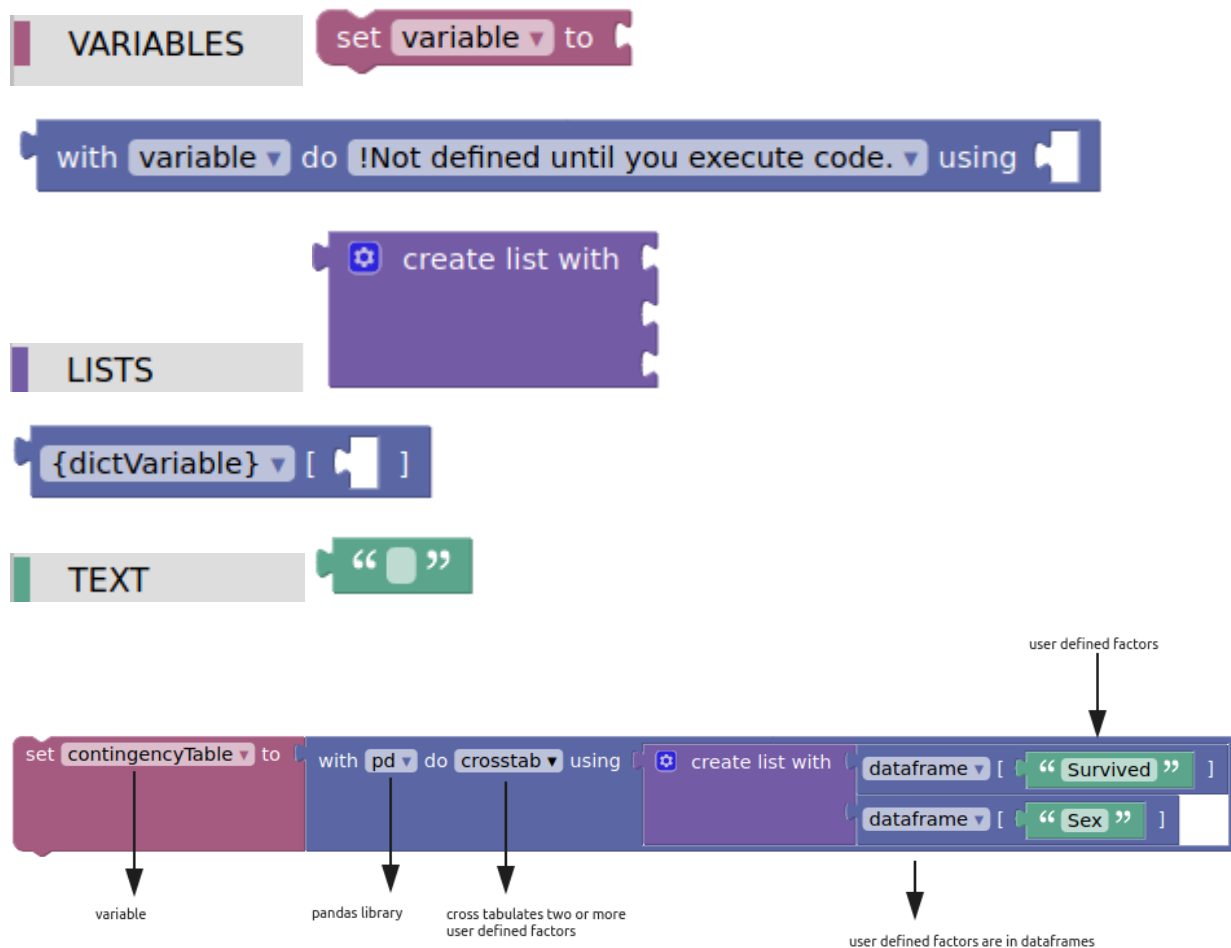
multichannel image data

Output:



Create a contingency table

This command displays a frequency distribution of user defined factors.



Output:

Sex	female	male
Survived		
0	81	464
1	233	109

Clustering

Load data with identifier (ID)

This command loads a dataset with a respective identity column, which is usually a variable that never repeats a value.

VARIABLES set variable ▼ to

with variable ▼ do !Not defined until you execute code. ▼ using

LISTS create list with

TEXT “ ”

FREESTYLE type code here...

csv file to be read

variable being created

pandas library

reads a given csv file

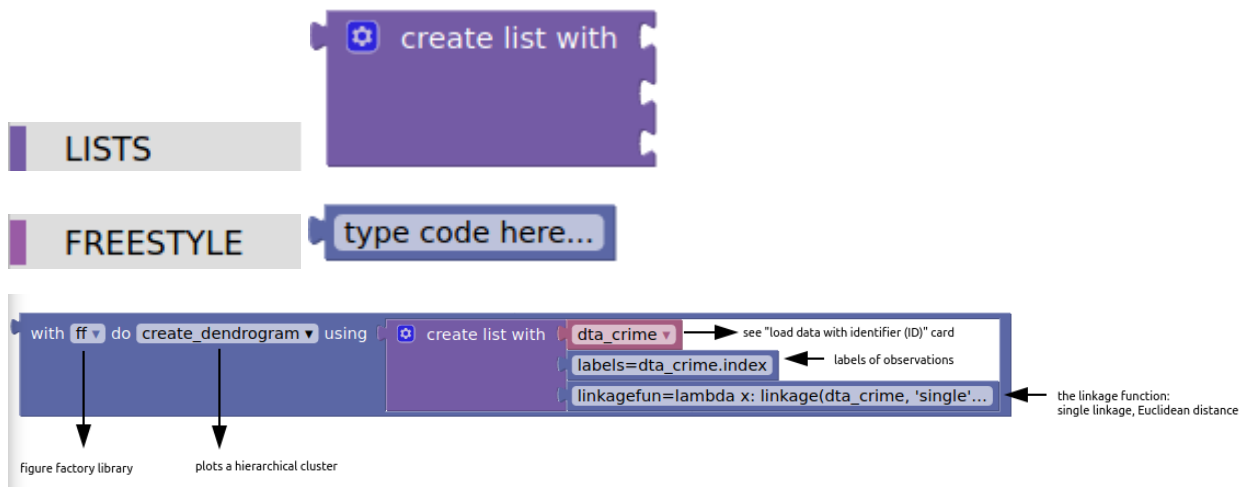
identifier/ID of csv file

Create a dendrogram

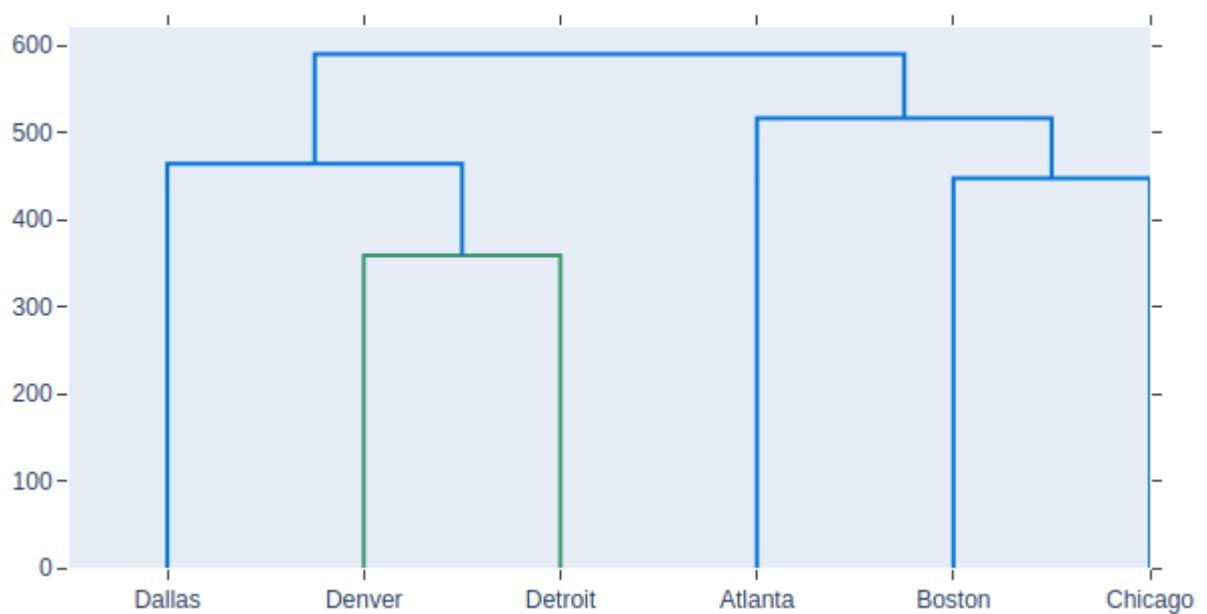
This command creates a hierarchical cluster out of a given dataset

VARIABLES variable ▼

with variable ▼ do !Not defined until you execute code. ▼ using



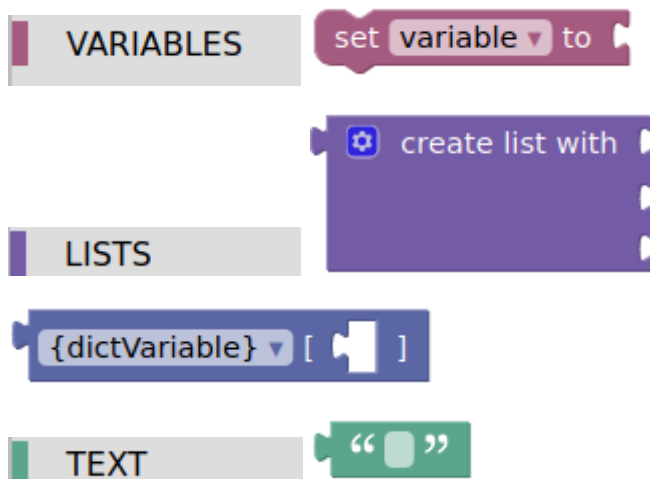
Output:

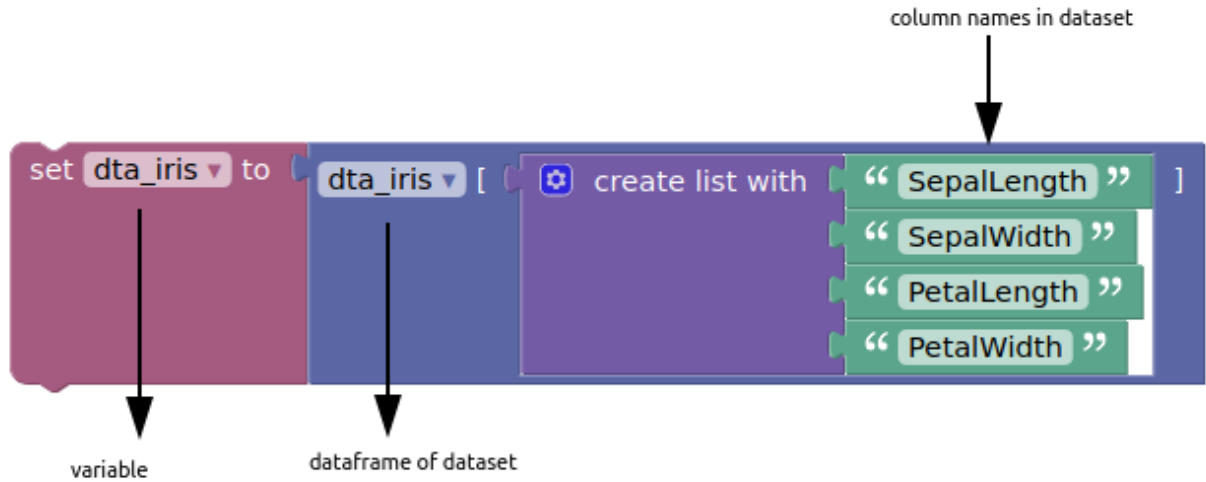


Plot Clusters

This series of commands runs a k-means clustering algorithm

1) Setup variable to run k-means using column names from the dataset



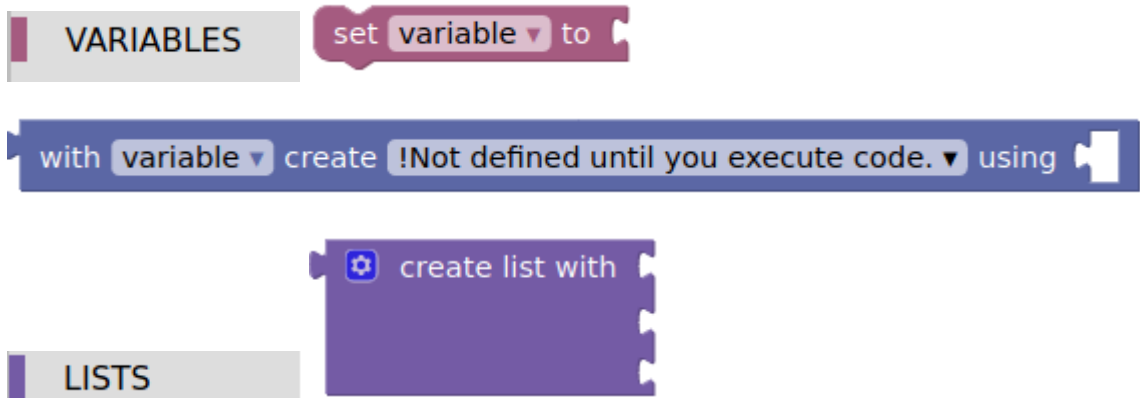


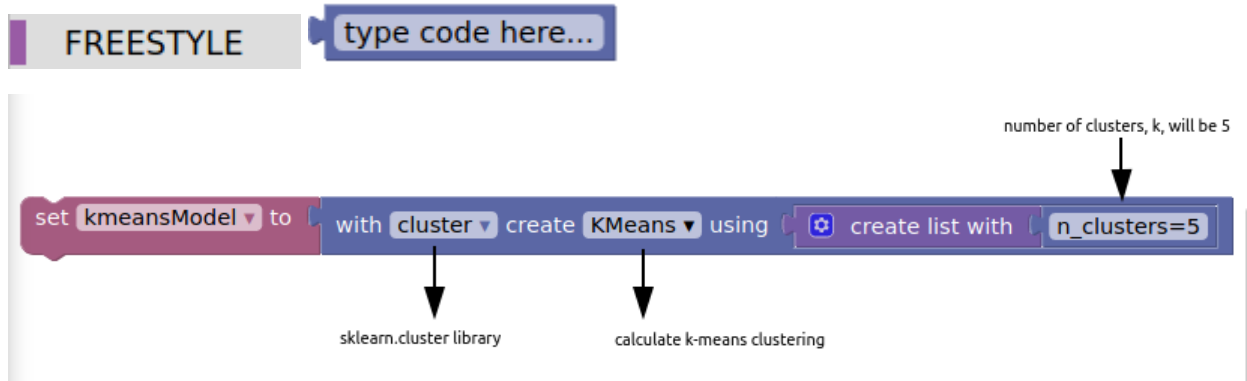
Output:

	SepalLength	SepalWidth	PetalLength	PetalWidth
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

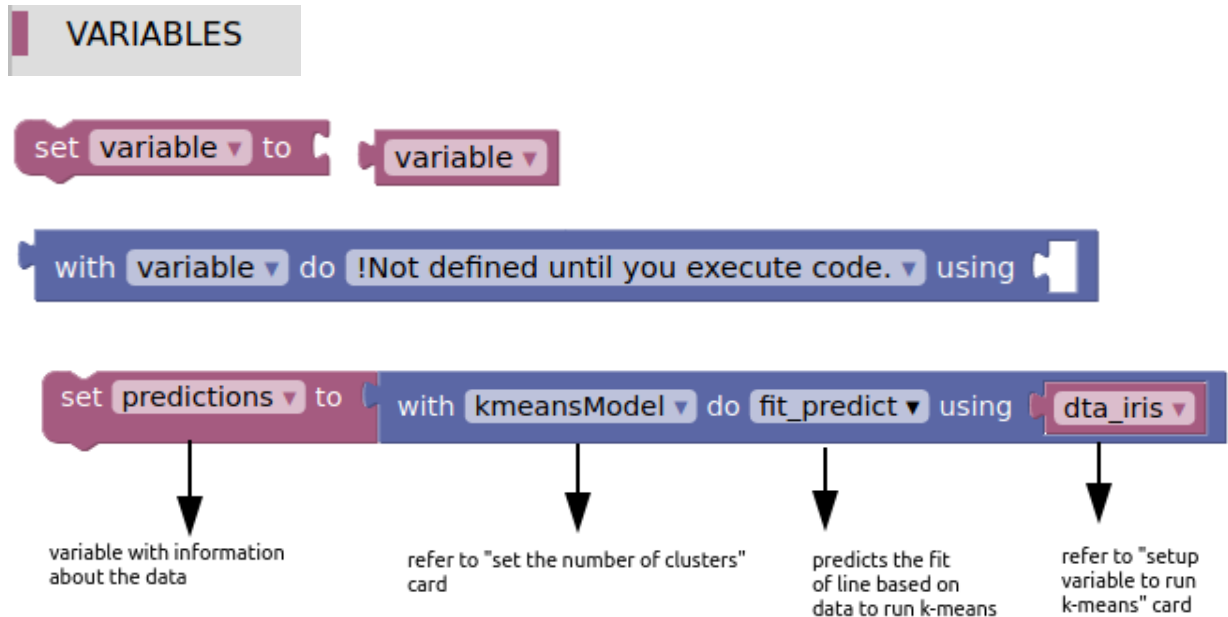
150 rows × 4 columns

2) Set the number of clusters (k) and create a cluster model





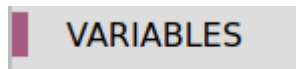
3) Tell the variable information about the data and make predictions of clusters for each datapoint.



Output:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 3, 3, 3, 4, 3, 3, 3, 4, 3, 4,
       4, 3, 4, 3, 4, 3,
       3, 4, 3, 4, 3, 4, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4,
       4, 3, 4, 3, 3, 3,
       4, 4, 4, 3, 4, 4, 4, 4, 4, 3, 4, 4, 1, 3, 2, 1,
       1, 2, 4, 2, 1, 2,
       1, 1, 1, 3, 1, 1, 1, 2, 2, 3, 1, 3, 2, 3, 1, 2,
       3, 3, 1, 2, 2, 2,
       1, 3, 3, 2, 1, 1, 3, 1, 1, 1, 3, 1, 1, 1, 3, 1,
       1, 3], dtype=int32)
```

4) Add cluster assignments as a new column.



with variable ▾ do !Not defined until you execute code. ▾ using

create list with

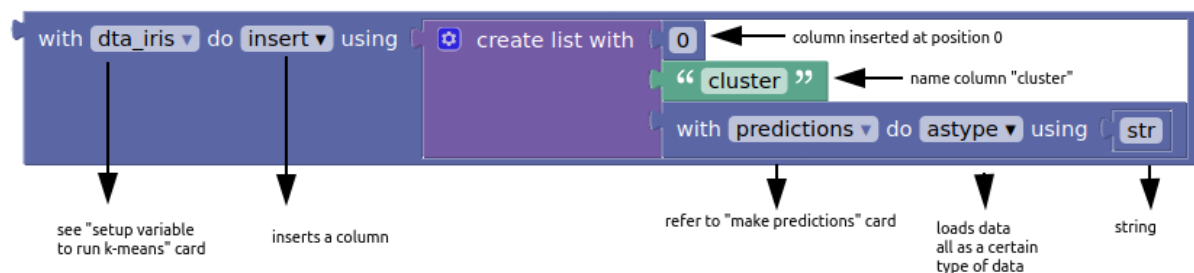
LISTS

FREESTYLE

type code here...

TEXT

“ ”



Output:

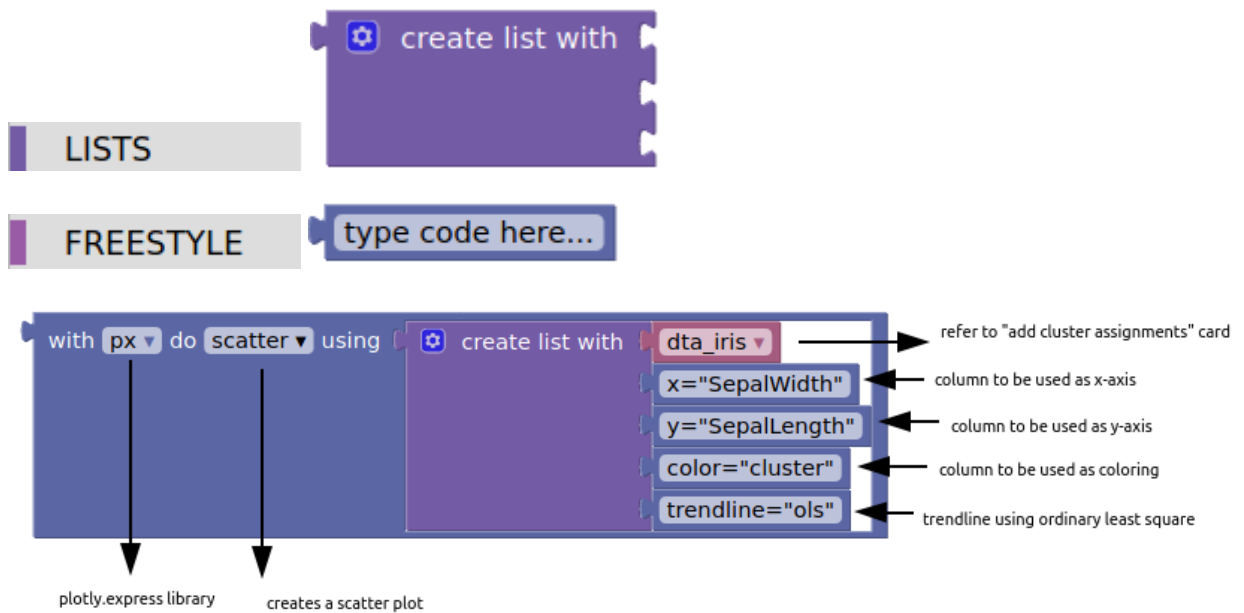
	cluster	SepalLength	SepalWidth	PetalLength	PetalWidth
0	0	5.1	3.5	1.4	0.2
1	0	4.9	3.0	1.4	0.2
2	0	4.7	3.2	1.3	0.2
3	0	4.6	3.1	1.5	0.2
4	0	5.0	3.6	1.4	0.2
...	...	...	...	...	...
145	1	6.7	3.0	5.2	2.3
146	3	6.3	2.5	5.0	1.9
147	1	6.5	3.0	5.2	2.0
148	1	6.2	3.4	5.4	2.3
149	3	5.9	3.0	5.1	1.8

150 rows × 5 columns

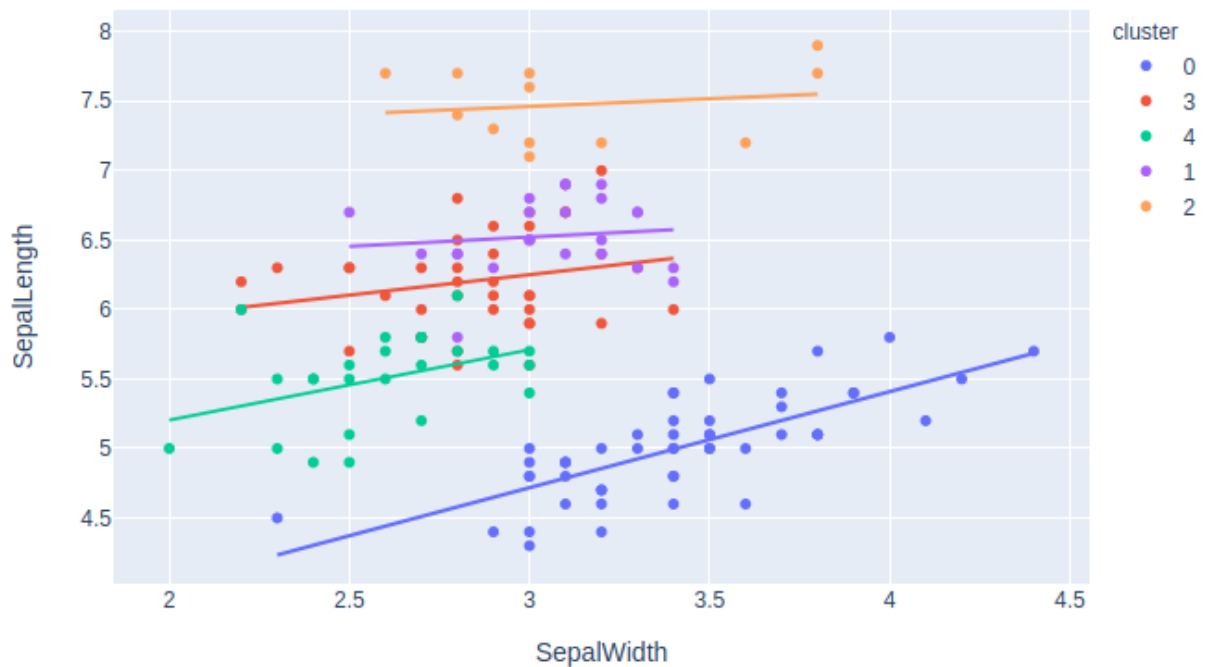
5) Create a scatterplot of clusters.

VARIABLES variable ▾

with variable ▾ do !Not defined until you execute code. ▾ using



Output:

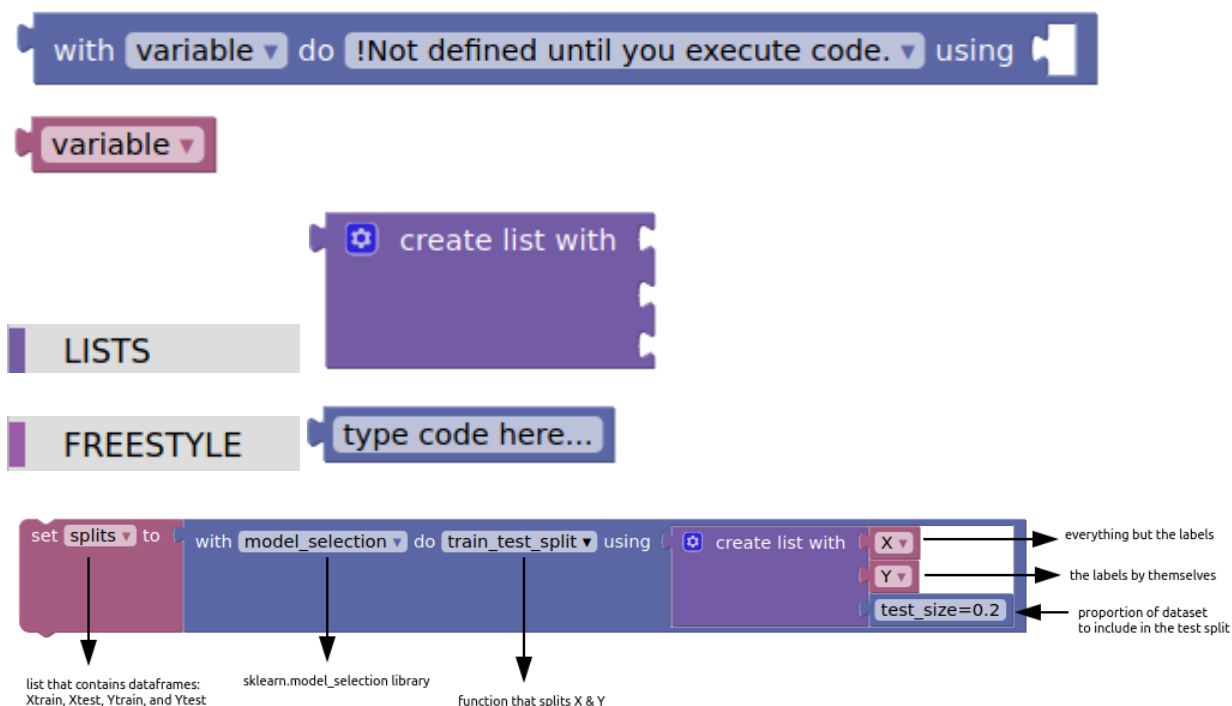


KNN Classification

Split X and Y into train and test sets

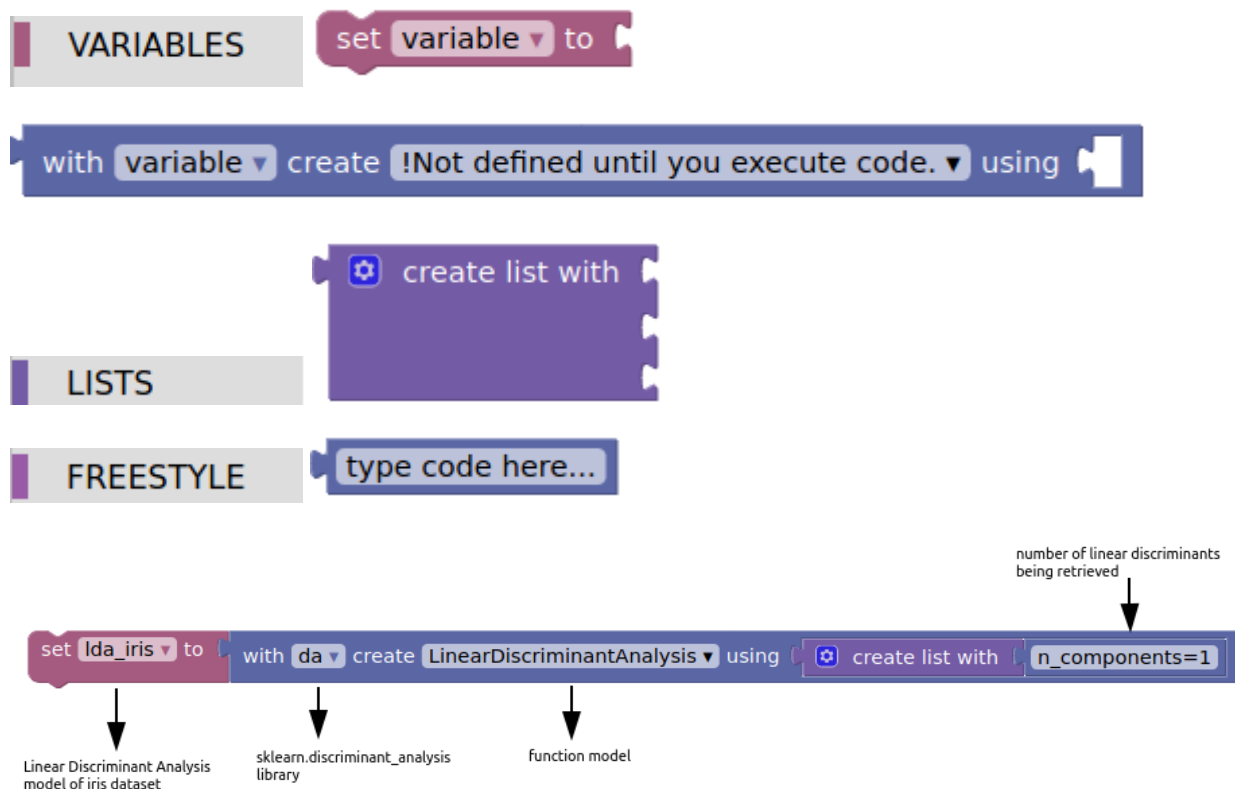
This command splits pre-defined variables X and Y into train and test sets. It outputs four dataframes: Xtrain, Xtest, Ytrain, and Ytest.





Define the LDA model

This command sets up a base LDA model.

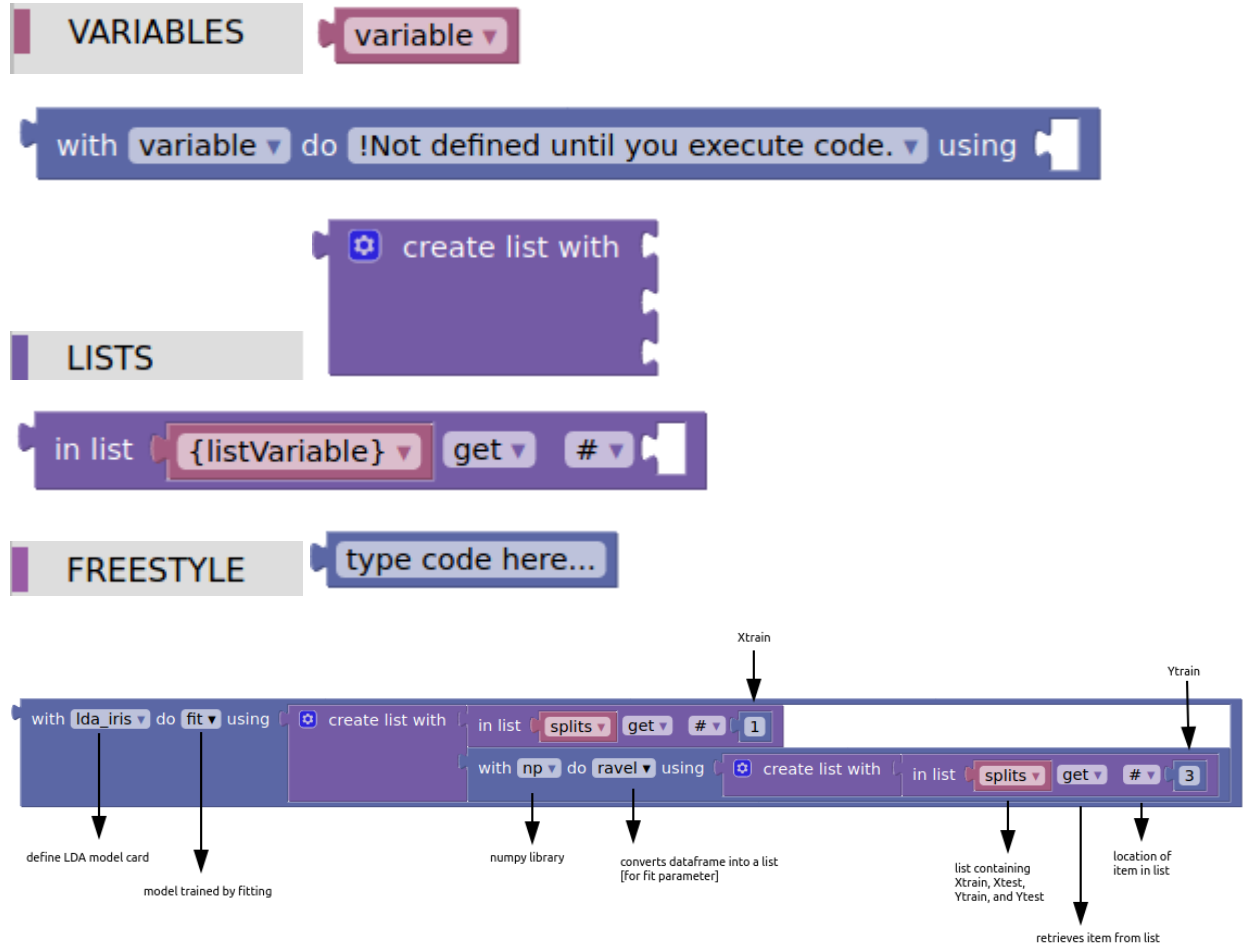


Output:

```
LinearDiscriminantAnalysis(n_components=1, priors=None, shrinkage=None,
                           solver='svd', store_covariance=False, tol=0.0001)
```

Train the LDA model

This command trains the previously defined LDA model with user given data.

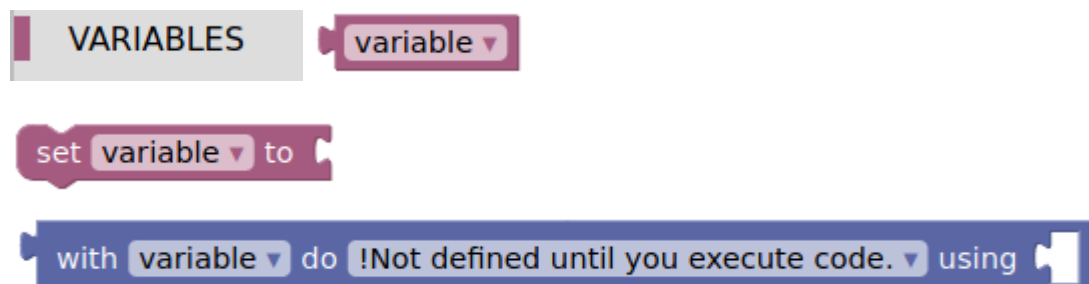


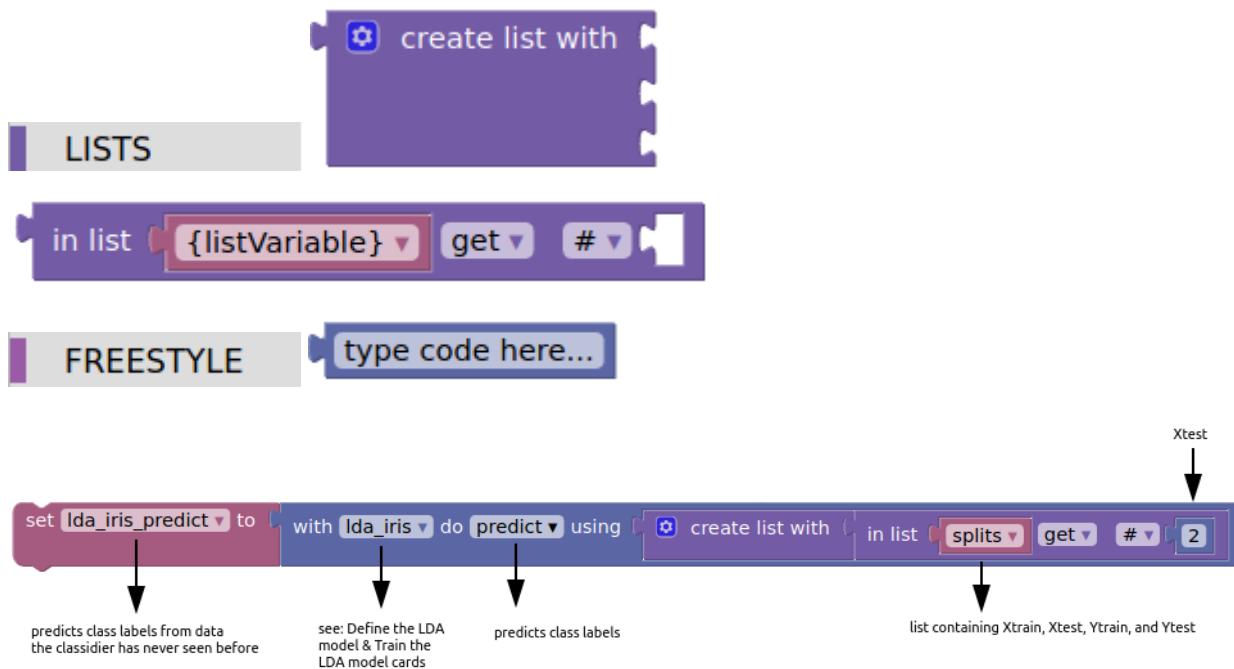
Output:

```
array(['virginica', 'virginica', 'setosa', 'versicolor', 'versicolor',
      'versicolor', 'virginica', 'virginica', 'versicolor', 'setosa',
      'setosa', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
      'virginica', 'setosa', 'versicolor', 'setosa', 'versicolor',
      'virginica', 'setosa', 'versicolor', 'virginica', 'versicolor',
      'versicolor', 'versicolor', 'virginica', 'versicolor', 'virginica'],
      dtype='<U10')
```

Predict classification with the LDA model

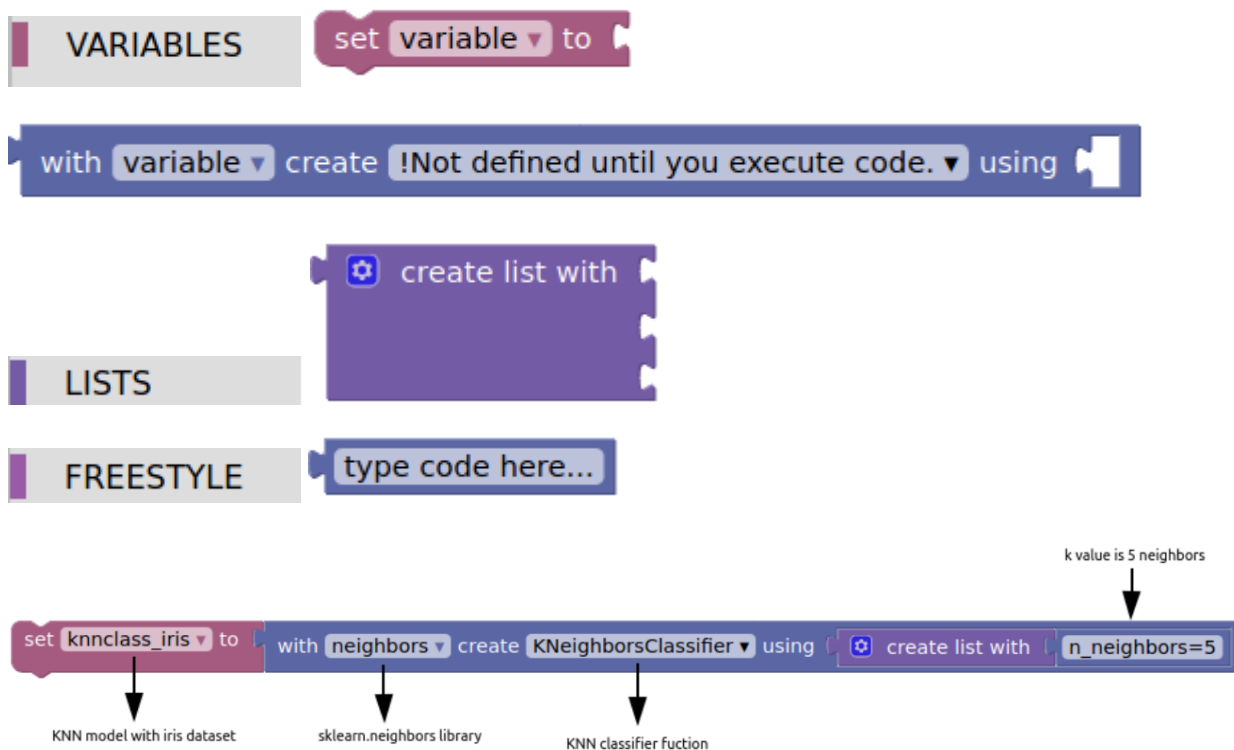
This command predicts classification using the trained LDA model.





Define the KNN classification model

This command sets up a base KNN model.



Train the KNN classification model

This command trains the previously defined KNN model with user given data.

[See **Train the LDA Model**. Replace variable "lda_iris" with "knnclass_iris".]

Output:

```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                    metric_params=None, n_jobs=None, n_neighbors=5, p=2,
                    weights='uniform')
```

Predict classification with the KNN model

This command predicts classification using the trained KNN model.

[See **Predict Classification with the LDA Model**. Replace "lda_iris_predict" with "knnclass_iris_predict" and replace "lda_iris" with "knnclass_iris".]

Output:

```
array(['virginica', 'virginica', 'setosa', 'versicolor', 'versicolor',
      'versicolor', 'versicolor', 'virginica', 'versicolor', 'setosa',
      'setosa', 'versicolor', 'versicolor', 'versicolor', 'virginica',
      'virginica', 'setosa', 'versicolor', 'setosa', 'versicolor',
      'virginica', 'setosa', 'versicolor', 'virginica', 'virginica',
      'versicolor', 'versicolor', 'virginica', 'versicolor', 'virginica'],
      dtype=object)
```

Get confusion matrix of a model

This command outputs an array representing the confusion matrix of a prediction model. This is used to assess the predictive value (performance) of the model.

VARIABLES variable

with variable do !Not defined until you execute code. using

LISTS create list with

in list {listVariable} get #

FREESTYLE type code here...

with metrics do confusion_matrix using create list with in list splits get # 4

lda_iris_predict

sklearn.metrics library

confusion matrix function

see Predict Classification with LDA model card

list containing Xtrain, Xtest, Ytrain, and Ytest

Ytest

Output:

```
array([[ 6,  0,  0],
       [ 0, 14,  0],
       [ 0,  1,  9]])
```

Get accuracy of a model

This command retrieves a decimal percentage representing the accuracy of the prediction model.

VARIABLES variable ▾

with variable ▾ do !Not defined until you execute code. ▾ using

create list with

LISTS

in list {listVariable} ▾ get ▾ # ▾

FREESTYLE type code here...

with metrics ▾ do confusion_matrix ▾ using

create list with

in list splits ▾ get ▾ # ▾ 4

lda_iris_predict ▾

sklearn.metrics library

confusion matrix function

see Predict Classification with LDA model card

list containing Xtrain, Xtest, Ytrain, and Ytest

Ytest

Output:

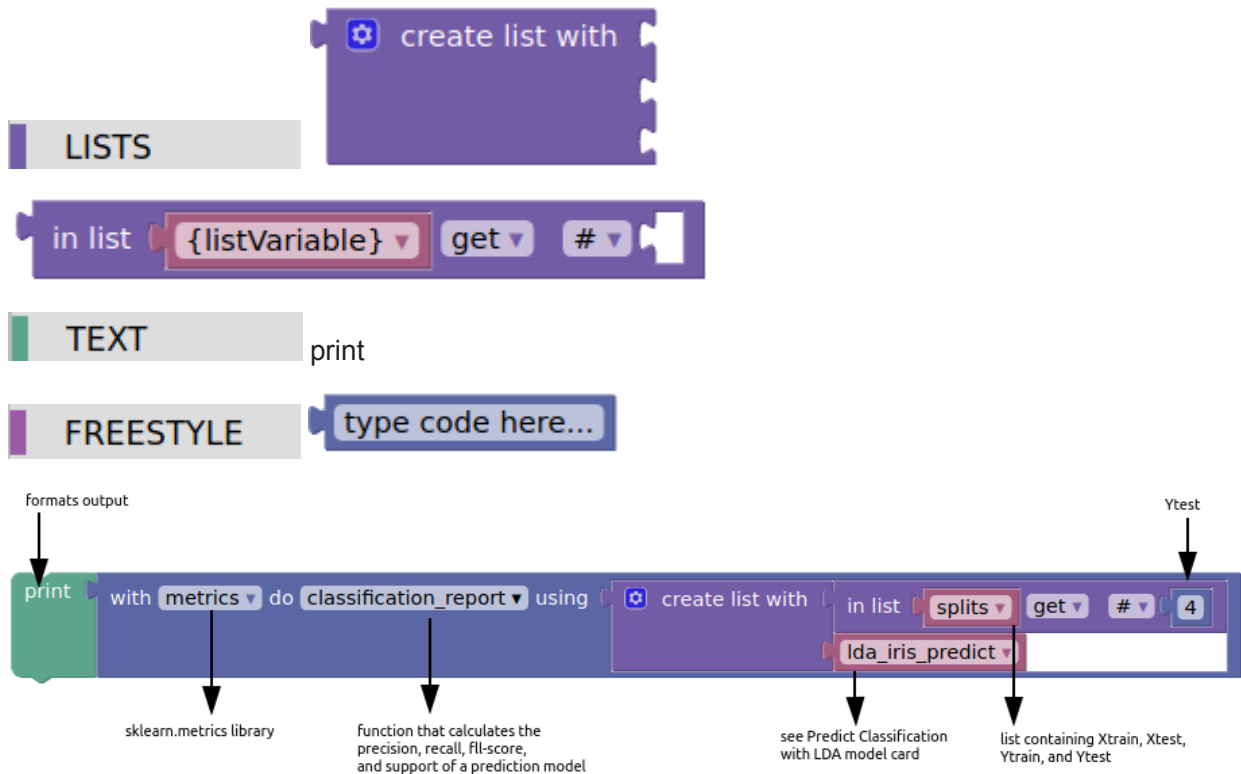
```
array([[ 6,  0,  0],
       [ 0, 14,  0],
       [ 0,  1,  9]])
```

Get recall and precision of a model

This command shows a classification report of the prediction model. The report includes decimal percentages of precision, recall, fl-score, and support.

VARIABLES variable ▾

with variable ▾ do !Not defined until you execute code. ▾ using

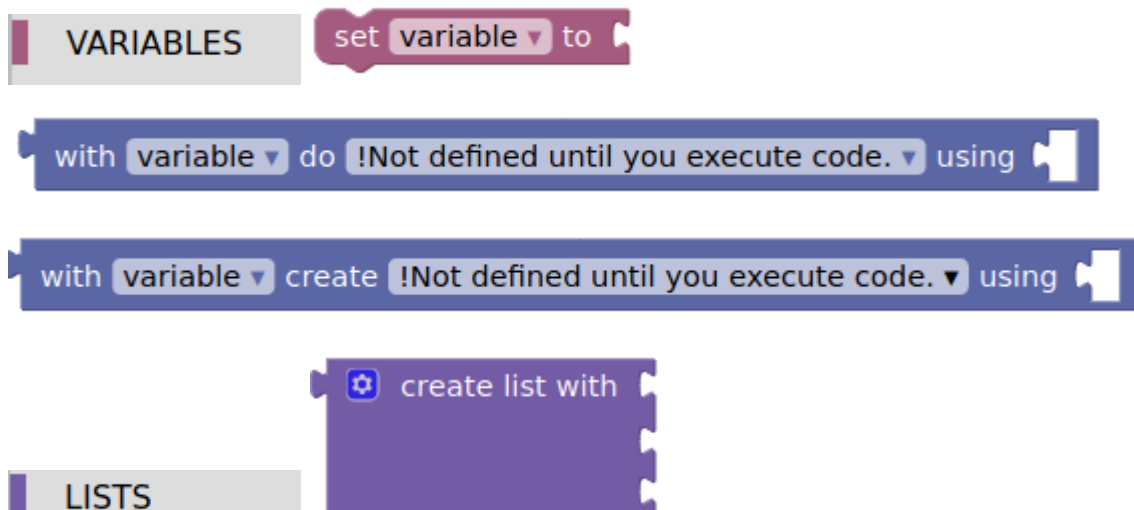


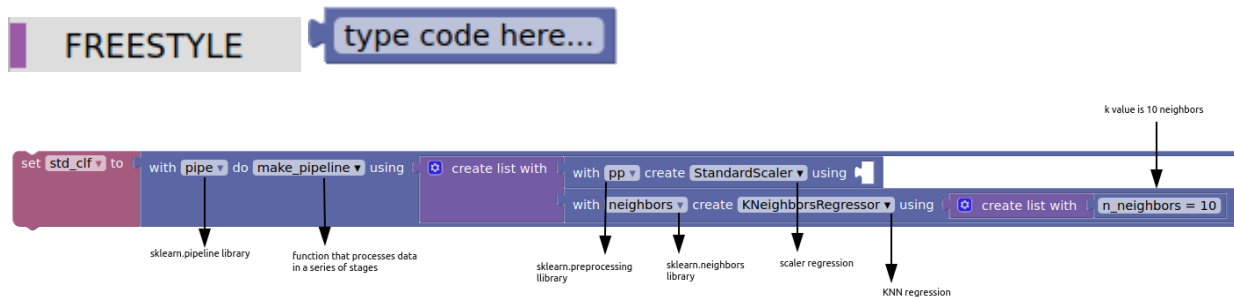
Output:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	6
versicolor	0.93	1.00	0.97	14
virginica	1.00	0.90	0.95	10
accuracy			0.97	30
macro avg	0.98	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

KNN Regression

Create a pipeline with a scaler and KNN regression





Train the KNN regression model

[See **Train the LDA Model**. Replace variable "lda_iris" with "std_clf".]

Output:

```
Pipeline(memory=None,
          steps=[('standardscaler',
                  StandardScaler(copy=True, with_mean=True, with_std=True)),
                 ('kneighborsregressor',
                  KNeighborsRegressor(algorithm='auto', leaf_size=30,
                                     metric='minkowski', metric_params=None,
                                     n_jobs=None, n_neighbors=10, p=2,
                                     weights='uniform'))],
          verbose=False)
```

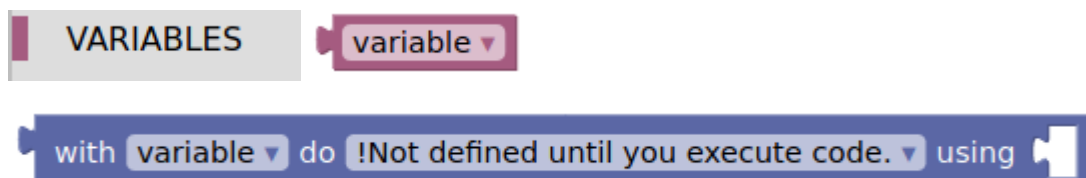
Predict classification using the KNN regression model

[See **Predict classification with the LDA Model**. Replace "lda_iris_predict" with "predictions" and replace "lda_iris" with "std_clf".]

Output:

```
array([14.92, 24.82, 22.83, 17.92, 10.48, 21.21, 29.42, 18.44, 21.17,
       16.67, 28.83, 23.84, 17.09, 37.8 ,  9.1 , 20.16, 20.17, 25.89,
       10.01, 30.62, 10.43, 17.46, 21.47, 24.87, 39.04, 26.15, 11.14,
       14.47, 22.75, 20.39, 16.13, 37.2 , 25.2 ,  8.9 , 14.99, 17.7 ,
       20.78, 23.84, 15.23, 10.62, 28.83, 22.24, 19.33, 13.12, 17.07,
       12.51, 28.04, 22.14, 11.23, 30.16, 18.65, 22.15, 28.13, 22.98,
       22.75, 25.01, 22.86, 15.78, 21.04, 21.96, 22.69, 20.95, 19.97,
       22. , 20.58, 16.21, 20.03, 35.91, 19.45, 20.33, 21.58, 19.29,
       37.38, 24.47, 21.64, 23.55, 11.98, 25.15, 10.97, 27.47, 27.99,
       17.46, 40.09, 24.85, 21.71, 16.43, 17.03, 24.49, 21.48, 20.58,
       25.3 ,  9.25, 18.33, 14.4 , 19.08, 24.1 , 14.95, 18.07, 14.32,
       16.67, 16.1 , 23.49])
```

Model scatterplot of observed data



LISTS

create list with

in list

{listVariable}

get

#

FREESTYLE

type code here...

with fig do add_scatter using

create list with

x=

y=

mode="markers"

name="observed"

line=dict(color='royalblue')

with np do ravel using lstat

with np do ravel using

create list with

in list splits

get

#

4

list containing Xtrain, Xtest, Ytrain, and Ytest

x-axis

y-axis

numpy library

converts dataframe into a list [for fit parameters]

column of lower status values

Ytest

Figure() function of the plotly.graph_objects library

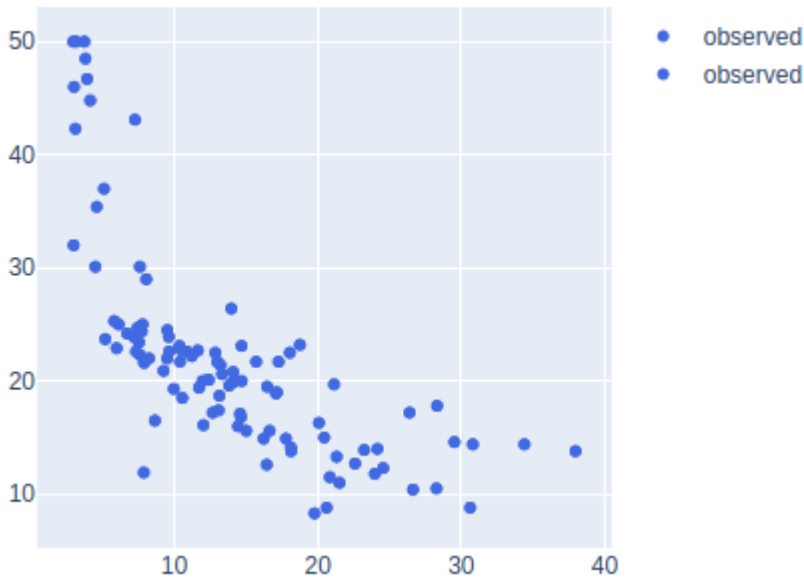
function that creates a scatter plot

representation of datapoints

name of datapoints

color of datapoints

Output:



Model scatterplot of predictions

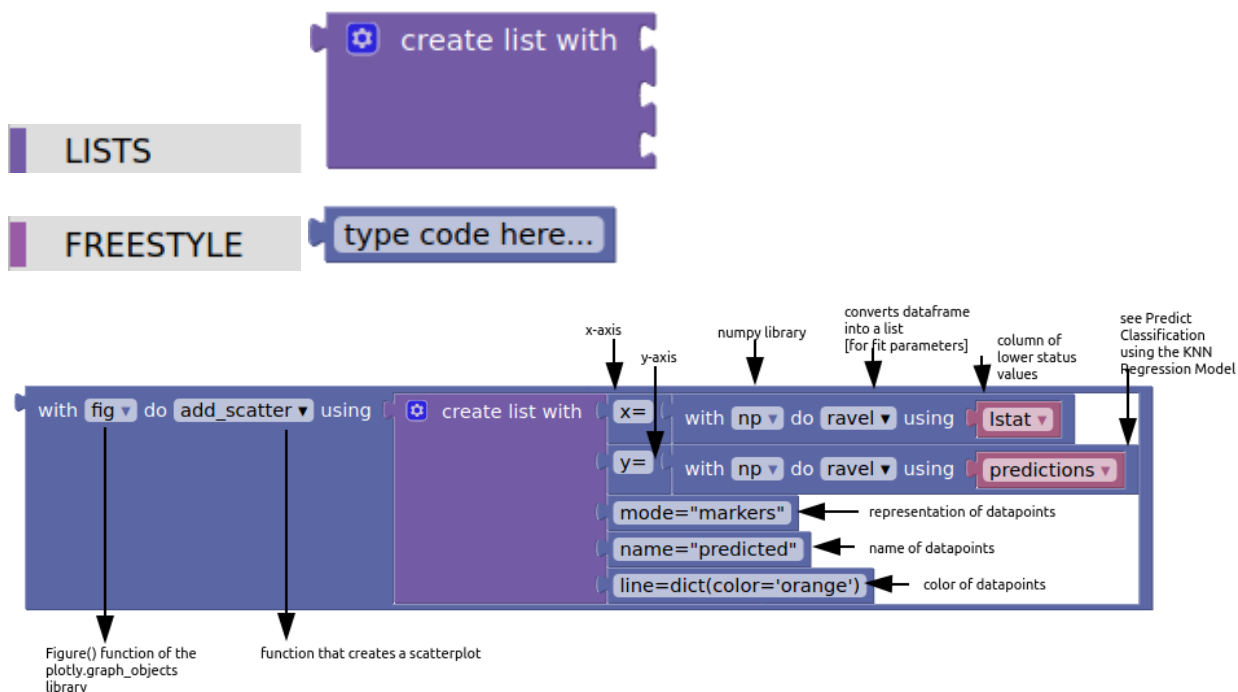
VARIABLES

variable

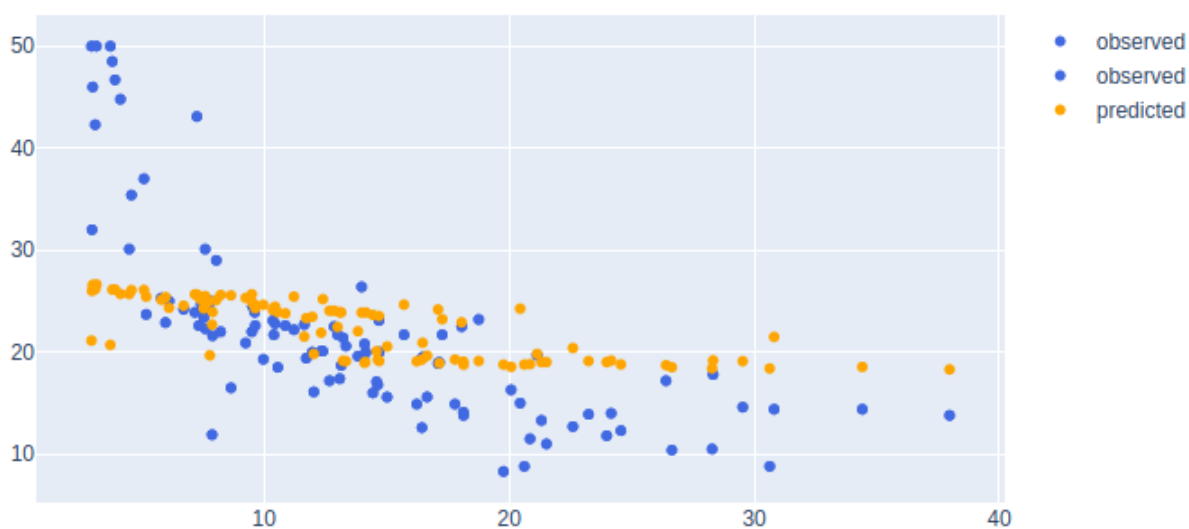
with variable do

!Not defined until you execute code.

using

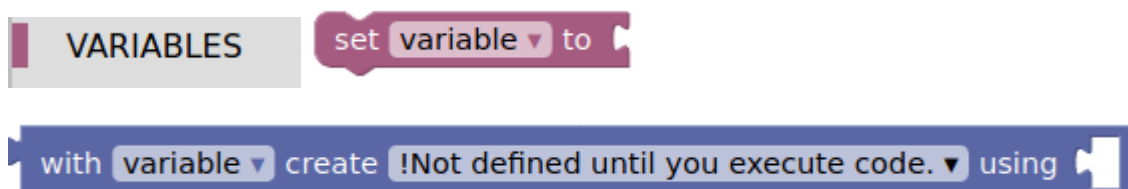


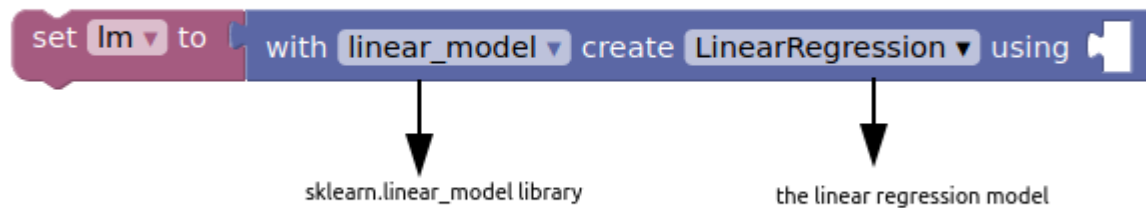
Output:



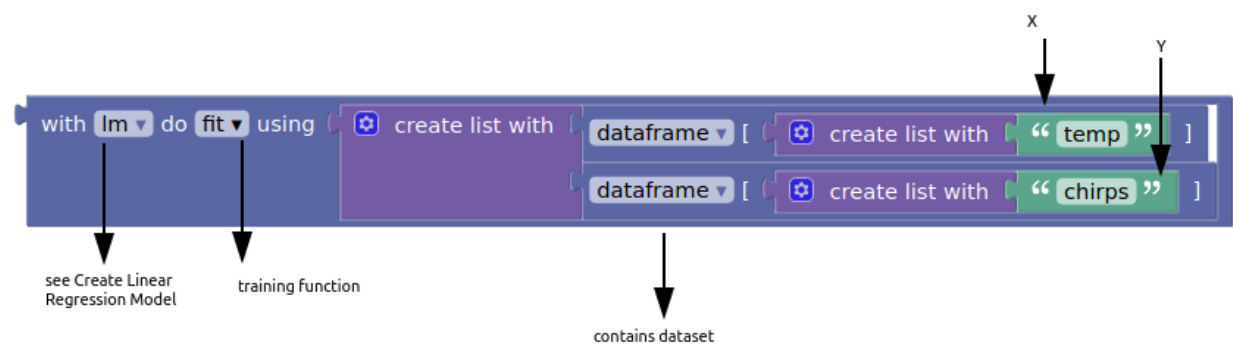
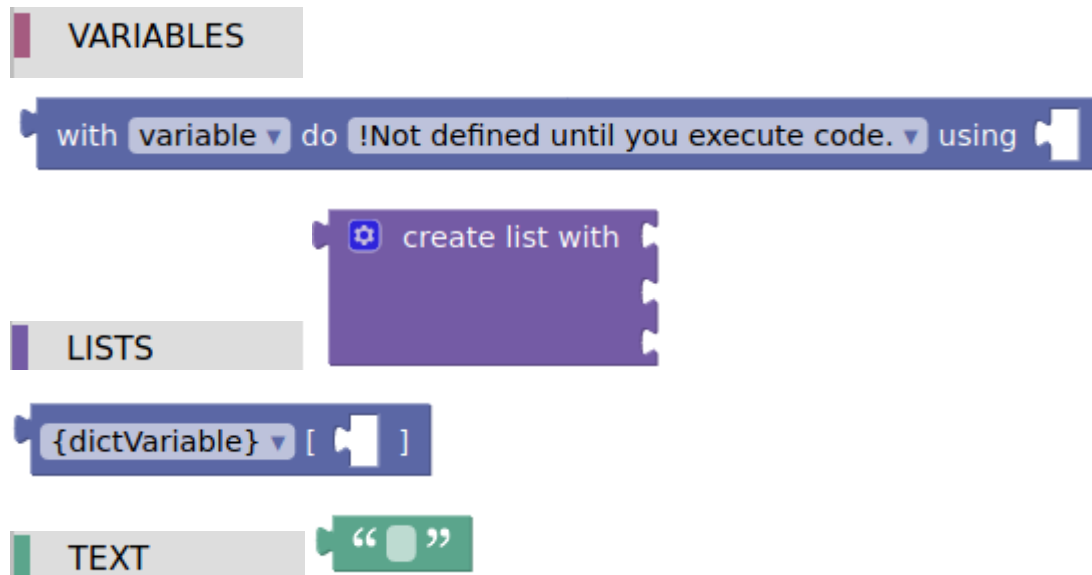
Simple linear regression

Define the linear regression model





Train the linear regression model



Output:

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

Multiple Linear Regression

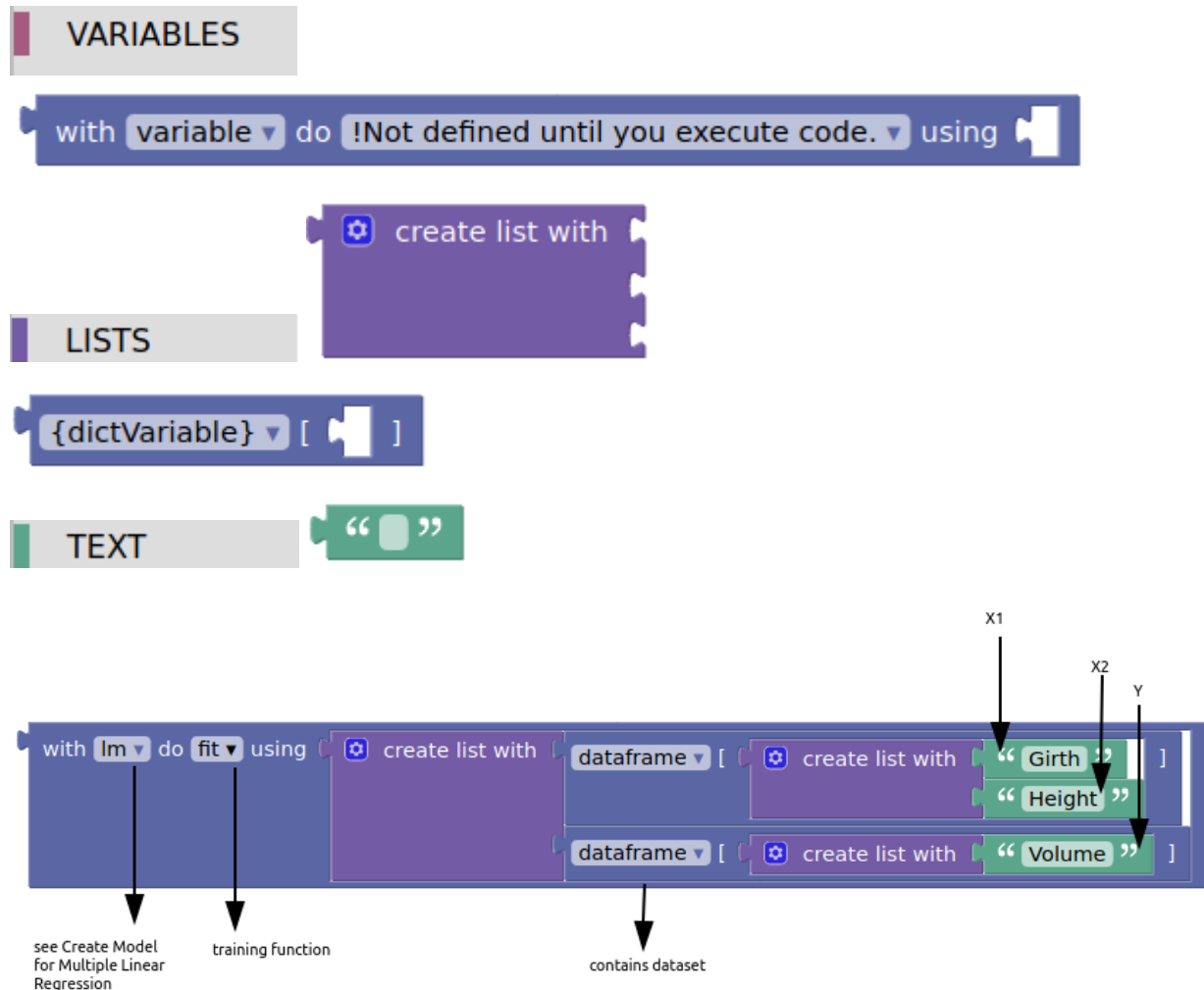
Define the Multiple Linear Regression Model

This command creates a multiple linear regression model

[See Create linear regression model; same command]

Train the Multiple Linear Regression Model

This command trains a multiple linear regression model.

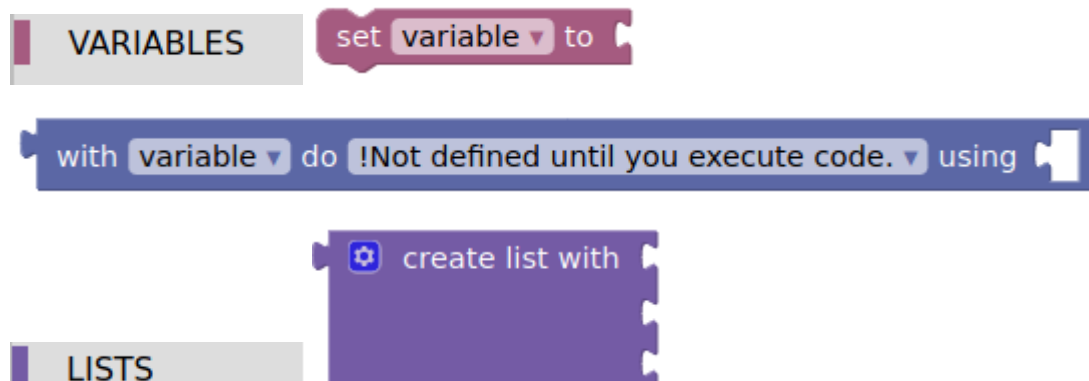


Output:

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

Get predictions from the mutiple linear regression model

This command gets predictions of data using a multiple linear regression model.



{dictVariable} []

TEXT “ ”

FREESTYLE type code here...

set dataframe to

with dataframe do

assign using

create list with predictions1=

with lm do

predict using

create list with dataframe

create list with “Girth” “Height”

dataframe to be added to

contains dataset

adds as a column to a dataframe

list of predicted data

see train Multiple Linear Regression

uses the model to predict new data

X1

X2

Output:

	Girth	Height	Volume	Type_plum	Type	predictions1
0	8.3	70.0	10.3	0	cherry	6.826375
1	8.6	65.0	10.3	0	cherry	6.493821
2	8.8	63.0	10.2	0	cherry	6.707254
3	10.5	72.0	16.4	0	cherry	17.006587
4	10.7	81.0	18.8	0	cherry	20.809891
...	...	...	...	...	...	...
57	17.9	69.2	47.1	1	plum	48.139871
58	18.8	67.3	50.5	1	plum	51.417419
59	19.7	67.4	55.6	1	plum	55.347671
60	17.2	67.3	42.3	1	plum	44.488323
61	21.1	73.8	69.8	1	plum	63.499282

Check linearity and equal variance

This command checks the linearity and equal variance of a multiple linear regression model.

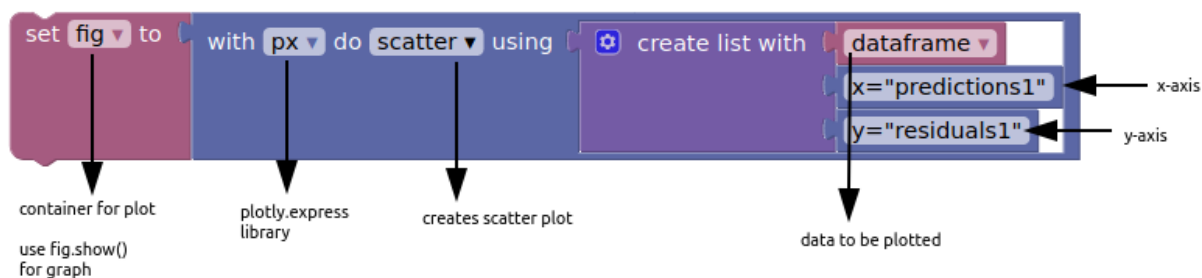
VARIABLES variable

set variable to

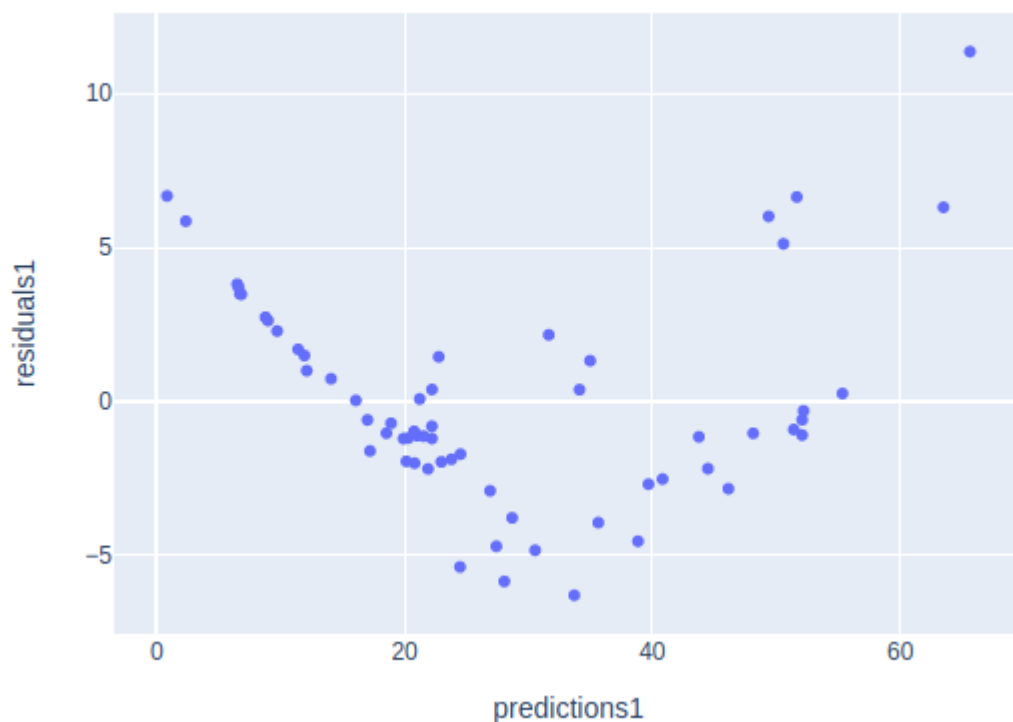
create list with

LISTS

FREESTYLE type code here...



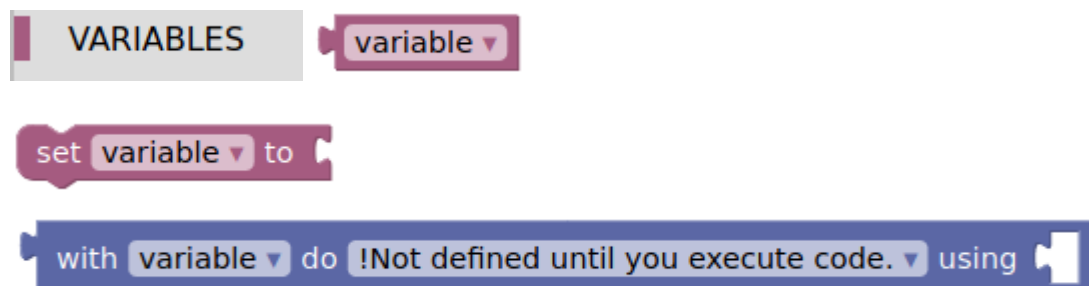
Output:

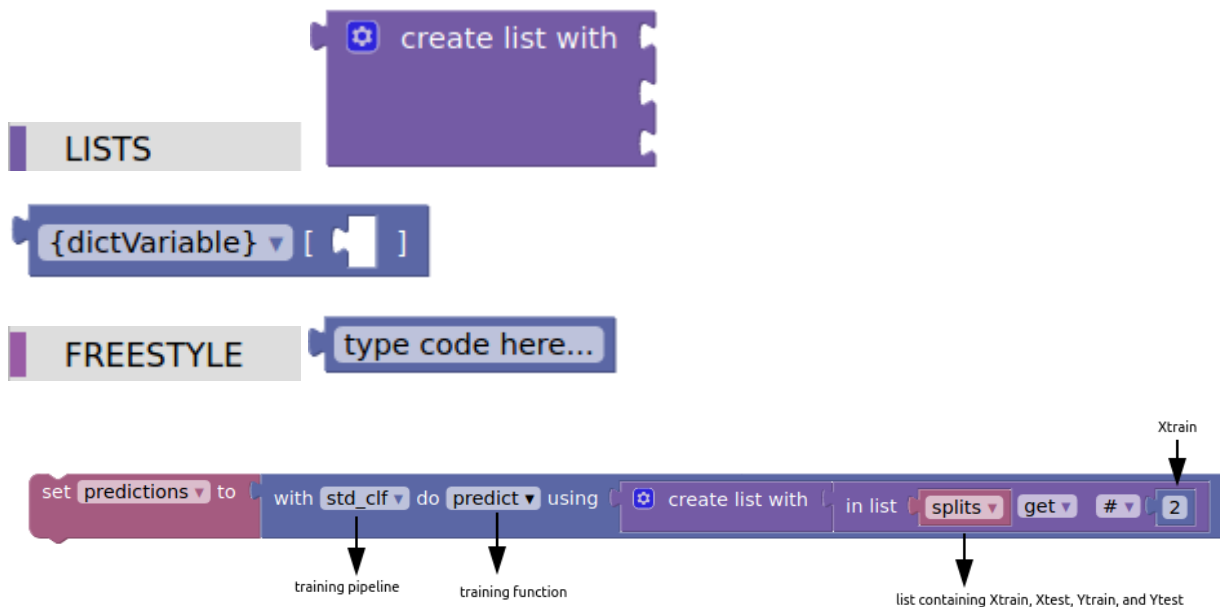


Logistic Regression

Define the Logistic regression model

This function creates the logistic regression model.





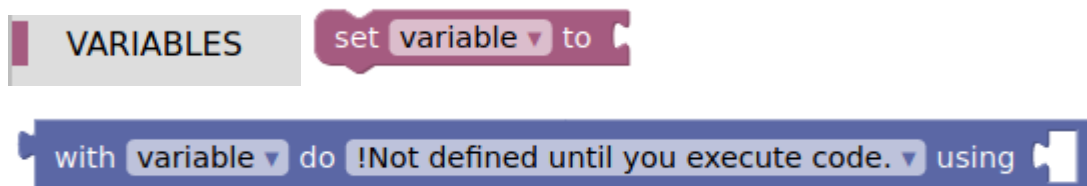
Output:

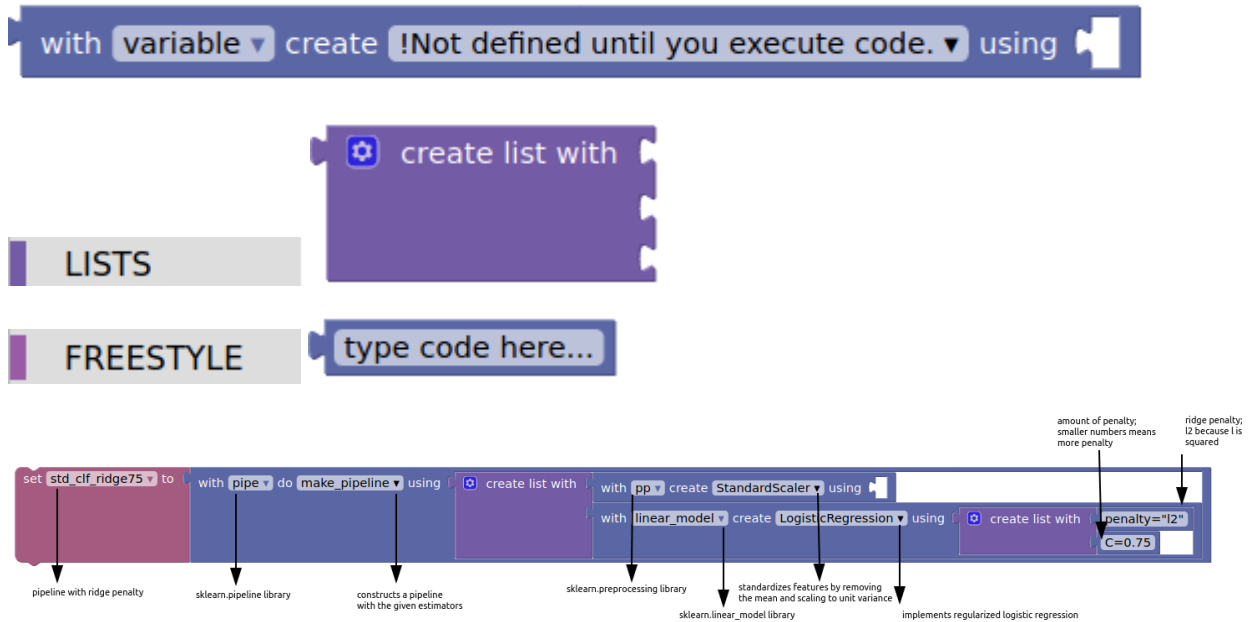
```
array([0., 1., 1., 0., 1., 0., 0., 0., 0., 1., 0., 0., 0.,
       0., 0., 0., 1.,
       0., 0., 0., 0., 1., 0., 1., 0., 0., 0., 0., 0.,
       0., 0., 0., 0.,
       0., 0., 0., 1., 1., 1., 0., 0., 0., 0., 0., 0.,
       1., 0., 1., 1.,
       0., 0., 0., 0., 0., 0., 1., 1., 0., 0., 0., 0.,
       1.,
       0., 1., 0., 1.,
       0., 0., 0., 0., 0., 1., 0., 1., 1., 0., 0., 0.,
       0.,
       1., 1., 0., 0.,
       0., 0., 1., 1., 0., 0., 1., 1., 0., 0., 0., 1.,
       0.,
       0., 1., 0., 0.,
       1., 1., 0., 1., 0., 1., 0., 0., 1., 0., 0., 0.,
       0.,
       0., 0., 1., 1.,
       0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0.,
       0.,
       1., 0., 0., 0.,
       0., 0., 0., 0., 1., 0., 0., 0., 0., 1., 0., 0.,
       0.,
       1., 0., 1., 0.,
       0.], dtype=float32)
```

Ridge and Lasso Regression

Train the logistic regression model with ridge penalty

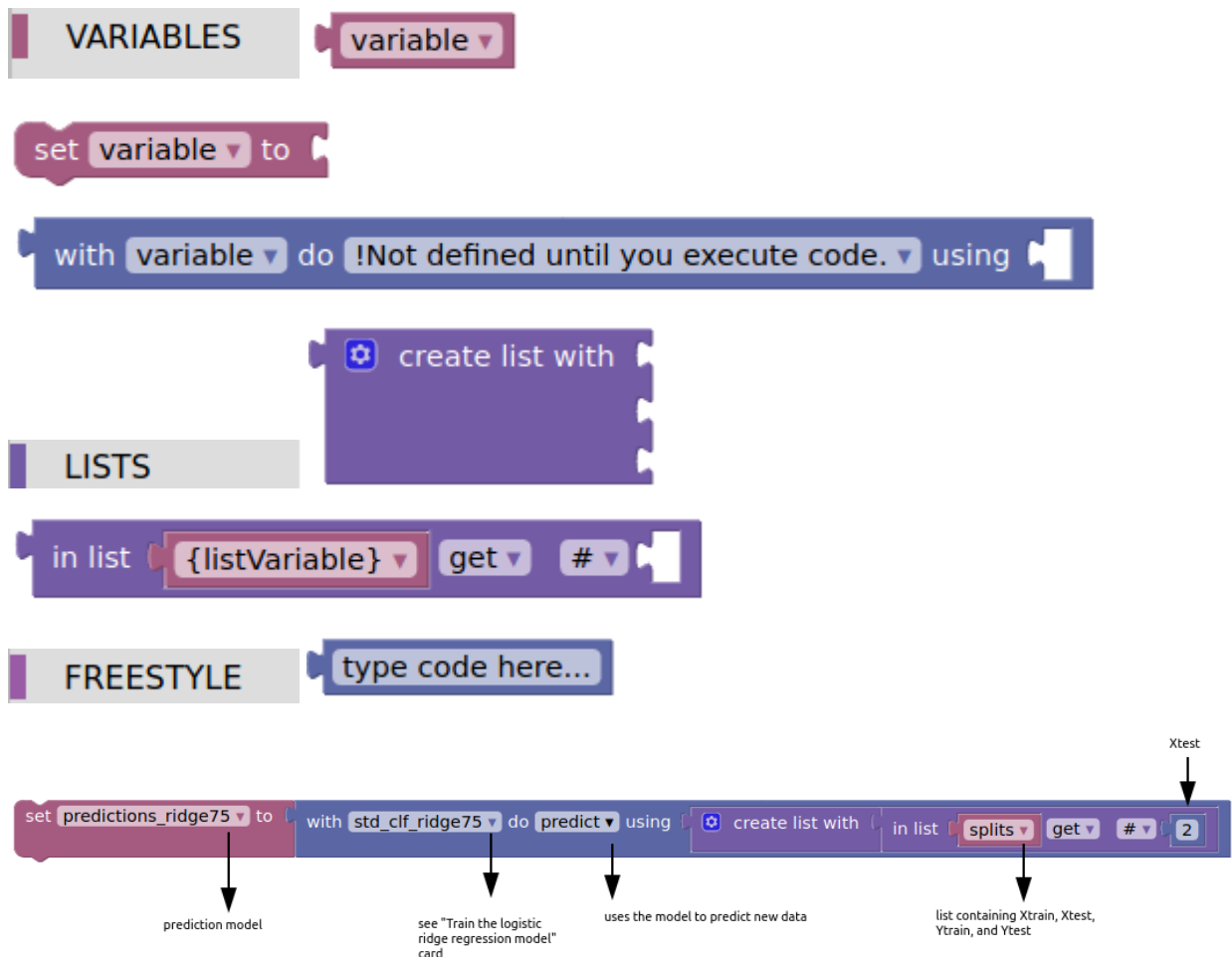
This training model makes a pipeline so that the model can be scaled and trained in one step.





Get predictions from the ridge regression model

This command uses the lasso model to make predictions about the data it is given.



Train the logistic regression model with lasso penalty

This command creates a logistic regression model with a lasso penalty.

VARIABLES

set **variable** to

with **variable** do !Not defined until you execute code. using

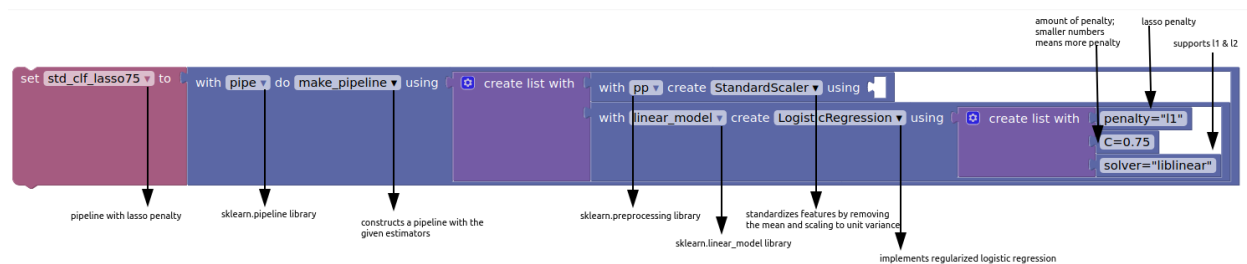
with **variable** create !Not defined until you execute code. using

LISTS

create list with

FREESTYLE

type code here...



Get predictions from the lasso regression model

This command uses the logistic regression model with lasso penalty to make predictions about the data it is given.

VARIABLES

variable

set **variable** to

with **variable** do !Not defined until you execute code. using

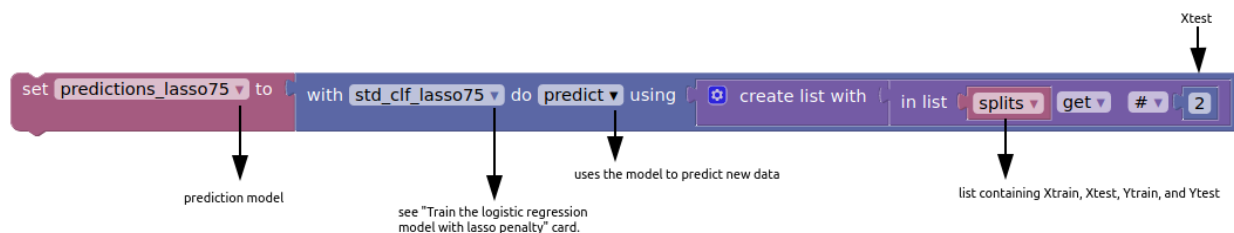
create list with

LISTS

in list {listVariable} get #

FREESTYLE

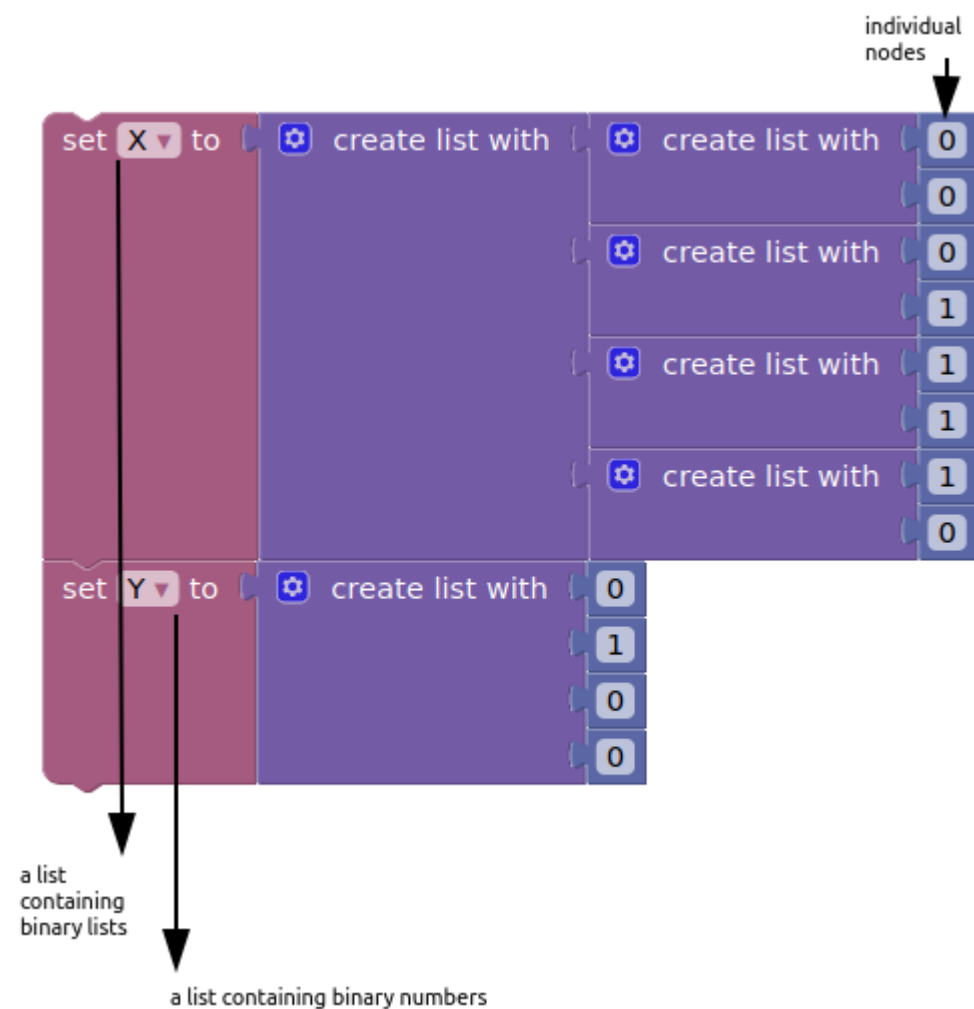
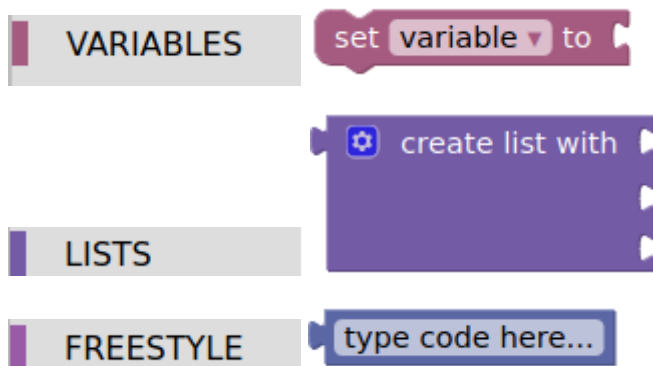
type code here...



Decision Trees

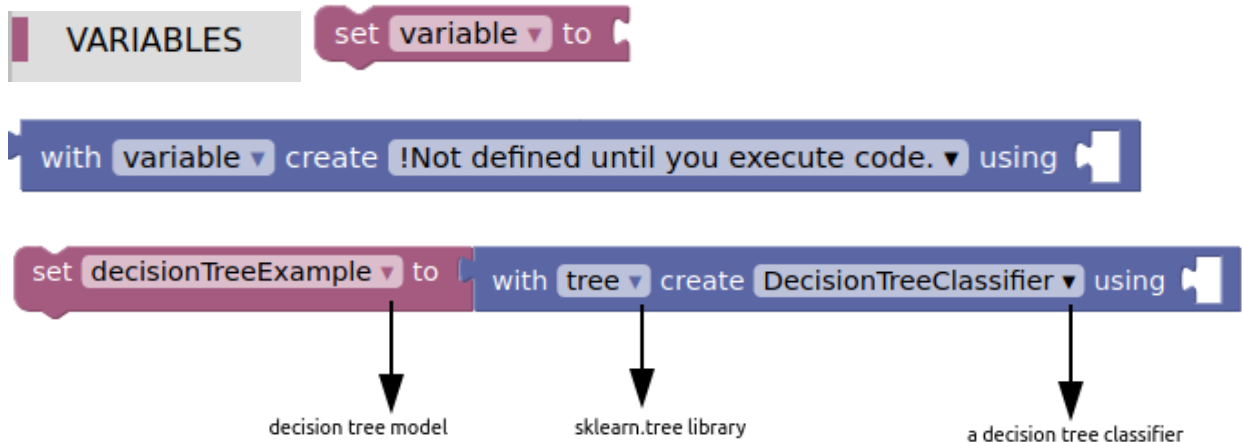
Build a decision tree

This command builds a generic decision tree.



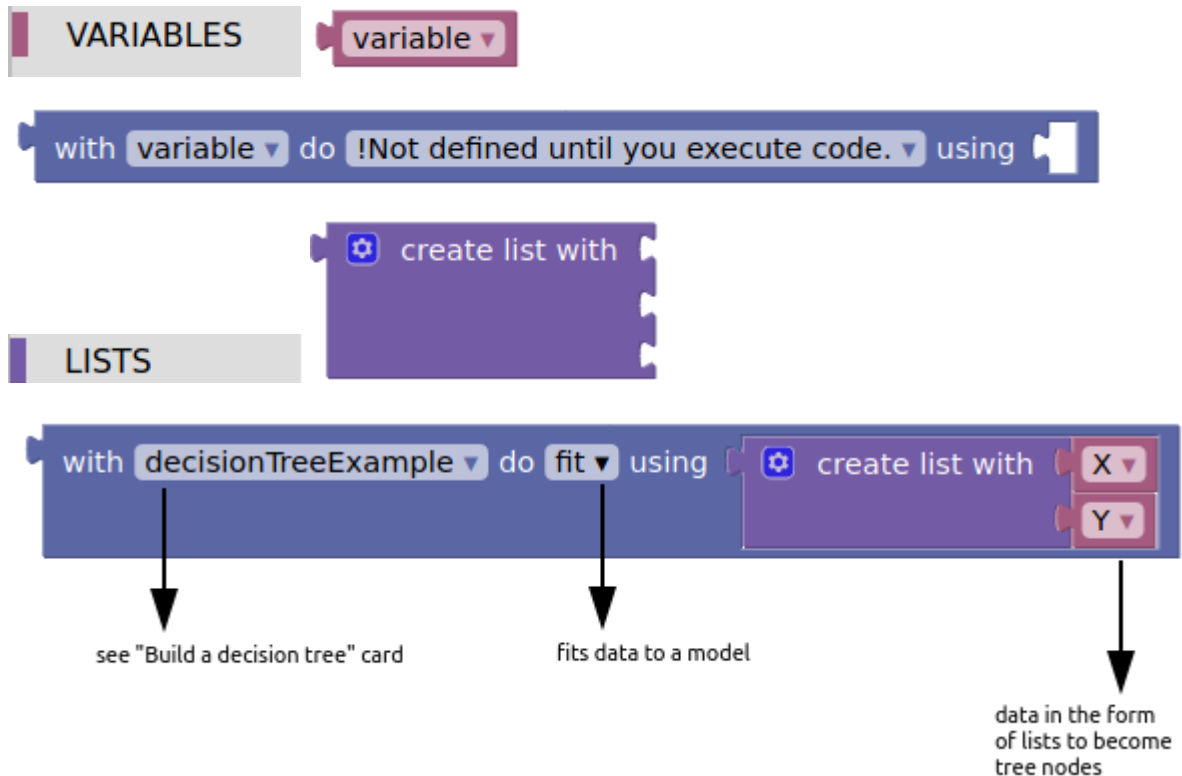
Create the decision tree model

This command creates a generic decision tree model



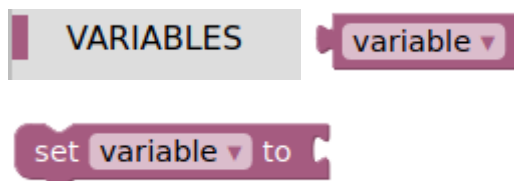
Fit the decision tree model

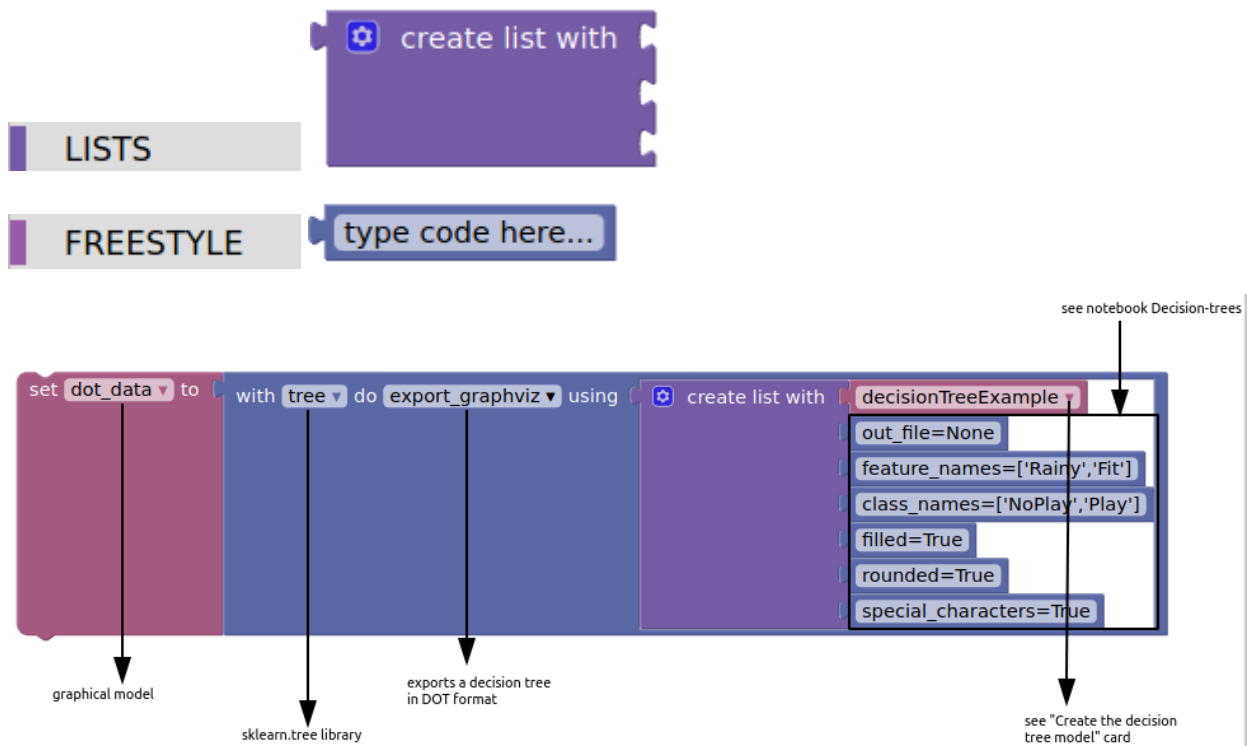
This command fits a generic decision tree into the model.



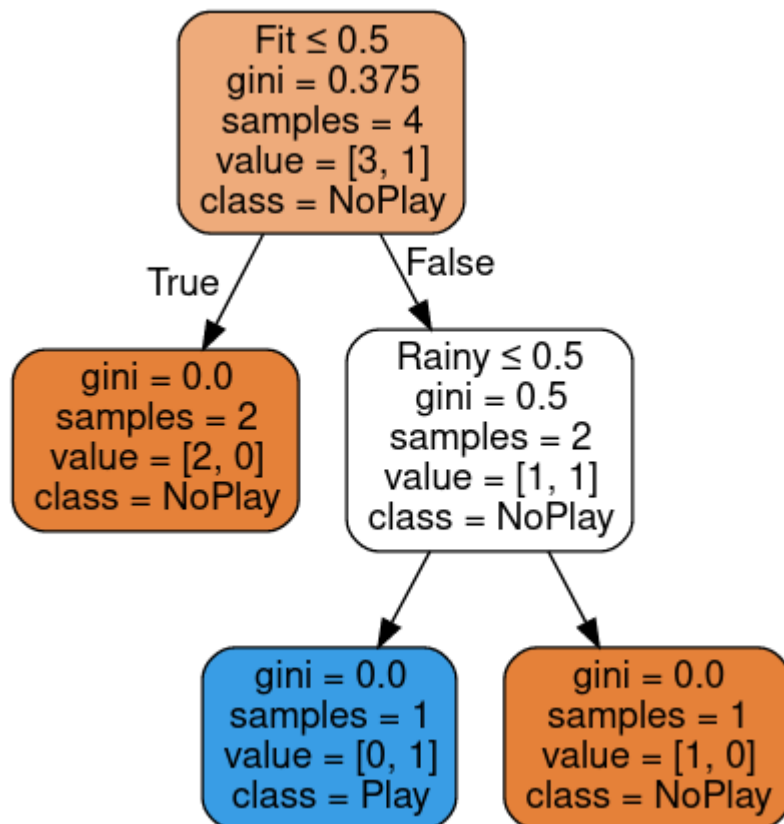
Visualize the decision tree model & interpret algorithm fit

This command visualizes the model and displays algorithm interpretation for each node.





Output:

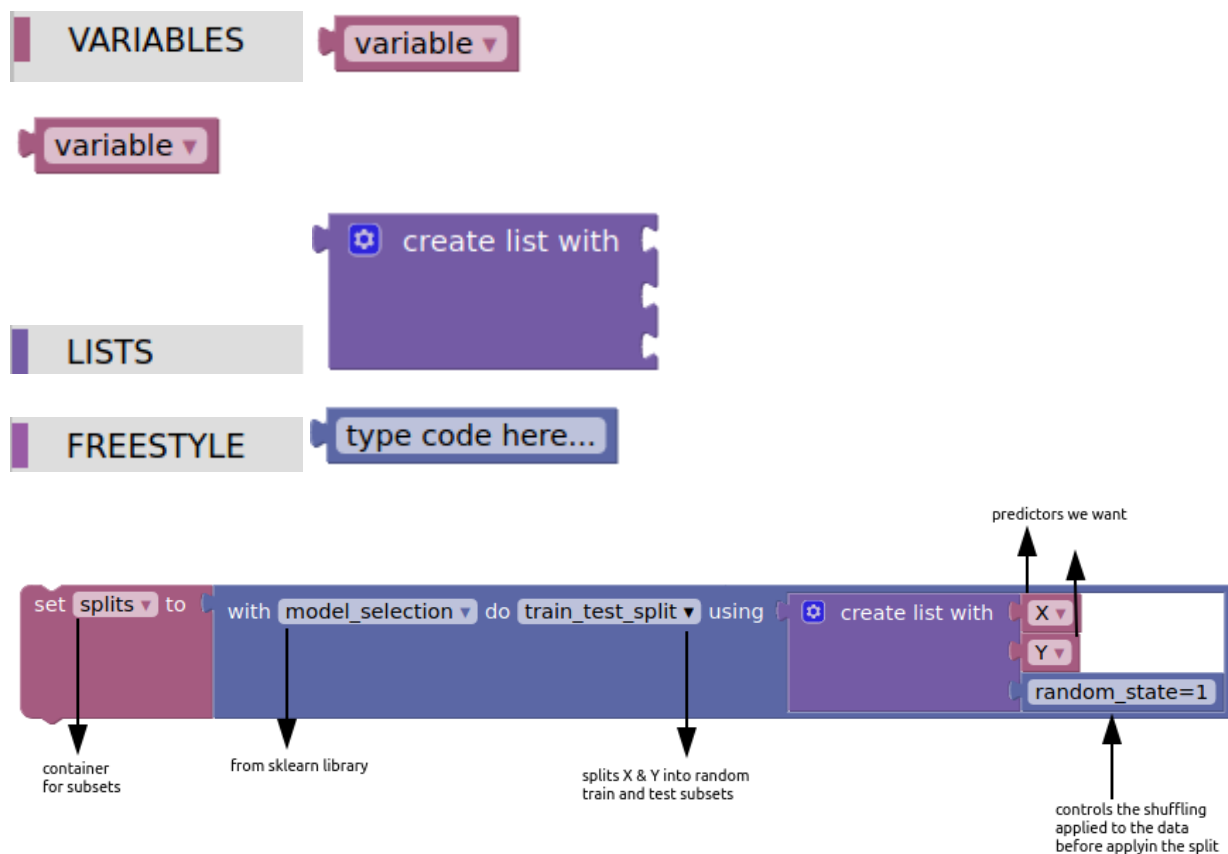


Regression Trees

Prepare train/test sets

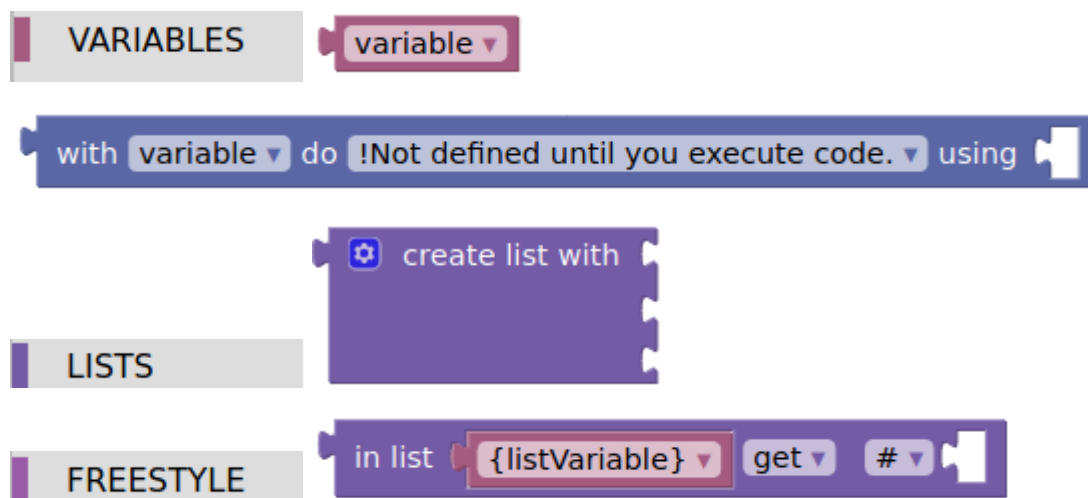
After setting X and Y to dataframes containing the two predictors we want, train and test sets can

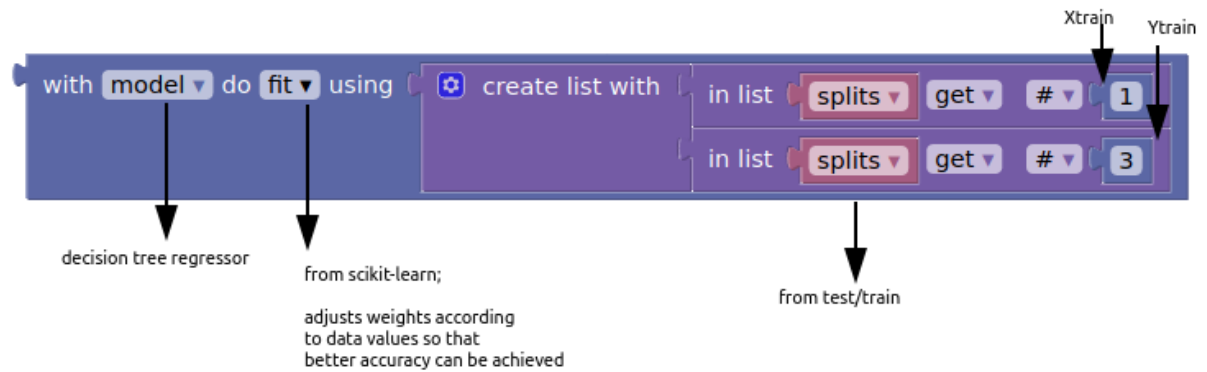
be split.



Fit the model

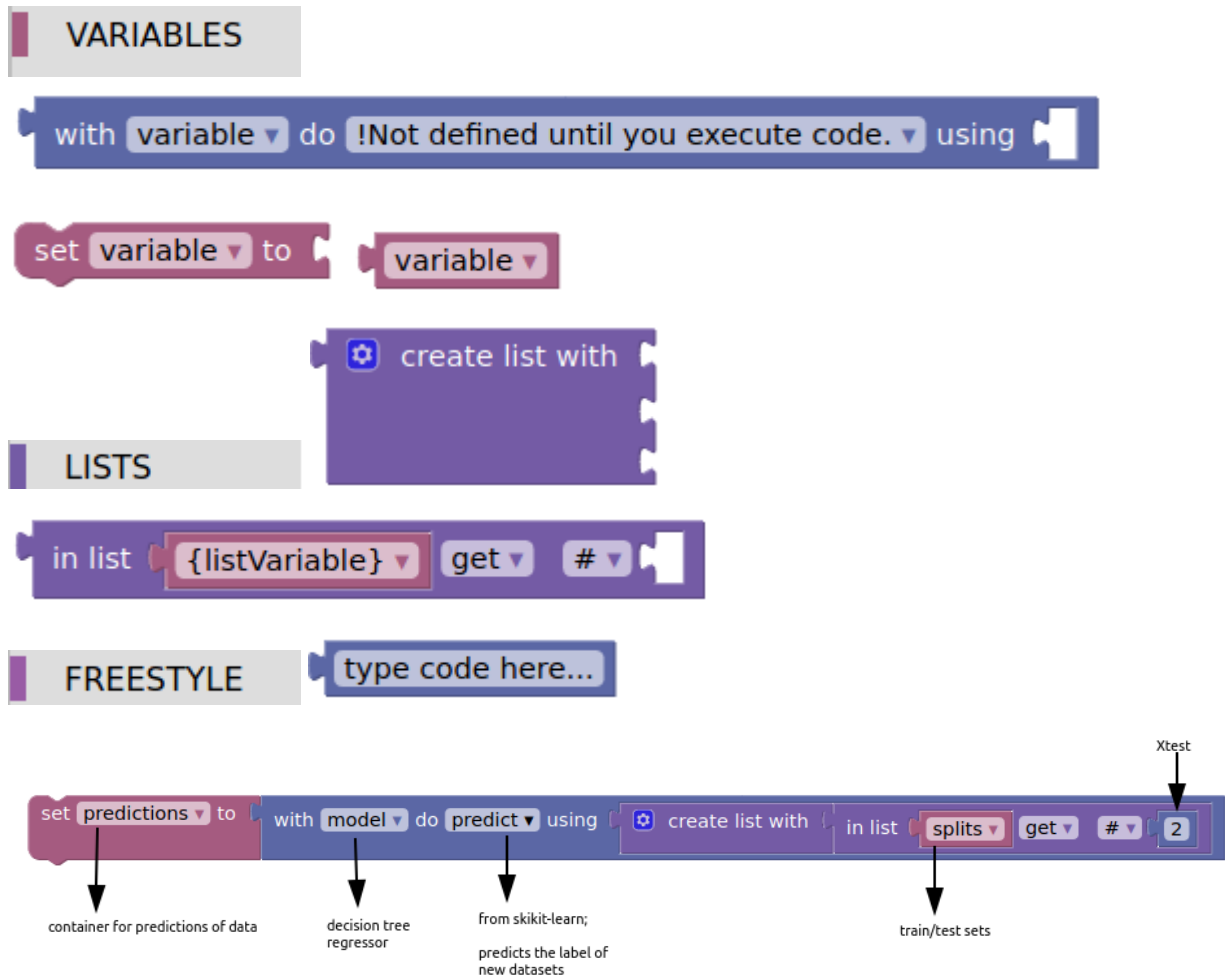
Fit the regression tree model with your data





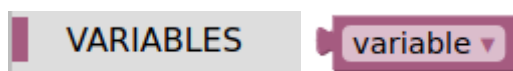
Get predictions

Predict the label of a new set of data using the model you just trained



Evaluate the model

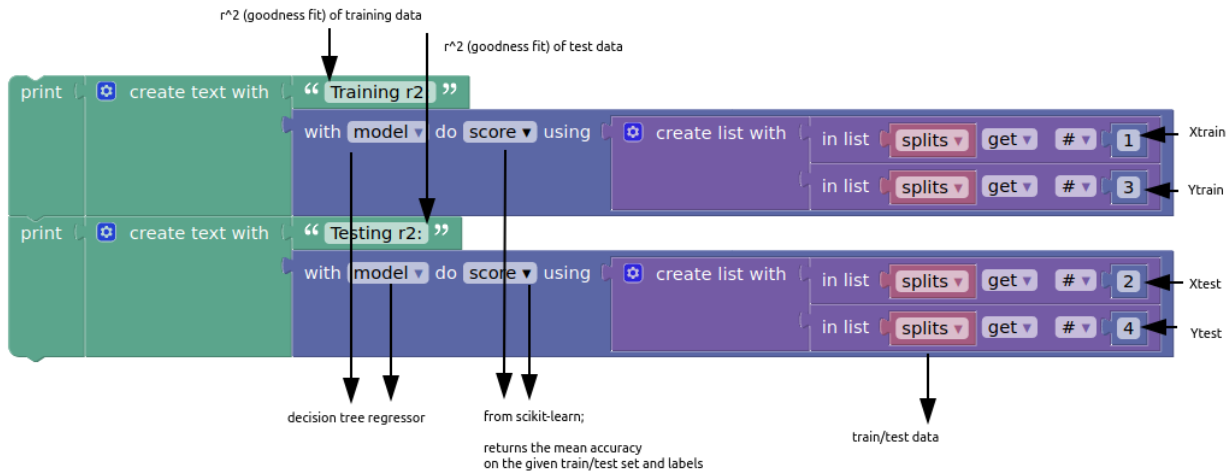
Evaluate your regression tree model trained on given data using the r^2 formula, which gives a goodness-of-fit estimation



with variable do !Not defined until you execute code. using

TEXT “ ”

FREESTYLE type code here...



Output:

Training r2:0.7228964338418474
Testing r2:0.5996223347282654

Visualize the model

Put data into a graphically visualized information tree

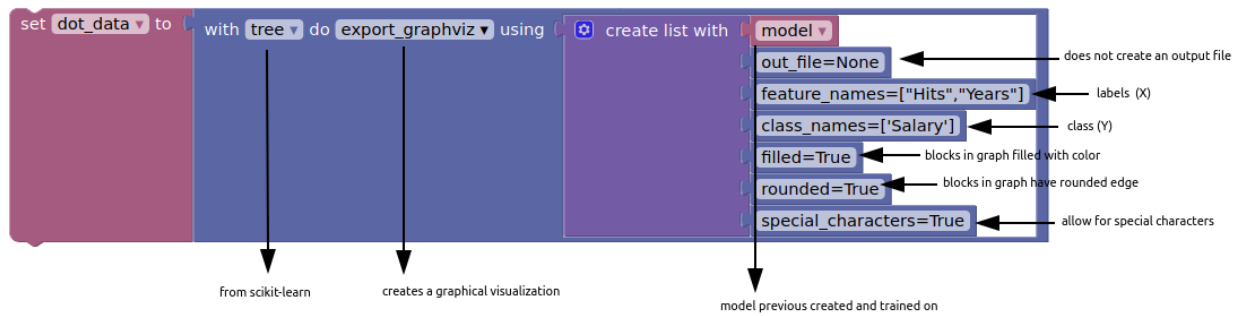
VARIABLES

with variable do !Not defined until you execute code. using

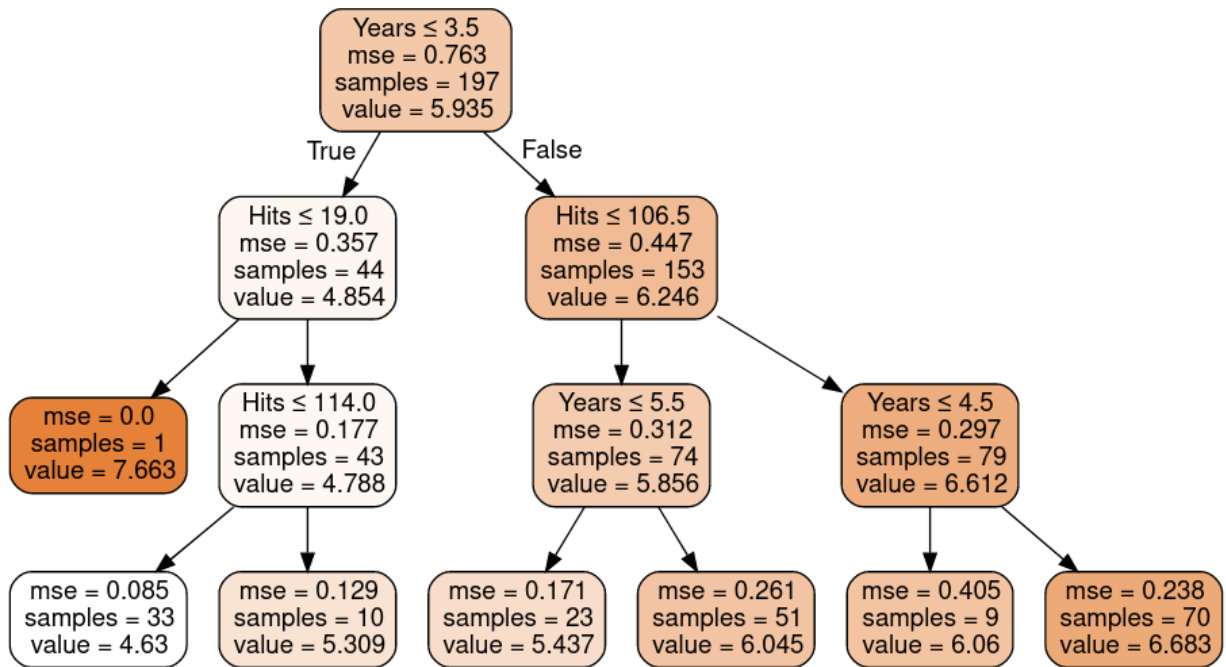
set variable to variable

LISTS create list with

FREESTYLE type code here...

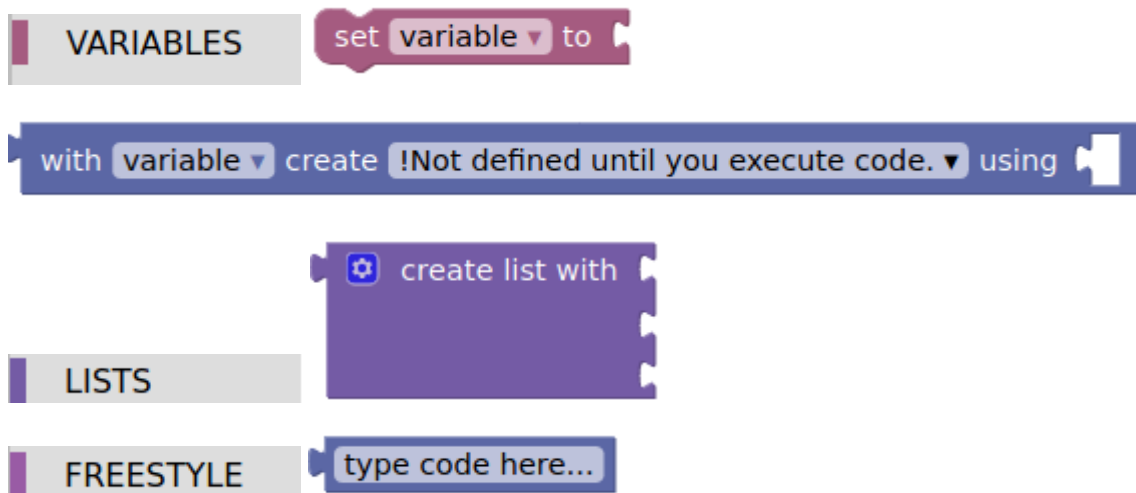


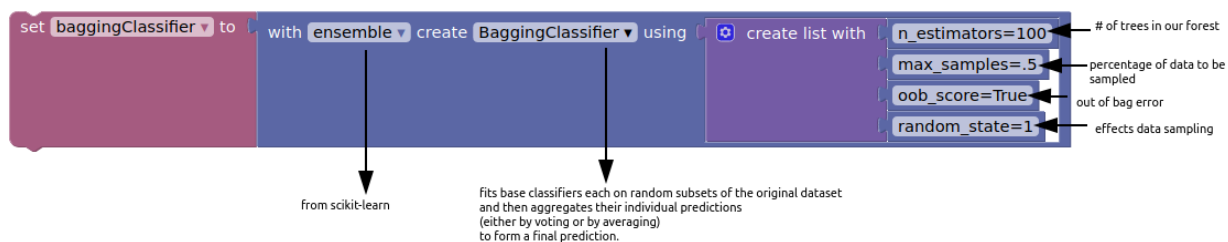
Output:



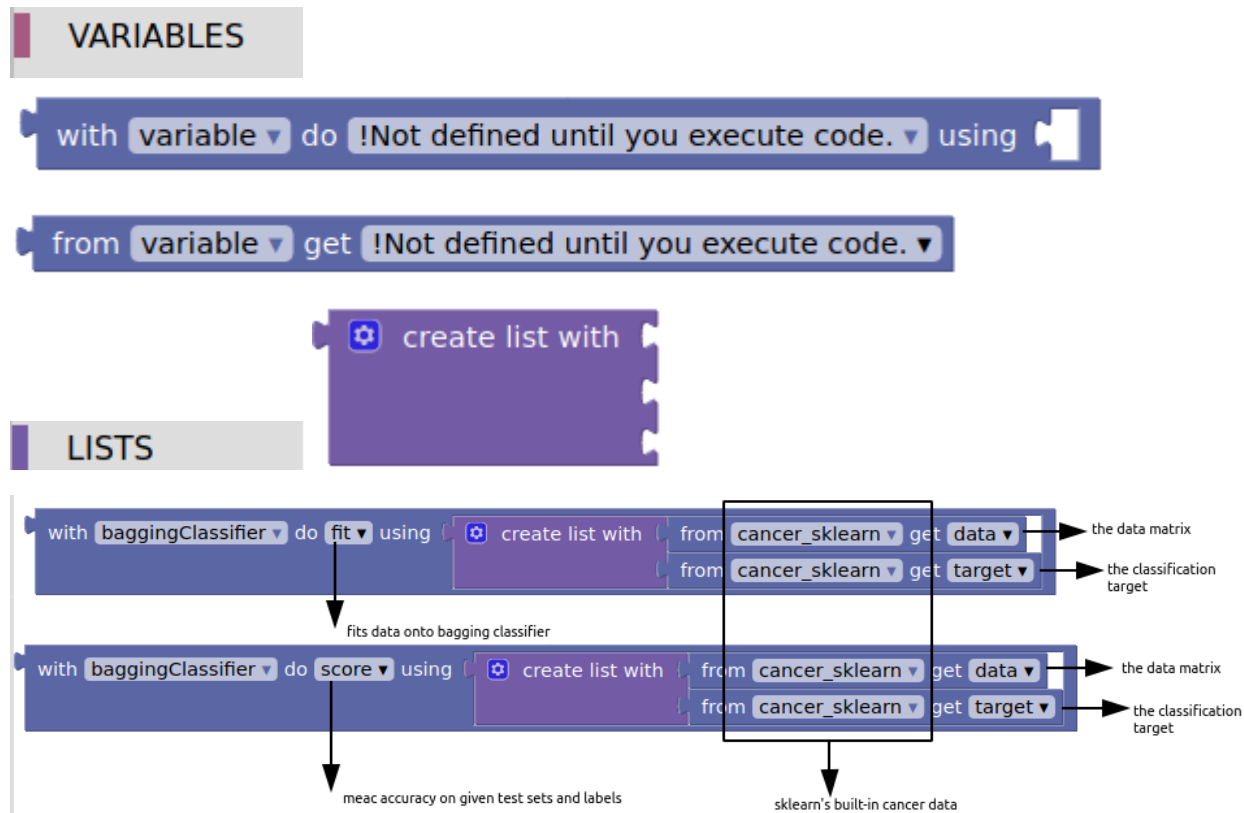
Random Forests

Create bagging classifier





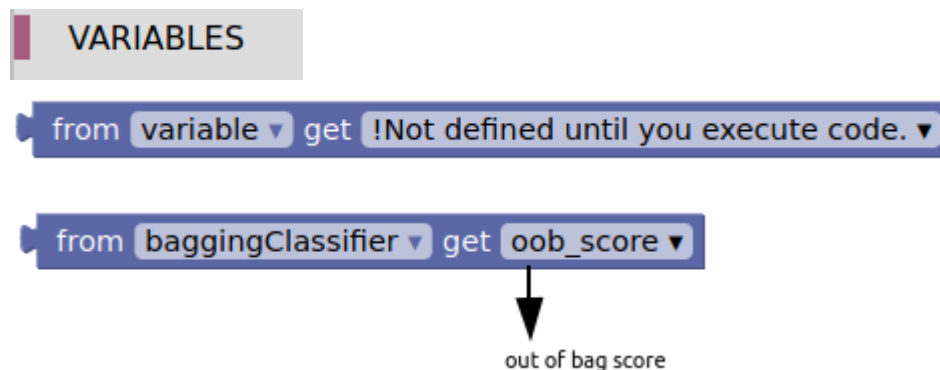
Fit and score bagging classifier



Output:

0.9912126537785588

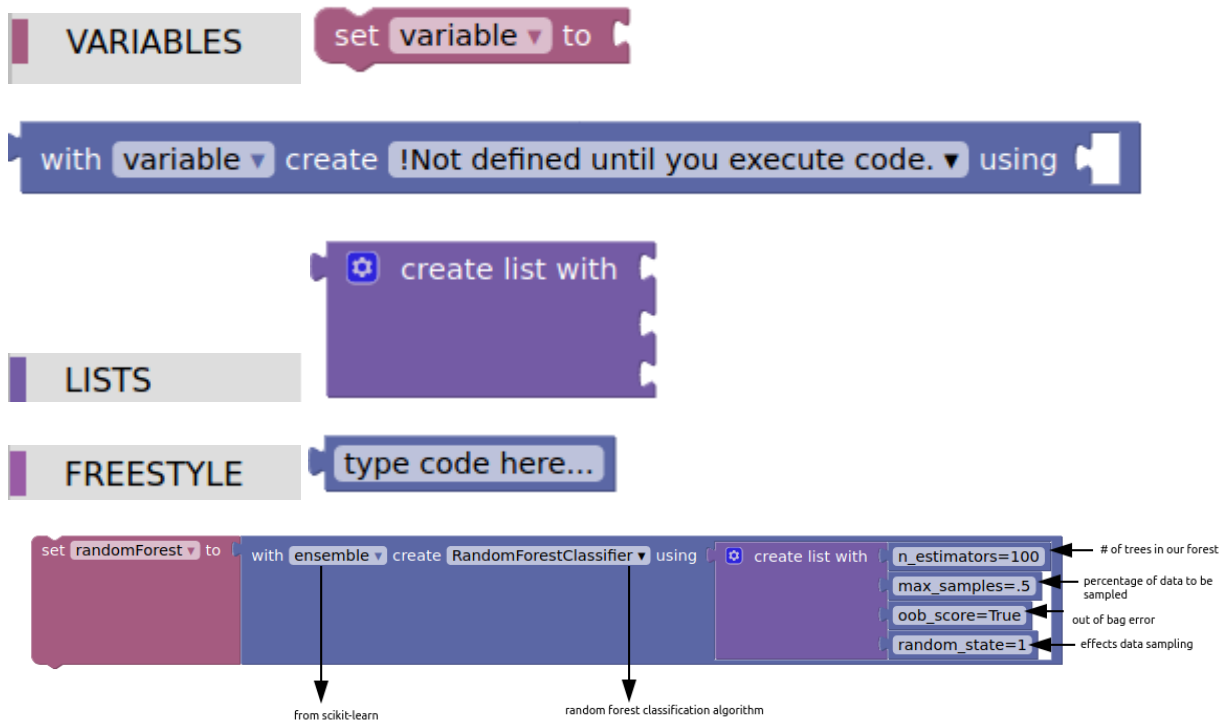
Get out of bag (OOB) score



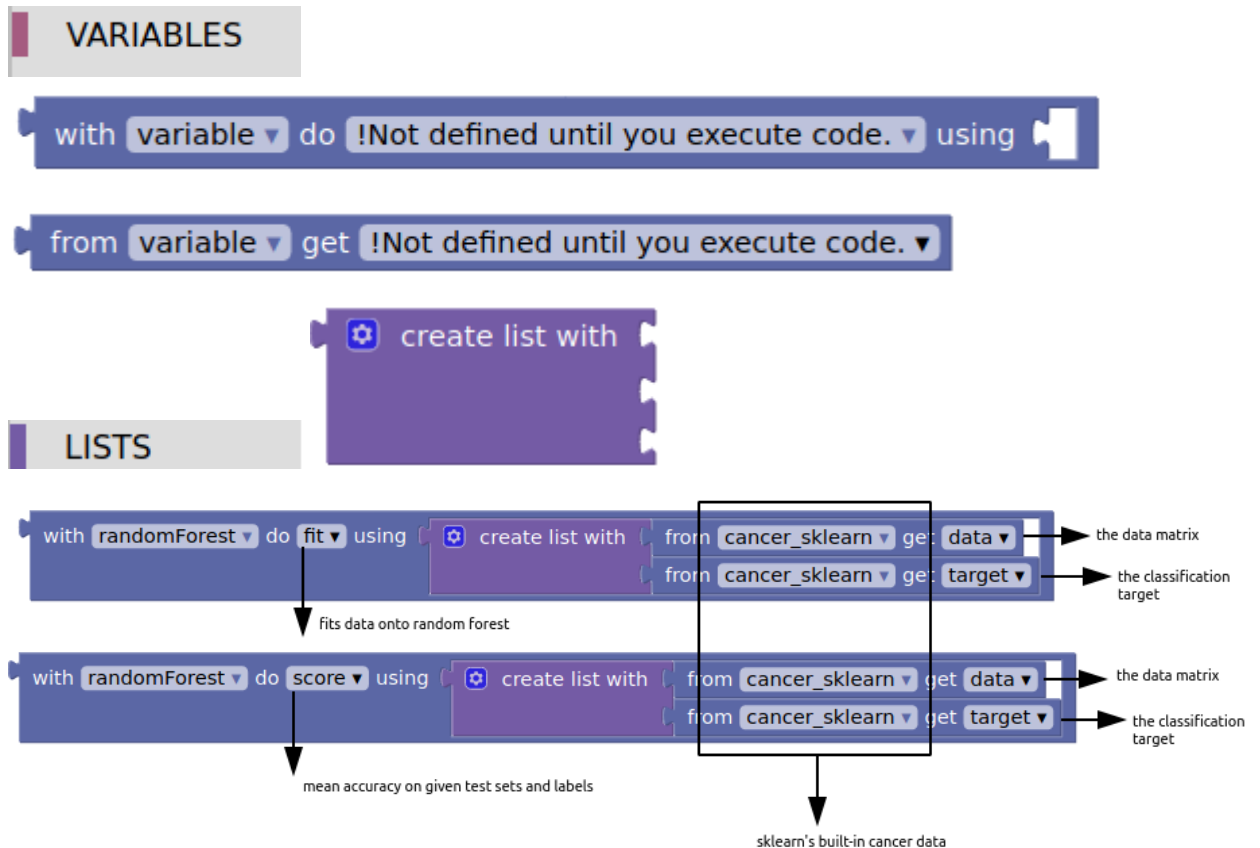
Output:

0.9648506151142355

Create random forest classifier



Fit and score random forest classifier



Output:

0.9929701230228472

Visualize ensemble model

VARIABLES

with variable ▾ do !Not defined until you execute code. ▾ using

from variable ▾ get !Not defined until you execute code. ▾

LISTS

create list with

FREESTYLE

type code here...

with px ▾ do bar ▾ using

create list with

x= from cancer_sklearn ▾ get feature_names ▾

y= from randomForest ▾ get feature_importances_ ▾

sklearn's built-in cancer data

names of dataset columns

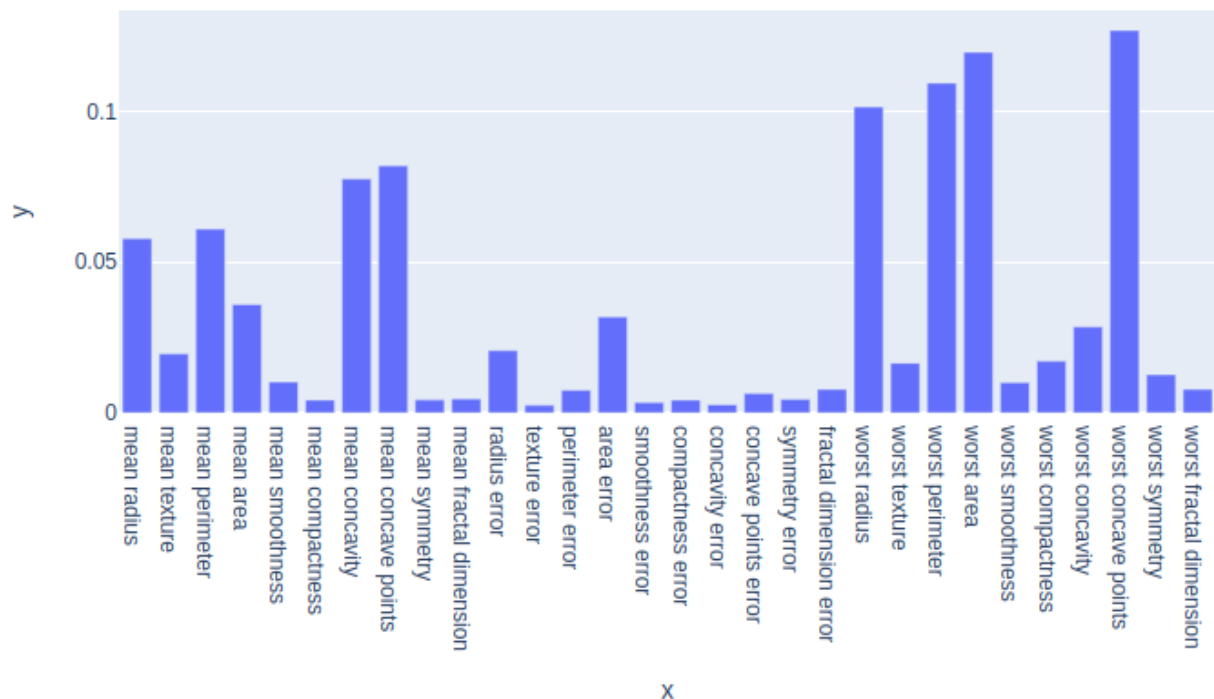
random forest tree

feature importances

plotly express

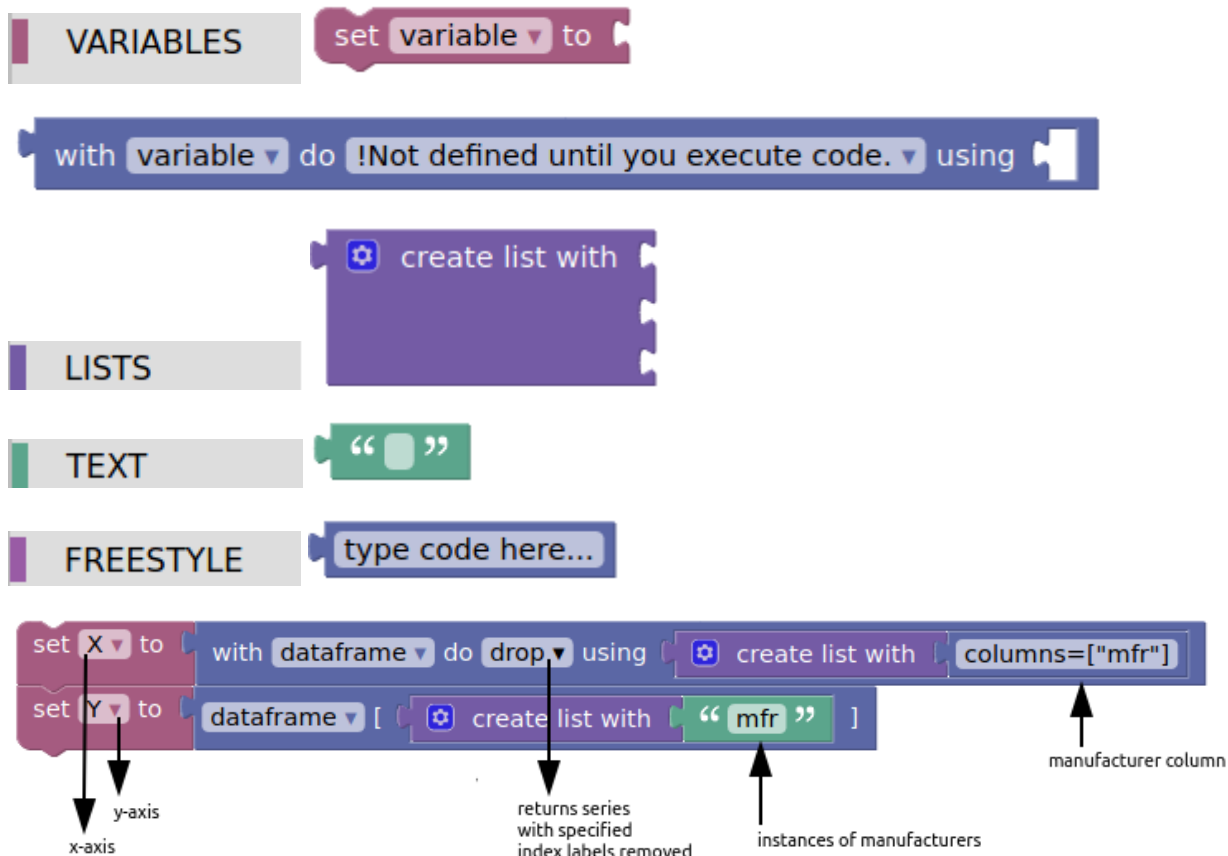
plot as bar graph

Output:

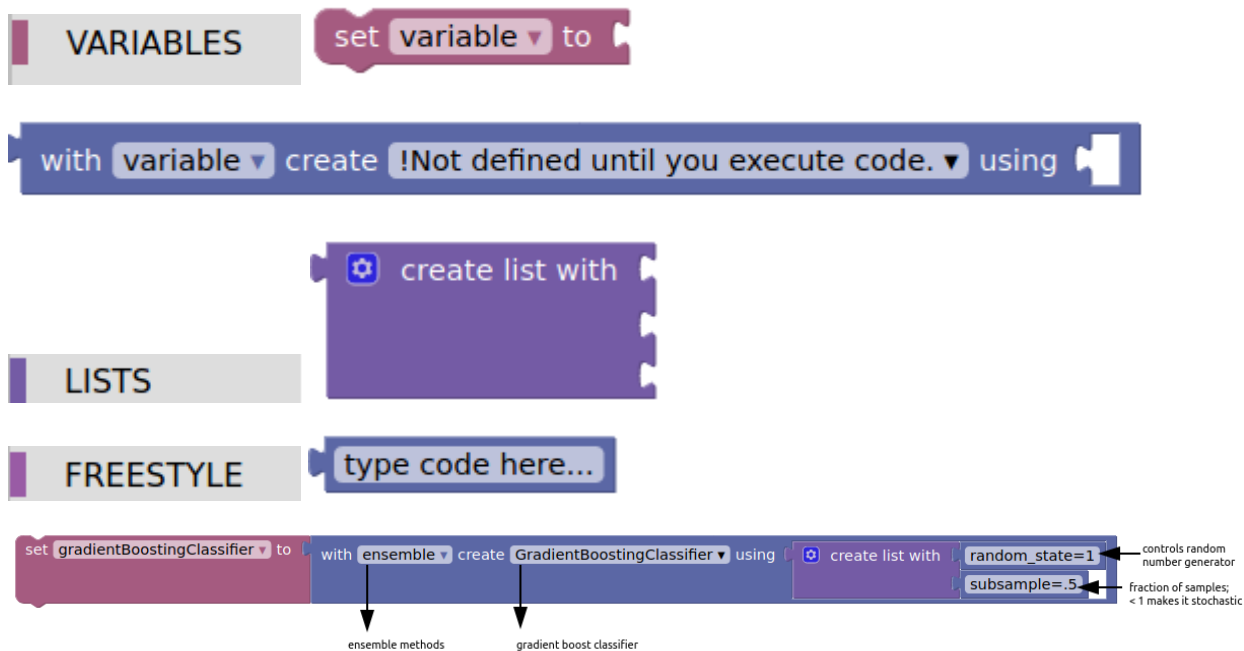


Gradient Boosting

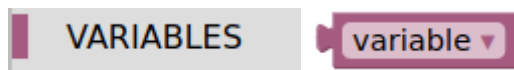
Prepare train/test sets

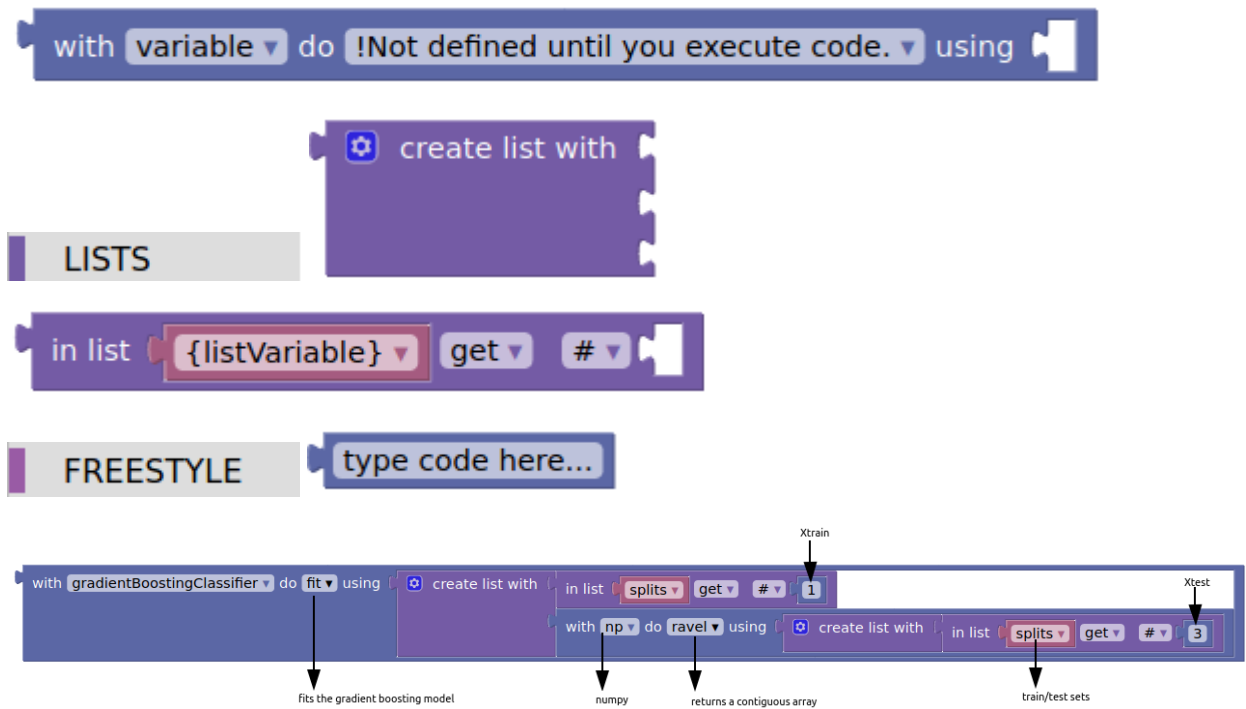


Create gradient boosting classifier



Fit the classifier

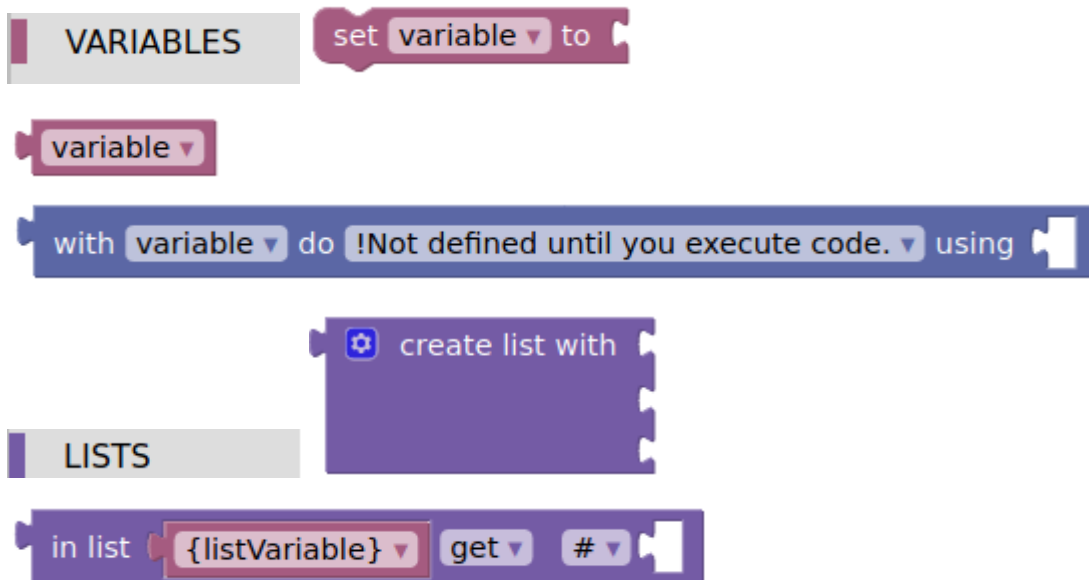




Output:

```
GradientBoostingClassifier(ccp_alpha=0.0, criterion='friedman_mse', init=None,
                           learning_rate=0.1, loss='deviance', max_depth=3,
                           max_features=None, max_leaf_nodes=None,
                           min_impurity_decrease=0.0, min_impurity_split=None,
                           min_samples_leaf=1, min_samples_split=2,
                           min_weight_fraction_leaf=0.0, n_estimators=100,
                           n_iter_no_change=None, presort='deprecated',
                           random_state=1, subsample=0.5, tol=0.0001,
                           validation_fraction=0.1, verbose=0,
                           warm_start=False)
```

Get predictions



FREESTYLE

type code here...

set predictionsGradientBoostingClassifier to with gradientBoostingClassifier do predict using create list with in list splits get # 2

predicts output

train/test sets

Ytrain

Evaluate the model

VARIABLES

variable

with variable do !Not defined until you execute code. using

create list with

LISTS

in list {listVariable} get #

FREESTYLE

type code here...

print with metrics do classification_report using create list with in list splits get # 4 predictionsGradientBoostingClassifier

sklearn.metrics

gets report of classification output performance measures

train/test set

Ytest

Output:

	precision	recall	f1-score	support
G	0.56	0.83	0.67	6
K	0.40	0.33	0.36	6
N	0.00	0.00	0.00	1
P	0.00	0.00	0.00	2
Q	1.00	0.50	0.67	2
R	1.00	0.50	0.67	2
accuracy			0.47	19
macro avg	0.49	0.36	0.39	19
weighted avg	0.51	0.47	0.47	19

Visualize results

VARIABLES

with **variable** do !Not defined until you execute code. using

from **variable** get !Not defined until you execute code.

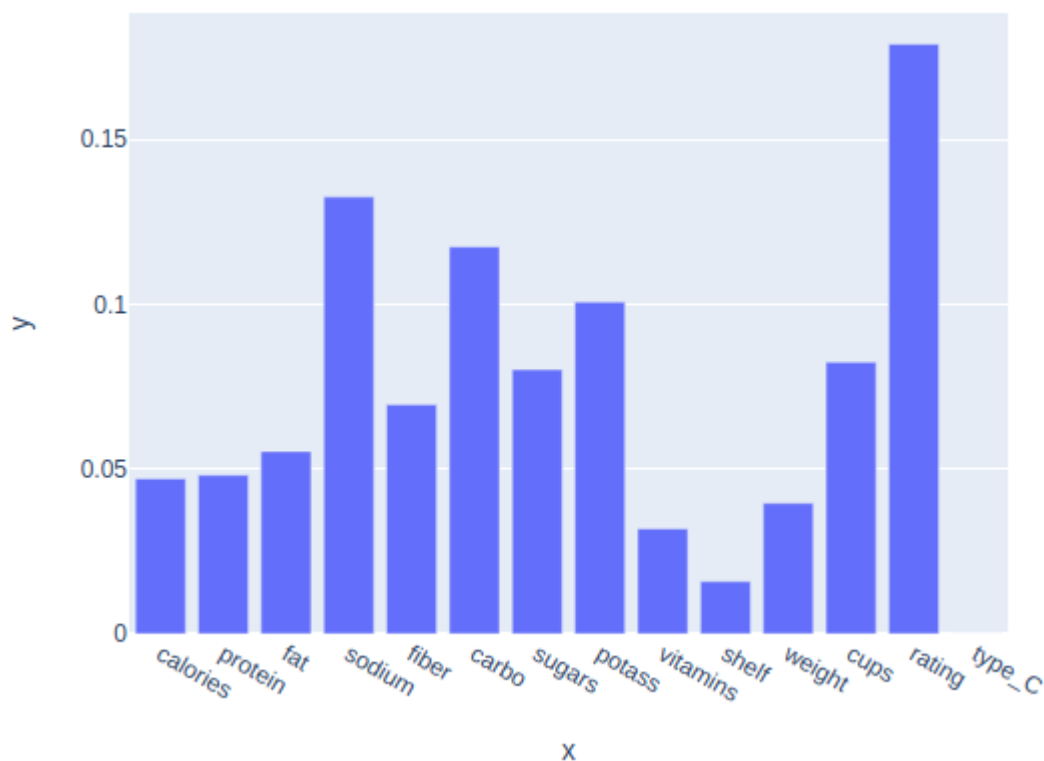
LISTS

FREESTYLE type code here...

with **px** do **bar** using **create list with** **x=** from **X** get **columns** **y=** from **gradientBoostingClassifier** get **feature_importances_**

plotly express plots bar graph x-axis y-axis column names feature importances of the gradient boosting classifier

Output:



Detecting overfitting with deviance

VARIABLES

with **variable** do !Not defined until you execute code. using

LISTS

FREESTYLE

create list with

with **fig** do **add_scatter** using

create list with

`x=np.arange(0,100)` ← x-axis: list of 1 to 100; 100 = # models in the boosted model

`y=gradientBoostingClassifier.train_score_` ← gets the training score of the gradient boosting classifier

`name="Training Deviance"` ← label

with **fig** do **add_scatter** using

create list with

`x=np.arange(0,100)` ← ""

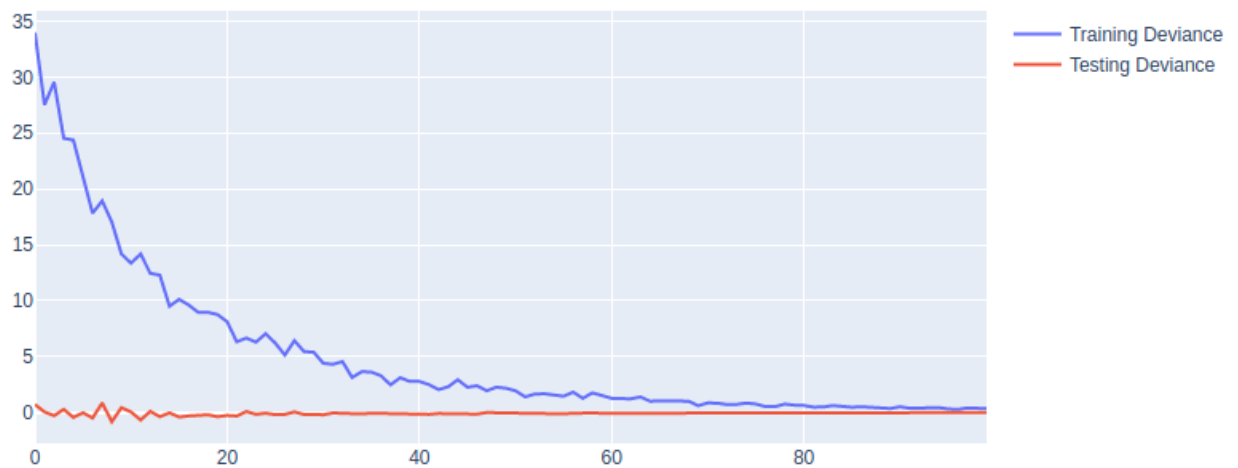
`y=gradientBoostingClassifier.oob_improvement_` ←

`name="Testing Deviance"` ← label

creates figure

figure is a line chart

Output:



Support Vector Machines

Prepare train/test sets

VARIABLES

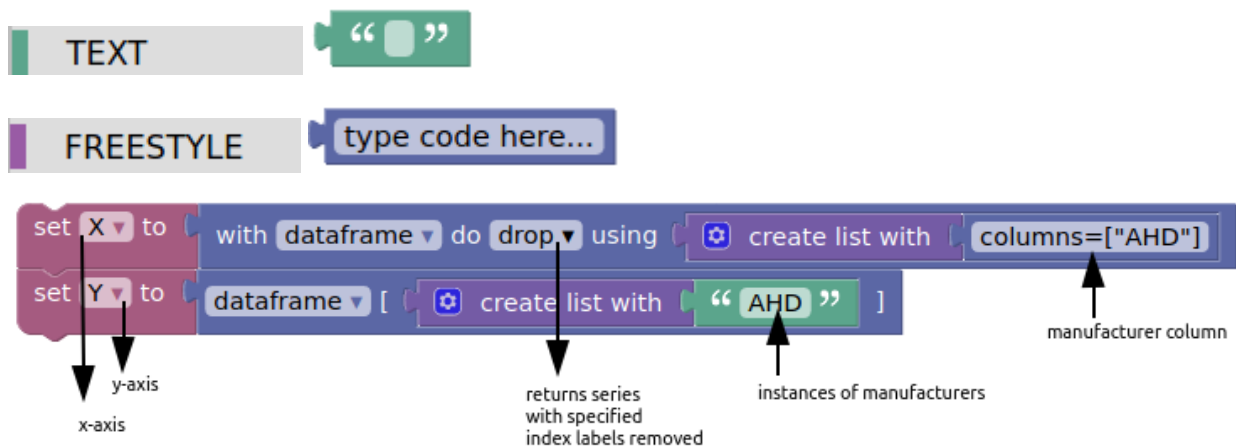
set **variable** to

with **variable** do !Not defined until you execute code. using

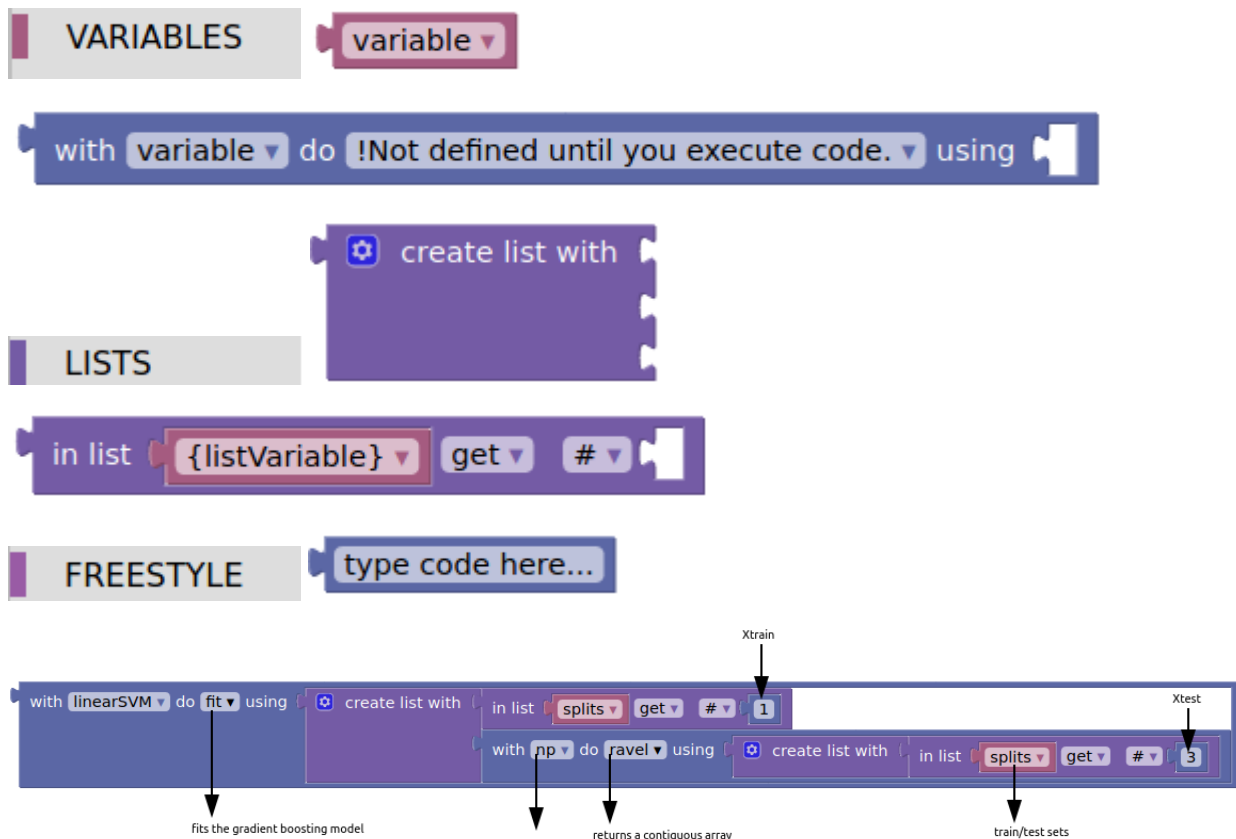
{dictVariable} []

create list with

LISTS



Fit the linear SVM model



Output:

```
Pipeline(memory=None,
          steps=[('standardscaler',
                  StandardScaler(copy=True, with_mean=True, with_std=True)),
                  ('linearsvc',
                  LinearSVC(C=1.0, class_weight=None, dual=True,
                             fit_intercept=True, intercept_scaling=1,
                             loss='squared_hinge', max_iter=1000,
                             multi_class='ovr', penalty='l2', random_state=1,
                             tol=0.0001, verbose=0))],
          verbose=False)
```

Predict using the linear SVM model

VARIABLES variable ▾

set variable ▾ to

with variable ▾ do !Not defined until you execute code. ▾ using

create list with

LISTS

in list {listVariable} ▾ get ▾ # ▾

FREESTYLE type code here...

set predictions ▾ to with linearSVM ▾ do predict ▾ using create list with in list splits ▾ get ▾ # ▾ 2

Ytrain

predicts output

train/test set

Evaluate the linear SVM model

VARIABLES variable ▾

with variable ▾ do !Not defined until you execute code. ▾ using

create list with

LISTS

in list {listVariable} ▾ get ▾ # ▾

FREESTYLE type code here...

print with metrics ▾ do classification_report ▾ using create list with in list splits ▾ get ▾ # ▾ 4

Ytest

sklearn.metrics

gets report of classification output performance measures

train/test set

predictions ▾

Output:

	precision	recall	f1-score	support
0	0.80	0.80	0.80	41
1	0.76	0.76	0.76	34
accuracy			0.79	75
macro avg	0.78	0.78	0.78	75
weighted avg	0.79	0.79	0.79	75

Fit the nonlinear SVM model

VARIABLES

variable

with variable do !Not defined until you execute code. using

LISTS

create list with

in list {listVariable} get #

FREESTYLE

type code here...

with nonlinearSVM do fit using

create list with

in list splits get # 1

with np do ravel using

create list with

in list splits get # 3

Xtrain

Xtest

fits the gradient boosting model

numpy

returns a contiguous array

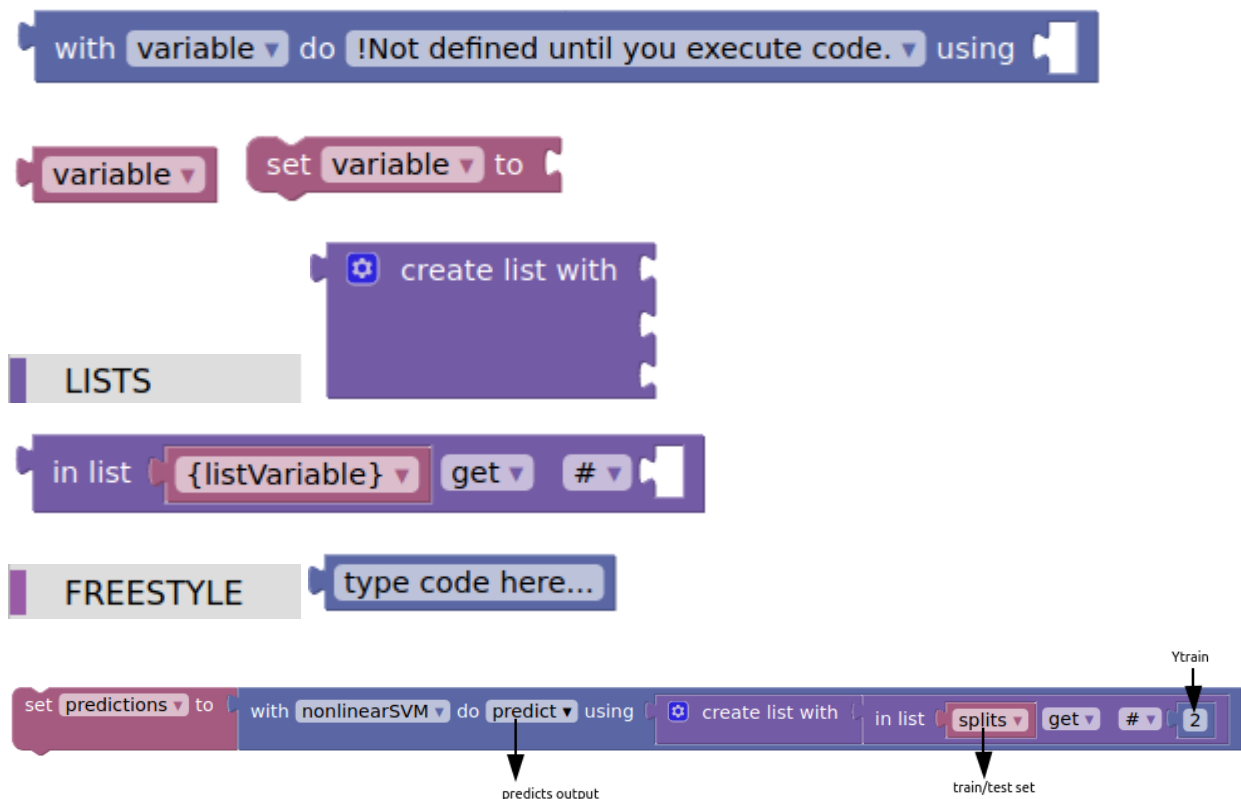
train/test sets

Output:

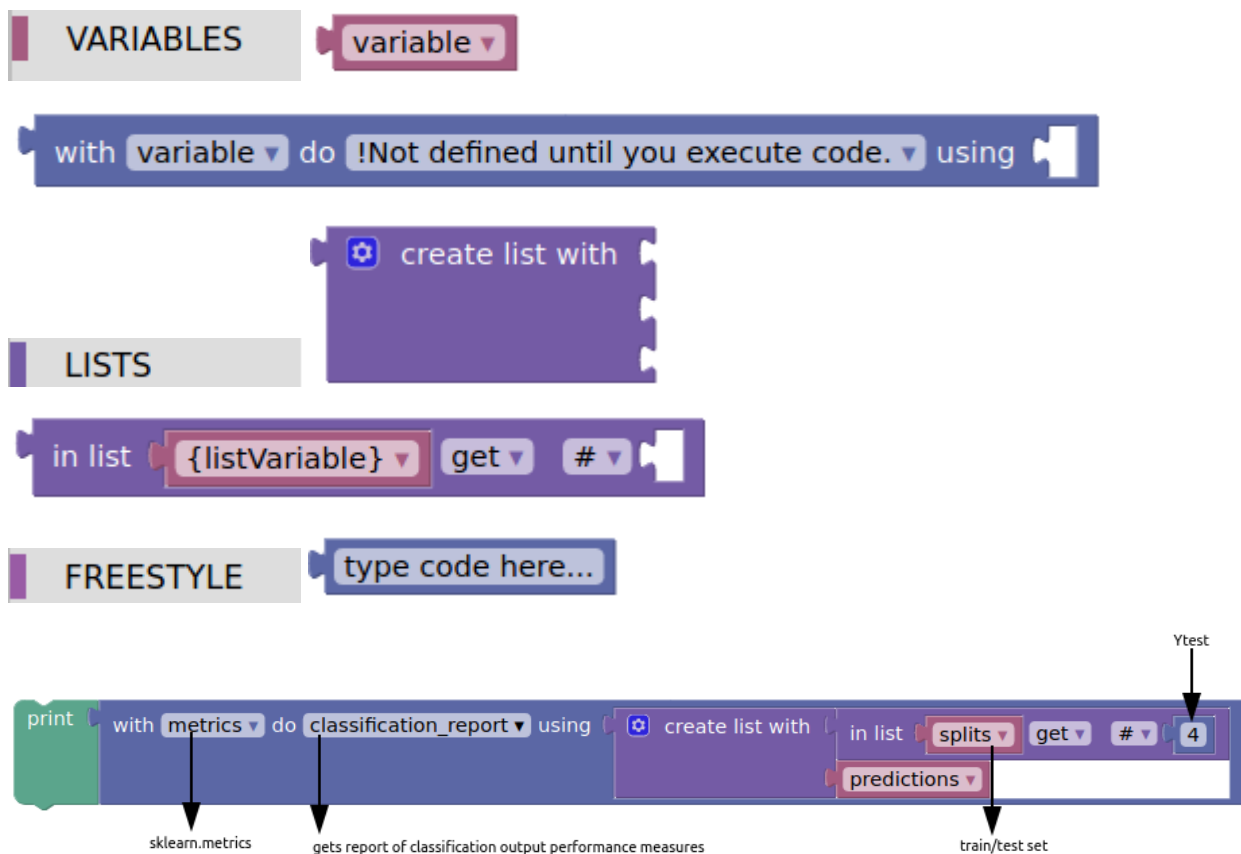
```
Pipeline(memory=None,
          steps=[('standardscaler',
                  StandardScaler(copy=True, with_mean=True, with_std=True)),
                  ('svc',
                   SVC(C=1.0, break_ties=False, cache_size=200, class_weight=None,
                       coef0=0.0, decision_function_shape='ovr', degree=3,
                       gamma='scale', kernel='rbf', max_iter=-1,
                       probability=False, random_state=1, shrinking=True,
                       tol=0.001, verbose=False))],
          verbose=False)
```

Predict with the nonlinear SVM model

VARIABLES



Evaluate the nonlinear SVM model



Output:

	precision	recall	f1-score	support
0	0.80	0.85	0.82	41
1	0.81	0.74	0.77	34
accuracy			0.80	75
macro avg	0.80	0.79	0.80	75
weighted avg	0.80	0.80	0.80	75

Naive Bayes

Prepare train/test sets

VARIABLES set variable ▼ to

with variable ▼ do !Not defined until you execute code. ▼ using

{dictVariable} ▼ []

LISTS create list with

TEXT “ ”

FREESTYLE type code here...

set X ▼ to with dataframe ▼ do drop ▼ using create list with columns=["rank"]

set Y ▼ to dataframe ▼ [create list with “rank ”]

predictors class label

returns series with specified index labels removed

instances of rank

rank column

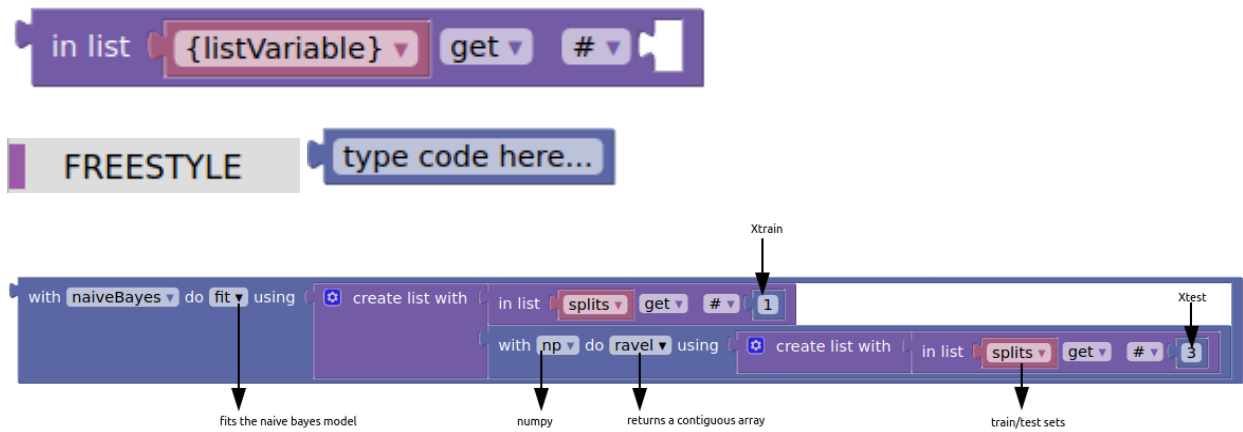
Fit the model

VARIABLES variable ▼

with variable ▼ do !Not defined until you execute code. ▼ using

create list with

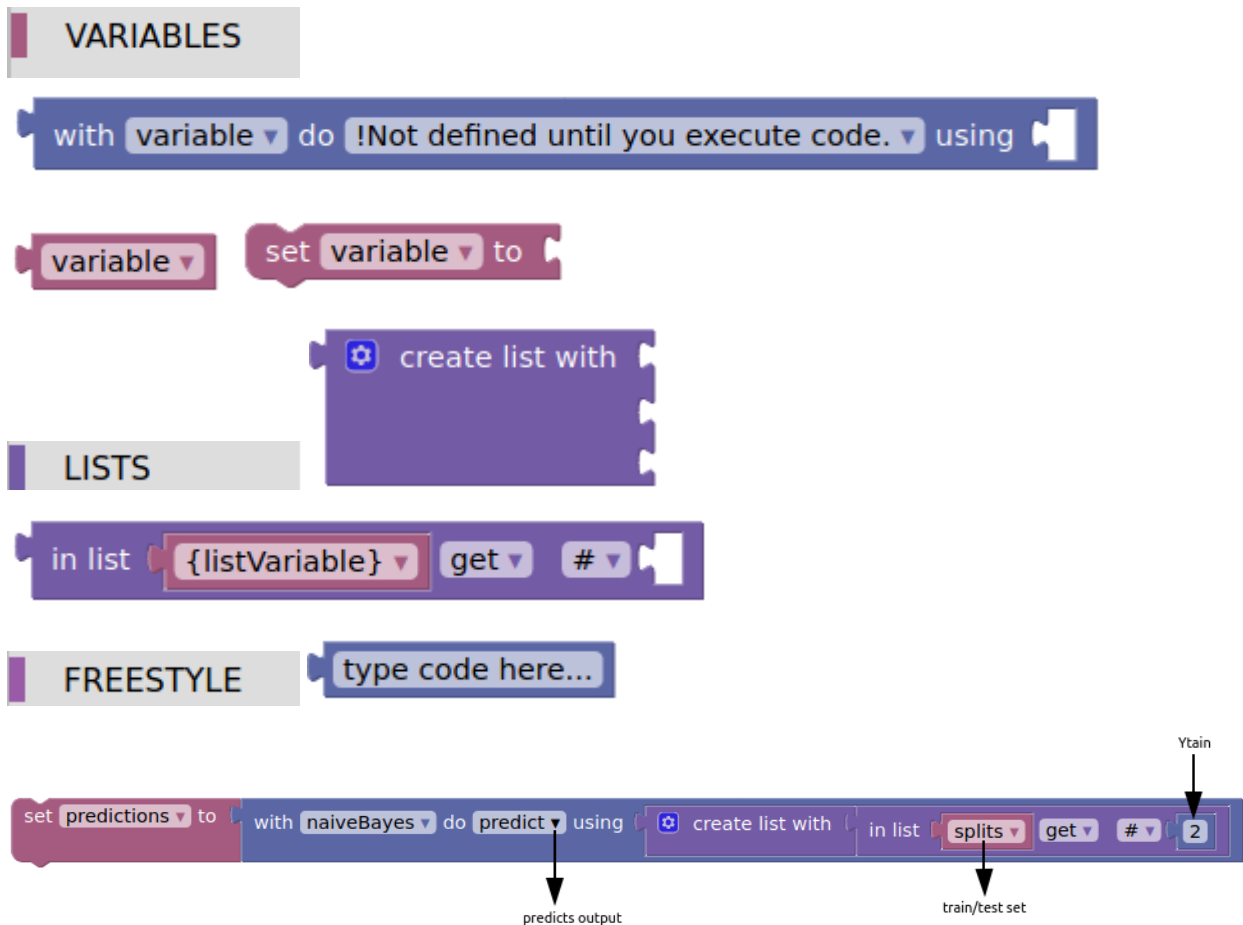
LISTS



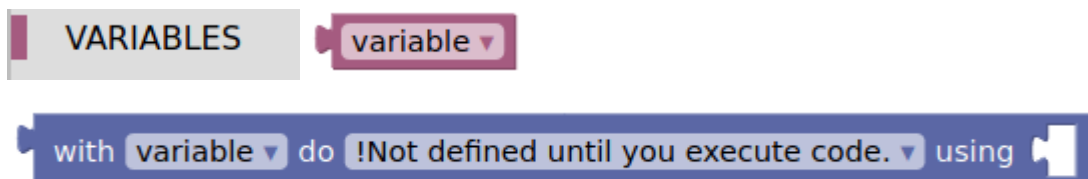
Output:

```
BernoulliNB(alpha=1.0, binarize=0.0, class_prior=None, fit_prior=True)
```

Get predictions



Evaluate the model



LISTS

create list with

in list {listVariable} get #

FREESTYLE type code here...

print with metrics do classification_report using create list with in list splits get # 4 predictions

sklearn.metrics get report of classification output performance measures train/test set

Ytest

Output:

	precision	recall	f1-score	support
not_recom	1.00	1.00	1.00	177
priority	0.79	0.89	0.84	155
spec_prior	0.90	0.84	0.87	153
very_recom	0.60	0.20	0.30	15
accuracy			0.89	500
macro avg	0.82	0.73	0.75	500
weighted avg	0.89	0.89	0.89	500

Crossvalidation

Prepare train/test sets

VARIABLES set variable to

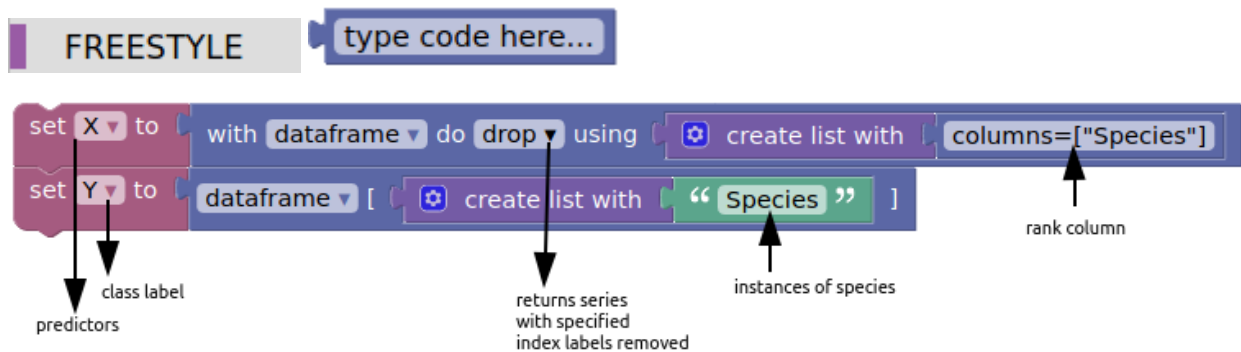
with variable do !Not defined until you execute code. using

{dictVariable} []

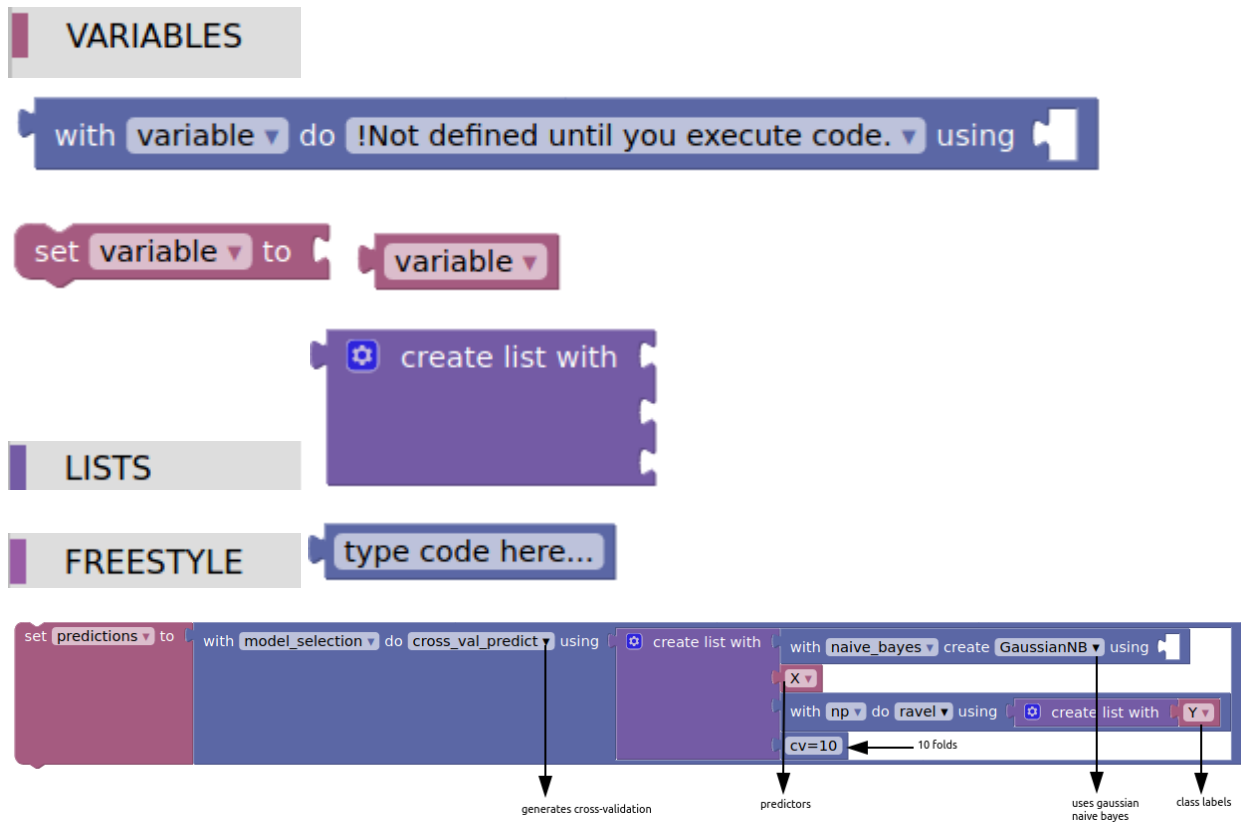
create list with

LISTS

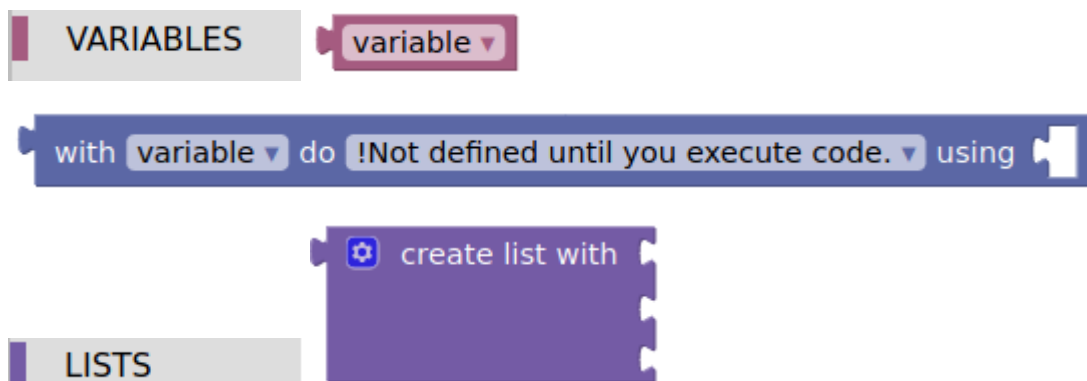
TEXT “ ”

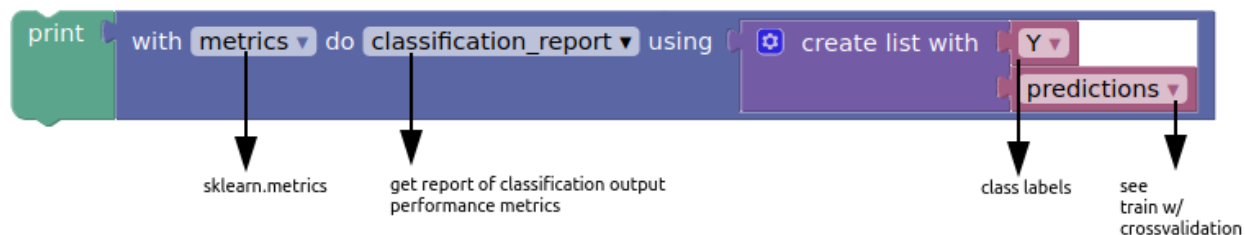


Train model & predict with crossvalidation



Evaluate the model

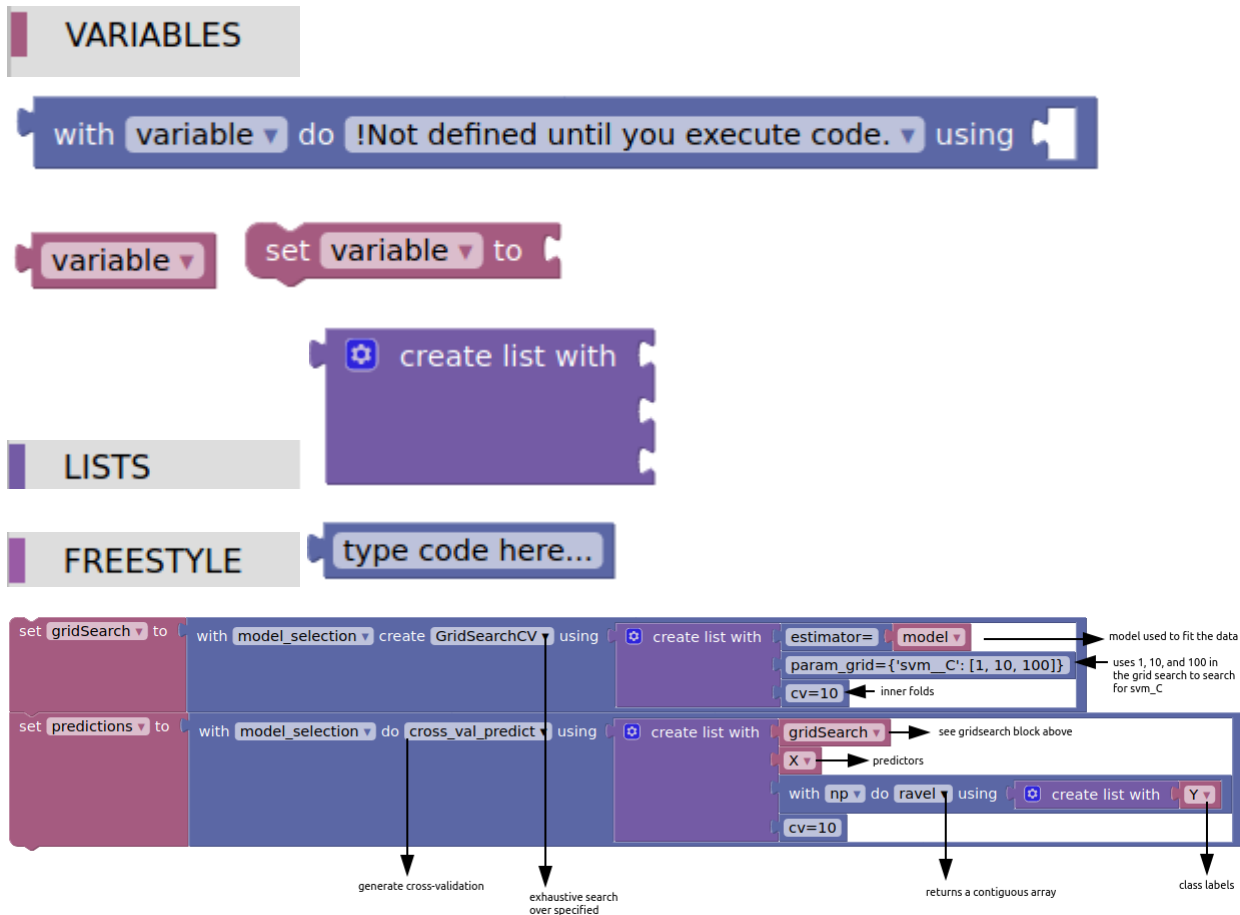




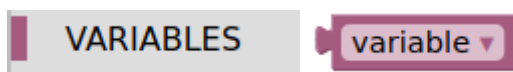
Output:

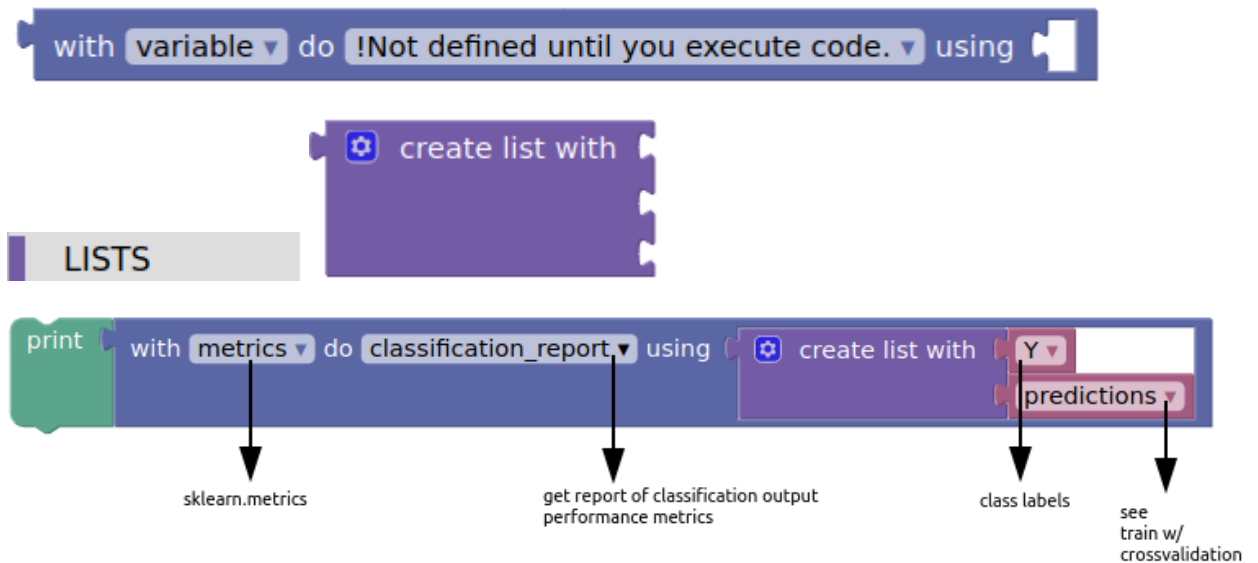
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	50
versicolor	0.92	0.94	0.93	50
virginica	0.94	0.92	0.93	50
accuracy			0.95	150
macro avg	0.95	0.95	0.95	150
weighted avg	0.95	0.95	0.95	150

Train model & predict with nested crossvalidation



Evaluate the model





Output:

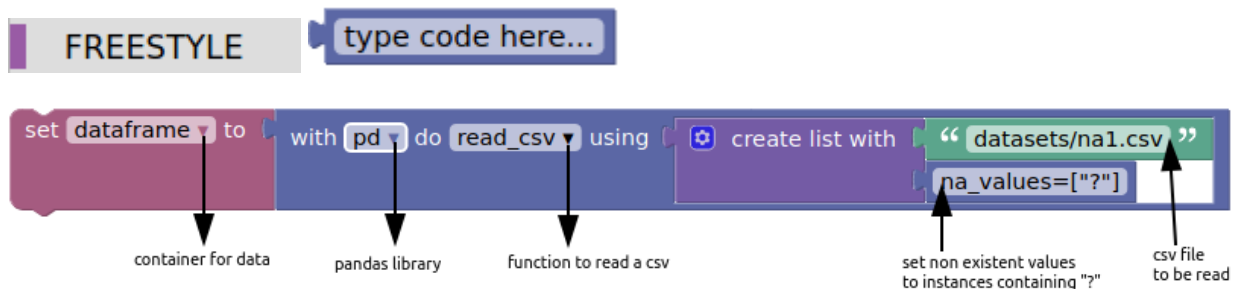
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	50
versicolor	0.94	0.92	0.93	50
virginica	0.92	0.94	0.93	50
accuracy			0.95	150
macro avg	0.95	0.95	0.95	150
weighted avg	0.95	0.95	0.95	150

Data cleaning, transformations, & versioning

Missing Data

After reading in a csv file "?" as unmarked values, convert those instances to NaN and either delete them from your set using `dataframe.dropna()` or fill them with median or mean values with `dataframe.fillna(dataframe.median())` or `dataframe.fillna(dataframe.mean())`. Conversion from "?" to NaN is seen below.





Before and after transformation:

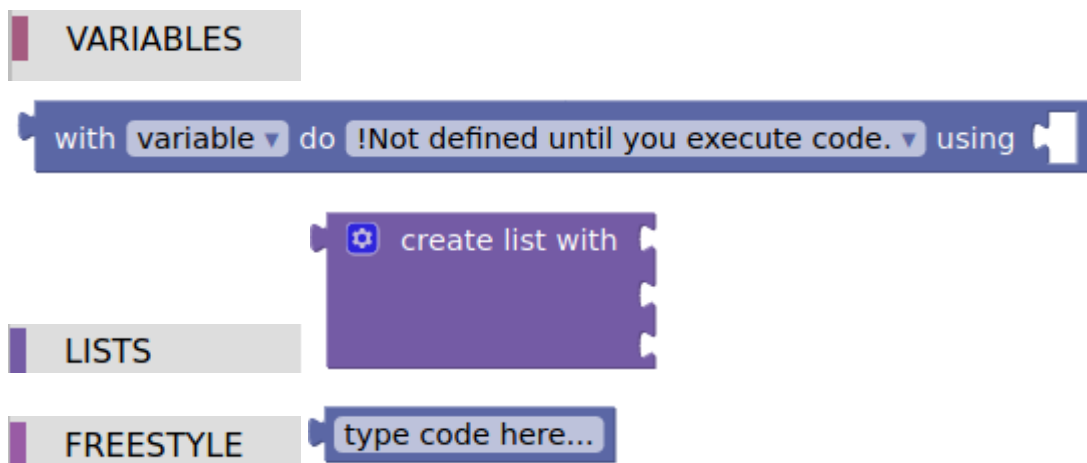
	x1	x2
0	1	2
1	?	3
2	3	2
3	5	?

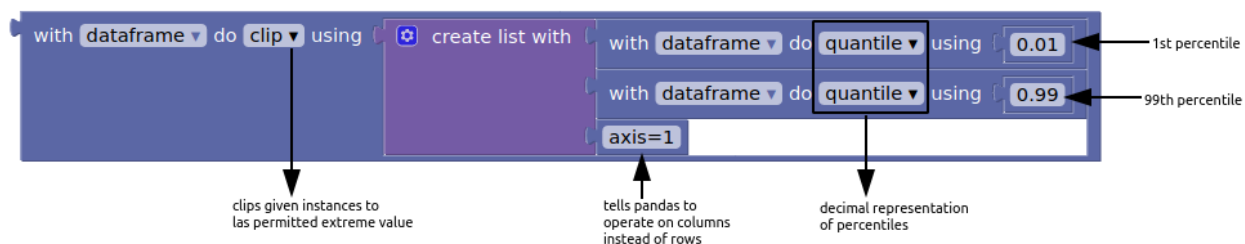
After:

	x1	x2
0	1.0	2.0
1	NaN	3.0
2	3.0	2.0
3	5.0	NaN

Transforming data

Outliers (values in the 1st and 99th percentile) can be converted to NaN and clipped as the last permitted extreme value.



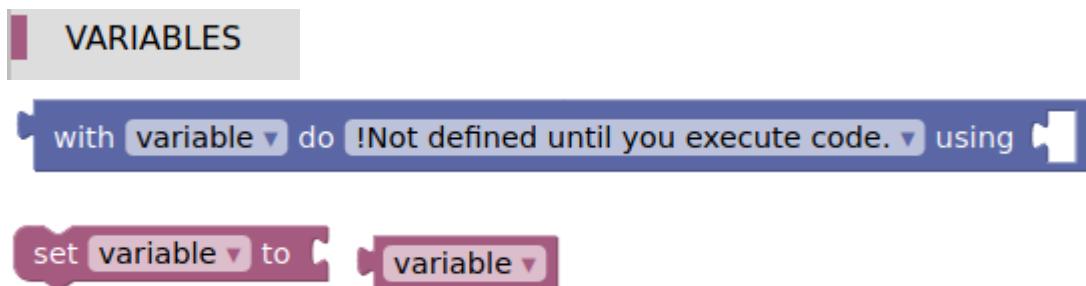


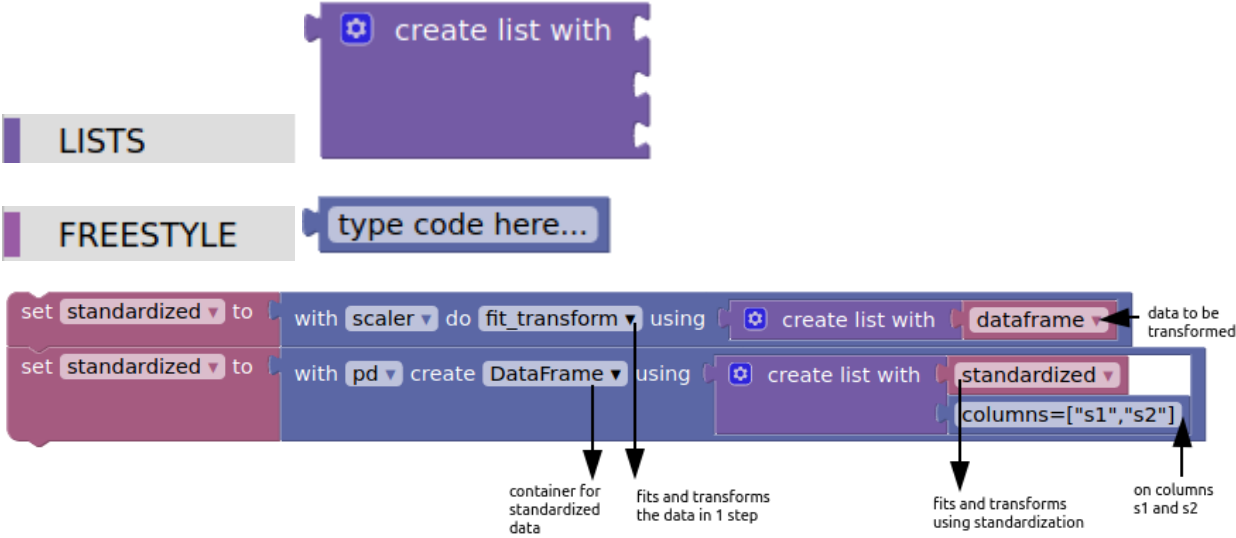
Before and after transformation:

	x1	x2		x1	x2
0	NaN	NaN	0	0.3558	0.2824
1	1.02	1.92	1	1.0200	1.9200
2	1.03	2.00	2	1.0300	2.0000
3	1.04	1.06	3	1.0400	1.0600
4	1.16	1.53	4	1.1600	1.5300
5	1.30	1.66	5	1.3000	1.6600
6	1.33	1.53	6	1.3300	1.5300
7	1.34	1.30	7	1.3400	1.3000
8	1.37	1.75	8	1.3700	1.7500
9	1.38	1.50	9	1.3800	1.5000
10	1.39	1.86	10	1.3900	1.8600
11	1.41	1.29	11	1.4100	1.2900
12	1.46	NaN	12	1.4600	5.2400
13	1.50	1.83	13	1.5000	1.8300
14	1.52	1.12	14	1.5200	1.1200
15	1.87	1.80	15	1.8700	1.8000
16	1.88	1.85	16	1.8800	1.8500
17	1.92	1.37	17	1.9200	1.3700
18	1.94	1.97	18	1.9400	1.9700
19	NaN	1.22	19	4.4186	1.2200

Common scale

Converts your data to a standardized scale (i.e. standard deviation of 1 unit).





Output:

	s1	s2
0	-1.538811	-1.528445
1	-0.606198	0.175027
2	-0.594825	0.249905
3	-0.583452	-0.629910
4	-0.446972	-0.190003
5	-0.287745	-0.068326
6	-0.253625	-0.190003
7	-0.242252	-0.405277
8	-0.208132	0.015912
9	-0.196759	-0.218082
10	-0.185385	0.118869
11	-0.162639	-0.414636
12	-0.105772	3.993801
13	-0.060279	0.090789
14	-0.037532	-0.573752
15	0.360534	0.062710
16	0.371908	0.109509
17	0.417401	-0.339759
18	0.440148	0.221826
19	3.920386	-0.480155

Non-normality

To get a normal distribution skew, try running a log transform.

VARIABLES

with **variable** do !Not defined until you execute code. using

LISTS

create list with

FREESTYLE

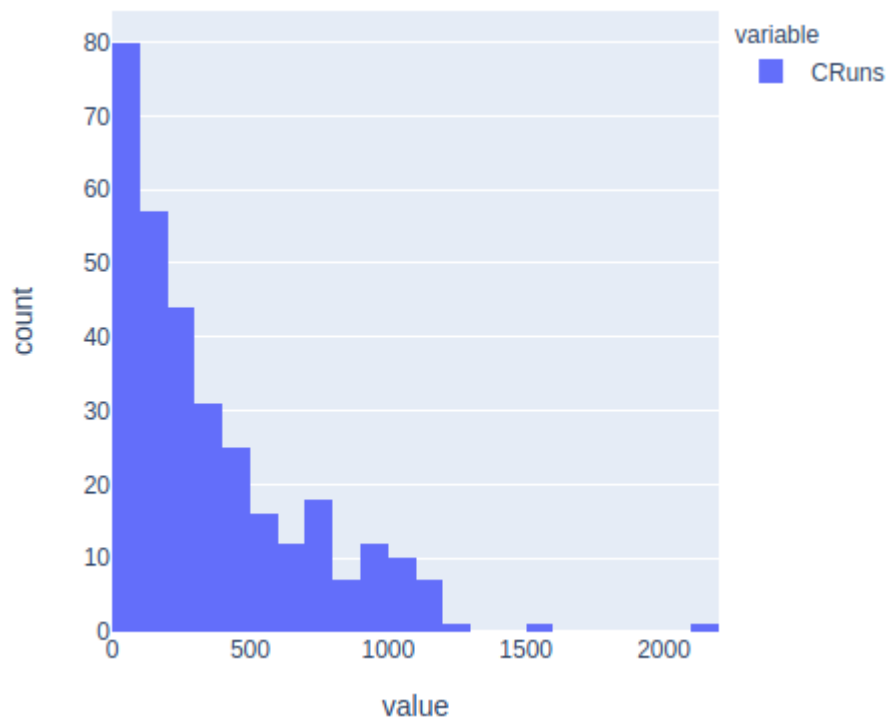
type code here...

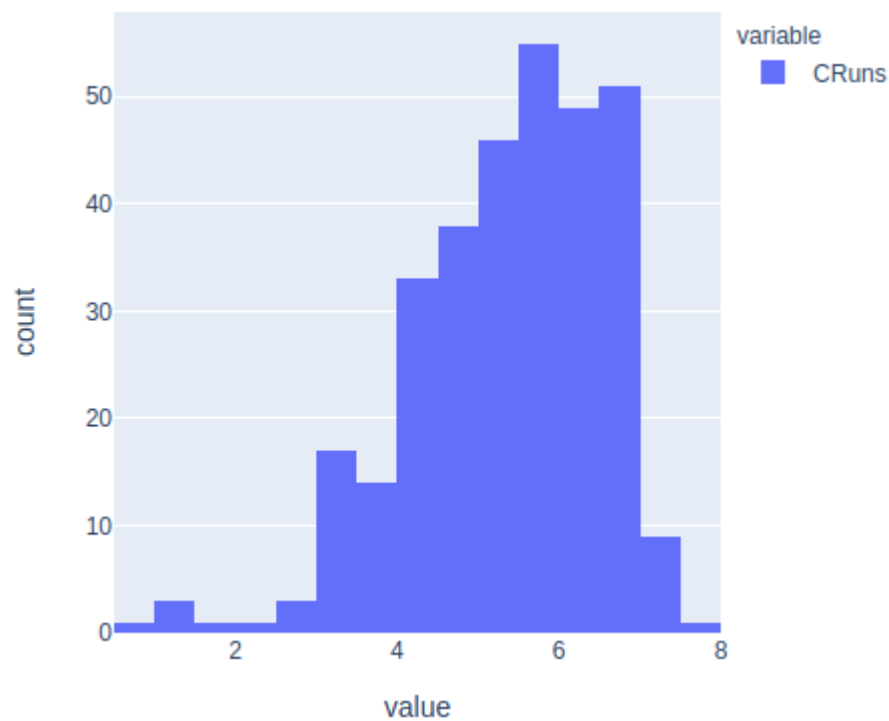
with **px** do **histogram** using create list with `np.log1p(dataframe["CRuns"])`

plots as a histogram

applies a log transform to the variable Cruns

Before and after transformation:





Create a new variable

Create a new variable as a column in your dataset

VARIABLES set variable ▼ to

with variable ▼ do !Not defined until you execute code. ▼ using

TEXT “ ”

FREESTYLE type code here...

set dataframe ▼ to with dataframe ▼ do assign ▼ using create list with ratio= dataframe ▼ [“tip”] ÷ dataframe ▼ [“total_bill”]

assigns new variable/column name of new variable tip column divided by total_bill column

Before and After transformation:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

	total_bill	tip	sex	smoker	day	time	size	ratio
0	16.99	1.01	Female	No	Sun	Dinner	2	0.059447
1	10.34	1.66	Male	No	Sun	Dinner	3	0.160542
2	21.01	3.50	Male	No	Sun	Dinner	3	0.166587
3	23.68	3.31	Male	No	Sun	Dinner	2	0.139780
4	24.59	3.61	Female	No	Sun	Dinner	4	0.146808
...	...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3	0.203927
240	27.18	2.00	Female	Yes	Sat	Dinner	2	0.073584
241	22.67	2.00	Male	Yes	Sat	Dinner	2	0.088222
242	17.82	1.75	Male	No	Sat	Dinner	2	0.098204
243	18.78	3.00	Female	No	Thur	Dinner	2	0.159744

244 rows × 8 columns